ICD122 · ICD125

Java

Object Oriented Programming

Complete Study Guide

From basics to advanced — step by step,
with examples, diagrams and real outputs

Class & Object | Encapsulation | Constructors | static | Inheritance | Polymorphism | Abstract & Interface | Exceptions | File IO | JDBC | Swing GUI | Collections

---

Based on the OOP Lecture Note (iCD122 / iCD125).
All Java code examples were compiled and run on Java 21.

Contents

Introduction — How to Use This Guide 5
About this guide 5
What the boxes mean 5
Learning roadmap 6
Prerequisites — PRF Recap 6
How to run a Java program 7

Part One — Foundations 9

Chapter 1 — Introduction to OOP: Class and Object 10
1.1 What is OOP? Why do we need it? 10
1.2 Class and Object 11
1.3 Primitive vs Reference — what happens in memory 12
1.4 Your first class — Student 13
1.5 Date and Box — Exercises (Q4 – Q7) 14
1.6 Important facts about references 16
Chapter 1 — Summary 17

Chapter 2 — Methods and the this keyword 19
2.1 Giving objects behavior — Methods 19
2.2 Methods with parameters 20
2.3 The this keyword 21
2.4 Methods with a return type 23
2.5 Extra: What happens when you pass an object to a method 25
Chapter 2 — Summary 25

Chapter 3 — Encapsulation (Protecting Data) 27
3.1 The problem: what if anyone can change the fields? 27
3.2 private fields and getters / setters 28
3.3 Adding validation to a setter 29
3.4 The Student class — a complete example 30
3.5 Access Modifiers 31
3.6 Benefits of Encapsulation 32
Chapter 3 — Summary 32

Chapter 4 — Constructors and Object Initialization 33
4.1 The initial values of an object 33
4.2 Types of constructors and cases 34
4.3 The order of things when an object is created 42
Chapter 4 — Summary 42

Chapter 5 — static Variables, Methods and Blocks 44
5.1 Instance variables vs static variables 44
5.2 static Methods 45
5.3 static Blocks 47
5.4 Real uses of static 48
Chapter 5 — Summary 49

Chapter 6 — Case Study: the Date Class Step by Step 50
6.1 Step I — Fields and a default constructor 50

6.2 Steps II & III — private fields, set(), a parameterized constructor, setters 51
6.3 Steps IV & V — Refactoring: removing duplication 51
6.4 Step VI — Constants and set(field, value) 52
6.5 Step VII — Copying another Date and today's date (a static factory) 53
6.6 Box — Exercise (Q56 – Q57) 54
Chapter 6 — Summary 55

Chapter 7 — String and Immutable Objects 57
7.1 A String is an object 57
7.2 String objects — Immutable 57
7.3 Our own Immutable class — Date 60
Chapter 7 — Summary 62

Chapter 8 — Object Reference Arrays 63
8.1 An array of objects 63
8.2 Student Array — a mini Student Management 64
Chapter 8 — Summary 66

Part Two — Inheritance and Polymorphism 68

Chapter 9 — Inheritance 69
9.1 Why do we need Inheritance? 69
9.2 extends — the simplest example 69
9.3 The subclass constructor and super() 71
9.4 Order with Static / Instance Blocks and Constructors in Inheritance 74
9.5 Reference type and Object type 75
9.6 Method Overriding 77
9.7 Q189 — Calling an overridden method inside a constructor (a trick question!) 79
Chapter 9 — Summary 80

Chapter 10 — Polymorphism 81
10.1 Compile-time Polymorphism — Overloading 81
10.2 Run-time Polymorphism — Overriding + Dynamic Method Dispatch 82
10.3 The Vehicle example — different actions through one interface 84
10.4 Polymorphism = Software Extensibility (Q197 – Q201) 86
10.5 Things to watch in Polymorphism questions 89
Chapter 10 — Summary 90

Chapter 11 — Abstract Classes and Interfaces EXTRA 91
11.1 What is Abstraction? 91
11.2 Abstract Class 91
11.3 Interface 93
11.4 Abstract class vs Interface 94
Chapter 11 — Summary 94

Chapter 12 — Collection Classes and class Object 96
12.1 Our own Collection class — IntegerCollection 96
12.2 StudentCollection — for Student objects 98
12.3 class Object — the root of all classes 100
12.4 The general Collection class — with Object[] 103
Chapter 12 — Summary 105

Part Three — Java Applications 106

Chapter 13 — Exception Handling 107
13.1 What is an Exception? 107

13.2 try – catch 107
13.3 Checked vs Unchecked Exceptions 108
13.4 The Exception hierarchy and our own Exception 110
Chapter 13 — Summary 111

Chapter 14 — Java IO: Reading and Writing Files 112
14.1 Why do we need files? 112
14.2 FileWriter — writing to a file 112
14.3 The File class 113
14.4 Reading a file 114
14.5 Student data in a file — CSV format 115
Chapter 14 — Summary 117

Chapter 15 — JDBC (Java Database Connectivity) 119
15.1 What is JDBC? 119
15.2 Setting up the database 119
15.3 Insert — adding data 120
15.4 Delete and Update 121
15.5 Select — reading data (ResultSet) 121
15.6 With OOP — a Student object into the DB 122
Chapter 15 — Summary 123

Chapter 16 — Java Swing: GUI Basics 124
16.1 What is a GUI? 124
16.2 Let's make a window — JFrame 124
16.3 Adding components 125
16.4 Layout Managers 126
16.5 JTextField, JLabel and JPanel 127
Chapter 16 — Summary 128

Chapter 17 — Swing + OOP: Forms, Inheritance and Events 129
17.1 Instead of building the GUI in Demo — extends JFrame 129
17.2 Forms — AddStudentForm, SearchStudentForm... 130
17.3 JTable and DefaultTableModel 131
17.4 Event Handling — making things happen when a button is clicked 132
17.5 The Main Form — opening several forms 133
Chapter 17 — Summary 134

Chapter 18 — Coursework: Student Marks Management System 135
18.1 The project 135
18.2 Student and StudentCollection (Q123 – Q126) 136
18.3 The Singleton Design Pattern 138
18.4 Steps to build the application (Roadmap) 140
Chapter 18 — Summary 140

Part Four — Advanced and Exam Preparation 142

Chapter 19 — Advanced OOP Topics EXTRA 143
19.1 Composition (HAS-A) vs Inheritance (IS-A) 143
19.2 instanceof and Casting 144
19.3 Overriding equals() and hashCode() 145
19.4 The final keyword 146
19.5 Generics and ArrayList (the Java Collections Framework) 146
19.6 enum 147
19.7 Packages and Imports 148

19.8 OOP Design Principles — SOLID (introduction) 148
19.9 UML Class Diagram — how to draw a class 149
Chapter 19 — Summary 149

Chapter 20 — Tricky Questions 150
T1 — An uninitialized reference (Q170) 150
T2 — A static counter (based on Q38 – Q40) 150
T3 — The constructor chain (this + super) 151
T4 — Overload resolution (Compile-time Polymorphism) 151
T4b — Recalling Q187: Overloading or Overriding? 152
T5 — String == 152
T6 — The Integer cache trick 152
T7 — A method that cannot swap (Pass by value) 153
T8 — Static methods are not overridden (Hiding) 153
T9 — Illegal reference assignment (Q181) 154
T10 — Static context (Q41) 154
T11 — Instance block + constructor + field initializer (Q36 – Q37) 155
T12 — An overridden method from a constructor (Q189 — recalled) 155
T13 — Quick True / False 156

Chapter 21 — Cheat Sheet (Quick Summary) 157
21.1 OOP concepts at a glance 157
21.2 Keywords 157
21.3 The Top 12 rules (for exams) 158
21.4 Access modifiers 158
21.5 Overloading vs Overriding 158
21.6 Abstract class vs Interface 159
21.7 Design Patterns (seen in this guide) 159
21.8 Good coding habits 159

Appendix — Answers to the Exercises (short) 160

Introduction — How to Use This Guide

About this guide

Your Lecture Note contains code examples from **Q1 to Q227**. They are good examples, but they are code *without explanations*. In this guide we take every topic and:

1. **Start from a simple idea** (why do we need this?)
2. **Build the code step by step**
3. Use **diagrams** to show what happens in memory
4. Show **real output** (run on Java 21)
5. Add **extra knowledge** beyond the lecture note

It is written so that someone who knows only PRF (Programming Fundamentals) and nothing about OOP can read it **from start to finish**.

> **Remember — Read alongside the Lecture Note**
>
> Every section starts with a tag like 📄 Lecture Note: Q23 – Q25 showing where that part lives in your note. The best method: run the code from the note, then read the explanation here.

What the boxes mean

Colored boxes appear throughout this guide:

> **Remember**
>
> The most important points. Great for exams too.

> **Watch out**
>
> Common mistakes, compile errors and runtime errors.

> **Extra knowledge**
>
> Things not in the lecture note that you still need for solid OOP knowledge.

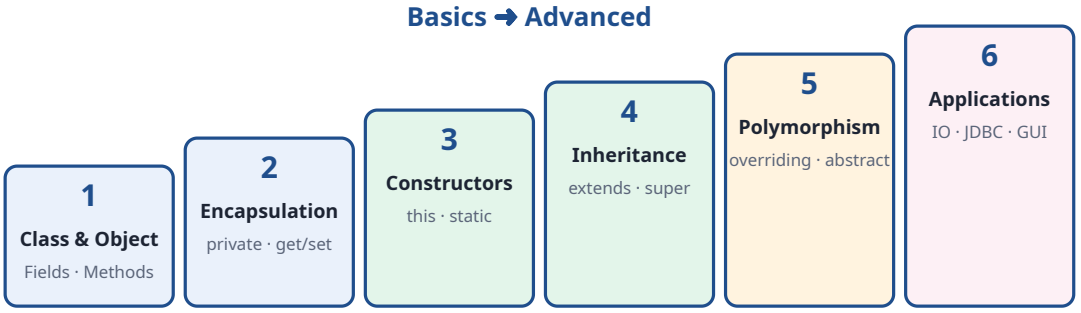
Definition

The precise meaning of a term or concept.

Try it yourself

Type the code on your own computer and run it. You cannot learn OOP just by reading!

Learning roadmap



Part	Chapters	What you learn
1. Foundations	1 – 8	Class, Object, Method, this, Encapsulation, Constructor, static, String, Arrays
2. Inheritance & Polymorphism	9 – 12	Inheritance, Polymorphism, Abstract, Interface, class Object, Collections
3. Java Applications	13 – 18	Exceptions, File IO, JDBC, Swing GUI, Student Marks System
4. Advanced & Exams	19 – 21	Composition, casting, equals/hashCode, Tricky Questions, Cheat sheet

Prerequisites — PRF Recap

Before learning OOP you should already know the following. If not, read this quick recap:

Concept	Java example	
Variable	`int age = 20;`	a place to hold data
Data types	`int` , `double` , `char` , `boolean` , `String`	the kind of data
Condition	`if (age > 18) {...} else {...}`	making decisions
Loop	`for` , `while` , `do-while`	repeating work
Array	`int[] ar = new int[3];`	several values of the same type
Method	`static int add(int a, int b){...}`	a reusable block of code

```java
class Demo {
    public static void main(String[] args) {
        int[] marks = {70, 85, 60};
        int total = 0;
        for (int i = 0; i < marks.length; i++) {
            total += marks[i];
        }
        System.out.println("Total = " + total);
        System.out.println("Average = " + (total / marks.length));
    }
}
```

OUTPUT

```
Total = 215
Average = 71
```

How to run a Java program

Running a Java file takes two steps:

```
javac Demo.java     <- 1) Compile (creates a byte code file named Demo.class)
java  Demo          <- 2) Run     (the JVM executes that byte code)
```

Every Java program starts at the `main` method:

```java
public static void main(String[] args) {
    // program execution starts here
}
```

> **Note — Don't worry about these words yet**
>
> We will learn the full meaning of `public` , `static` and `void` later (especially in **Chapters 3 and 5**). For now just remember that `main` is the door where the program starts.

Watch out — File name and class name

If a class is `public`, the file name must match the class name (`public class Student` → `Student.java`). In the lecture note the classes are not `public`, so the file name can be anything.

Part One — Foundations

Chapter 1 — Introduction to OOP: Class and Object

1.1 What is OOP? Why do we need it?

Look at the world around you. It is full of **things (objects)** — a student, a car, a bank account, a mobile phone. Each one has:

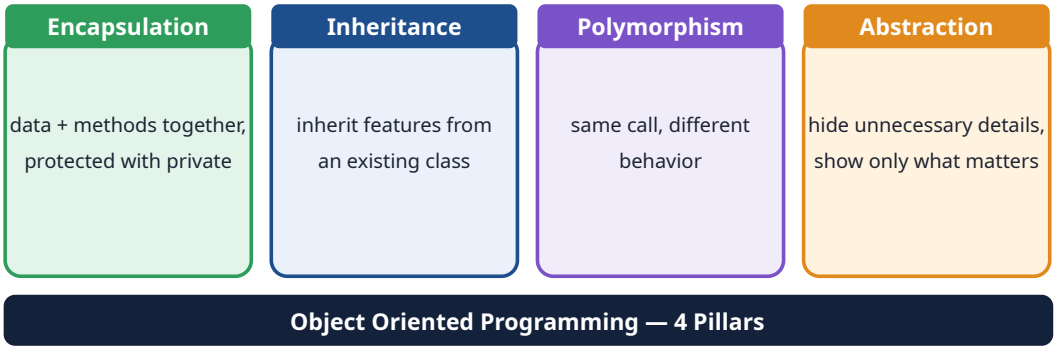
- **Attributes (data)** — a student has a name, an ID, marks.
- **Behavior (methods)** — a student can update marks, print details.

In PRF we wrote variables on one side and functions on another. That is fine for small programs. But in a program with the data of 500 students and 100 functions working on them, the link between data and functions gets lost and things become messy.

OOP (Object Oriented Programming) is a way of programming in which **data and the methods that work on that data are bundled together as objects**, just like modelling real-world things.

	Procedural (PRF)	Object Oriented (OOP)
Structure	Built around functions	Built around objects
Data	Separate from functions	Together with its methods
Protection	Anyone can change data	Data can be hidden and protected
Large programs	Hard to manage	Split into parts, easy to manage
Code reuse	Low	High, through Inheritance

The 4 pillars of OOP



We will study all four pillars in depth later. For now just know their names; each becomes clear in its own chapter.

1.2 Class and Object

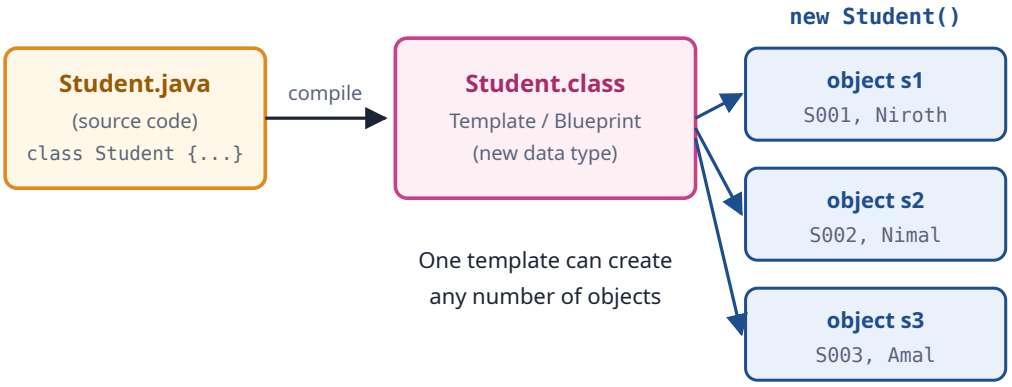
Definition — Class

A **class** is the **template / blueprint** that describes *how an object should look*. In it you write which attributes (fields) and methods an object will have.

Definition — Object

An **object** is the **real thing (instance)** built from a class. It is the object — not the class — that takes up memory.

Analogy: think of a house **plan** (blueprint) and the actual **houses**. From one plan you can build 100 houses. Each house has a different color and owner, but the structure is the same. The plan is the Class; the houses are Objects.



Class = Template. Object = the real thing built from that template.

	Class	Object
Nature	A concept / template	A real thing
Memory	Exists as code (Student.class)	Takes space in Heap memory
Count	Defined once	You can create any number
Example	`Student`	Nimal, Amal, Bimal (`s1` , `s2` , `s3`)

1.3 Primitive vs Reference — what happens in memory

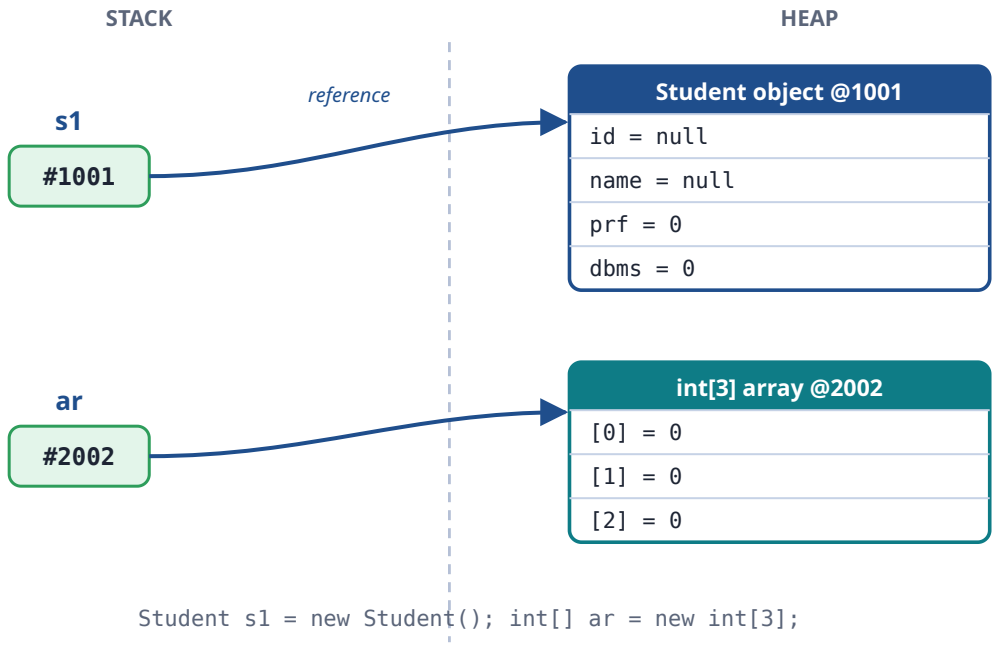
📄 Lecture Note: Q1 – Q3

Java has two kinds of variables:

1. **Primitive variables** — `int`, `double`, `char`, `boolean` and so on. The value is stored **inside the variable itself**.
2. **Reference variables** — `String`, arrays, and any class type you create. The variable does not hold the value; it holds the **address where the object lives**.

Two areas of memory matter:

- **Stack** — where local variables (primitive values and reference variables) live.
- **Heap** — where all **objects and arrays** live. `new` is the keyword that asks for space in the Heap.



The reference variable (Stack) holds only an address — the real object is on the Heap.

You already know arrays. In `int[] ar = new int[3];` the reference variable `ar` points to an array object in the Heap. **Exactly the same thing** happens with objects made from classes. The difference: all elements of an array have the same type, whereas an object's fields can have **different types** (`String`, `int`, ...).

> **Remember — Reference variable ≠ Object**
>
> `Student s1;` is **only a reference variable** (there is no object yet). `new Student()` is **what creates the object**. You can write both together as `Student s1 = new Student();`.

1.4 Your first class — Student

📄 Lecture Note: Q1 – Q3

Let's make a class with the things a student needs (id, name, PRF marks, DBMS marks) as fields.

```
class Student {
    String id;      // fields (attributes / instance variables)
    String name;
    int prf;
    int dbms;
}

class Demo {
    public static void main(String[] args) {
        Student s1;               // 1) declared a reference variable
        s1 = new Student();       // 2) created the object and stored its address in s1

        System.out.println("Default id   : " + s1.id);    // null
        System.out.println("Default prf  : " + s1.prf);   // 0

        s1.id = "S001";           // the dot (.) operator reaches the field
        s1.name = "Niroth";
        s1.prf = 45;
        s1.dbms = 65;

        System.out.println("St Id : " + s1.id);
        System.out.println("Name  : " + s1.name);
        System.out.println("PRF   : " + s1.prf);
        System.out.println("DBMS  : " + s1.dbms);
    }
}
```

OUTPUT

```
Default id   : null
Default prf  : 0
St Id : S001
Name  : Niroth
PRF   : 45
DBMS  : 65
```

What happens when you write `new Student()` (3 steps)

1. **Space in the Heap** is reserved for the object.
2. The fields get **default values**: `int → 0`, `double → 0.0`, `boolean → false`, `char → '\u0000'`, **reference (including String) →** `null`.
3. The **address (reference)** of that object is handed back, and we assign it to `s1`.

Remember — The dot operator (.)

`s1.name` means "the `name` field of the object `s1` points to". The two objects `s1` and `s2` each have their own `name` field.

Creating several objects

You can create many objects from one class. **Each object has its own set of fields**:

```
class Student {
    String id;
    String name;
}

class Demo {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.id = "S001";  s1.name = "Nimal";

        Student s2 = new Student();
        s2.id = "S002";  s2.name = "Amal";

        System.out.println(s1.name);   // Nimal
        System.out.println(s2.name);   // Amal
        s2.name = "Bimal";             // affects only s2
        System.out.println(s1.name + " " + s2.name);
    }
}
```

OUTPUT

```
Nimal
Amal
Nimal Bimal
```

Note — Several classes in one file

In Q2 of the lecture note, `Student.java` and `Demo.java` are two separate files. In Q3 both classes are in one file. Both are fine; `javac` produces a separate `.class` file for every class. You run the class that has the `main` method: `java Demo` .

1.5 Date and Box — Exercises (Q4 – Q7)

📄 Lecture Note: Q4 – Q7

In Q4 of the lecture note, `Demo` is written without the `Date` class. If we compile that:

```java
class Demo {
    public static void main(String[] args) {
        Date d1 = new Date();      // we have not created the Date class yet
        d1.year = 1999;
    }
}
```

COMPILE ERROR

```
Demo.java:3: error: cannot find symbol
        Date d1 = new Date();      // we have not created the Date class yet
        ^
  symbol:   class Date
  location: class Demo
Demo.java:3: error: cannot find symbol
        Date d1 = new Date();      // we have not created the Date class yet
                      ^
  symbol:   class Date
  location: class Demo
2 errors
```

The compiler is saying "I don't know the type `Date`". The fix: **create that class** (Q5):

```java
class Date {
    int year;
    int month;
    int day;
}

class Demo {
    public static void main(String[] args) {
        Date d1 = new Date();
        d1.year = 1999;
        d1.month = 12;
        d1.day = 31;
        System.out.println("Year : " + d1.year);
        System.out.println("Month: " + d1.month);
        System.out.println("Day  : " + d1.day);
    }
}
```

OUTPUT

```
Year : 1999
Month: 12
Day  : 31
```

Here we created **our own data type** (`Date`). Just like `int` and `String`, it is now a type. That is what the diagram in the lecture note means by "new data type (custom data type)".

Next, **Box** (Q6, Q7). Let's calculate the volume from length, width and height:

```java
class Box {
    int length;
    int width;
```

```
    int height;
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();
        b1.length = 12;
        b1.width = 5;
        b1.height = 3;

        int volume = b1.length * b1.width * b1.height;
        System.out.println("Volume of the box : " + volume);   // 180
    }
}
```

OUTPUT

```
Volume of the box : 180
```

> **Why is it like this? — What is wrong with calculating the volume in main?**
>
> The logic that calculates a Box's volume lives in `Demo`. If we use Box somewhere else, we have to write that formula again and again. In the next chapter we move that logic **inside Box** (as a method). That is the heart of OOP!

1.6 Important facts about references

(a) One object, two references

```
class Box { int length; }

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();
        b1.length = 10;

        Box b2 = b1;            // the object was NOT copied! only the reference was copied
        b2.length = 99;

        System.out.println(b1.length);   // changed through b2, but b1 sees it too
        System.out.println(b1 == b2);    // is it the same object?

        Box b3 = new Box();
        b3.length = 99;
        System.out.println(b1 == b3);    // values are equal, but they are different objects
    }
}
```

OUTPUT

```
99
true
false
```

When `==` is applied to reference variables it compares **addresses** (is it the same object?), not the values in the fields.

(b) The `null` reference

If a reference variable points to no object, its value is `null`. Trying to reach a field or method through `null` throws a **NullPointerException**:

```java
class Box { int length; }

class Demo {
    public static void main(String[] args) {
        Box b1 = null;
        System.out.println(b1.length);   // no object -> crash
    }
}
```

OUTPUT

```
Exception in thread "main" java.lang.NullPointerException: Cannot read field "length" because
"b1" is null
        at Demo.main(Demo.java:6)
```

> **Watch out — NullPointerException — the most common runtime error**
>
> You get this when you use `.` on a reference that was never given an object with `new`. When you read the error, the part "because **b1** is null" tells you which variable is null.

Chapter 1 — Summary

- **Class** = template; **Object** = the real thing made from it.
- Objects and arrays live in the **Heap**; reference variables live on the **Stack** (as an address).
- `new ClassName()` → space in the Heap + default values + the reference returned.
- Field defaults: `0`, `0.0`, `false`, `null`.
- `b2 = b1` is not a copy of the object; it is **two references to the same object**.
- Using `.` on a `null` reference = **NullPointerException**.

Try it yourself — Exercises (answers are in the Appendix at the end)

1.1 Create a `Book` class with `title` (String), `price` (double), `pages` (int). Make two objects, set values and print them.

1.2 After `Student s1 = new Student(); Student s2 = s1; s2.name = "Amal";` what is printed by `s1.name` ? Why?

1.3 Create a `Rectangle` class (`width` , `height`), calculate the area in main and print it.

Chapter 2 — Methods and the `this` keyword

2.1 Giving objects behavior — Methods

📄 Lecture Note: Q8 – Q9

In the last chapter we calculated the Box volume inside `Demo`. That was the wrong place. The volume should be calculated by **the Box itself** — it is the Box's responsibility. So we write a **method** inside the Box class.

```
class Box {
    //------ attribute declarations ------
    int length;
    int width;
    int height;

    //------ method declarations ------
    public void printVolume() {
        int volume;
        volume = length * width * height;
        System.out.println("Volume of the box : " + volume);
    }
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();
        b1.length = 12;
        b1.width = 5;
        b1.height = 3;

        b1.printVolume();      // calling the object's method
    }
}
```

OUTPUT

```
Volume of the box : 180
```

Parts of a method

```
public   void    printVolume   (  )   {  ...body...  }
  |       |          |          |
access  return    method     parameters
modifier  type     name       (if needed)
```

Part	Meaning
`public`	Access modifier — from where this method can be called (Chapter 3)
`void`	Return type — `void` = returns nothing
`printVolume`	Method name — normally **camelCase**, a name that describes an action
`( )`	Parameters — to receive data from outside, if needed

> **Remember — Why can we just write `length` inside the method?**
>
> When you call `b1.printVolume()`, the method runs **for the object `b1`**. So inside the method, `length` means `b1.length`. With `b2.printVolume()` it means `b2.length`. (This becomes even clearer in the `this` section below.)

2.2 Methods with parameters

📄 Lecture Note: Q10 – Q11

Instead of writing three lines `b1.length = 12; b1.width = 5; ...` to set the Box size, let's use **one method call**:

```
class Box {
    int length;
    int width;
    int height;

    public void printVolume() {
        int volume = length * width * height;
        System.out.println("Volume of the box : " + volume);
    }

    public void setSize(int l, int w, int h) {   // parameters: l, w, h
        length = l;
        width = w;
        height = h;
    }
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();
        b1.setSize(12, 5, 3);       // arguments: 12, 5, 3
        b1.printVolume();
    }
}
```

```
OUTPUT
Volume of the box : 180
```

> **Definition — Parameter vs Argument**
>
> A **parameter** is the variable in the method definition (`int l`). An **argument** is the actual value you give when calling (`12`). The number, types and order of arguments must match the parameters.

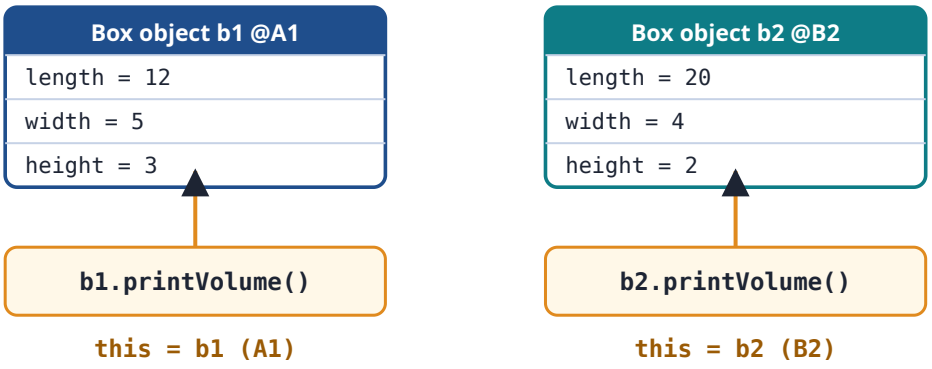
2.3 The `this` keyword

📄 Lecture Note: Q12 – Q14

What is `this`?

With two objects (`b1` , `b2`), the `printVolume()` method exists in **only one place** (the class). So how does the method know whose `length` to use?

The answer: Java secretly gives every instance method a reference called `this` . `this` = **the object that called the method just now**.



`this` is the object that called the method.

`this` = the reference to the object currently running the method.

```
class Box {
    int length;
    Box getThis() {
        return this;          // returns the same object that called this method
    }
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();
        Box b2 = new Box();
```

```
        System.out.println(b1.getThis() == b1);   // true
        System.out.println(b2.getThis() == b2);   // true
        System.out.println(b1.getThis() == b2);   // false
    }
}
```

OUTPUT

```
true
true
false
```

Note — `System.out.println(this)` in lecture note Q12

Its output is an address like `Box@1b6d3586`. The hash value can differ on every run, which is why I showed the `==` comparisons above. The key point: with `b1.print()` the `this` is `b1`, with `b2.print()` the `this` is `b2`.

The main use of `this` — when names are the same

What happens when a parameter and a field have the **same name**? (Q13)

```
class Box {
    int length;
    int width;
    int height;

    public void setSize(int length, int width, int height) {
        length = length;      // parameter = parameter !!!
        width = width;
        height = height;
    }
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();
        b1.setSize(12, 5, 3);
        System.out.println("Length : " + b1.length);
        System.out.println("Width  : " + b1.width);
        System.out.println("Height : " + b1.height);
    }
}
```

OUTPUT

```
Length : 0
Width  : 0
Height : 0
```

The output is `0 0 0`! There is no compile error, yet our idea did not work. The reason: inside the method, `length` means the **parameter** (a local variable hides the field — *shadowing*). `length = length` assigns the parameter to itself.

The fix: refer to the field explicitly as `this.length` (Q14):

```
class Box {
    int length;
    int width;
    int height;

    public void setSize(int length, int width, int height) {
        this.length = length;    // this.length = the field ;  length = the parameter
        this.width = width;
        this.height = height;
    }
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();
        b1.setSize(12, 5, 3);
        System.out.println(b1.length + " " + b1.width + " " + b1.height);
    }
}
```

OUTPUT

```
12 5 3
```

> **Remember — The rule**
>
> Giving parameters the same names as fields is a **good habit** (it reads well). But then **always use** `this.` to reach the fields.

2.4 Methods with a return type

📄 Lecture Note: Q15 – Q16

`printVolume()` **prints** the volume. But what if we need that value for another calculation (say, doubling it)? Then the method has to **give the value back** (`return`).

```
class Box {
    private int length;
    private int width;
    private int height;

    public void setSize(int length, int width, int height) {
        this.length = length;
        this.width = width;
        this.height = height;
    }

    public int getVolume() {            // returns an int value
        return length * width * height;
```

```
    }

    public void printVolume() {         // calling one method from inside another
        System.out.println("Volume of the box : " + getVolume());
    }
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();
        b1.setSize(12, 5, 3);
        b1.printVolume();                   // 180

        b1.setSize(120, 50, 30);
        int volume = b1.getVolume();        // the returned value goes into a variable
        System.out.println("New Volume : " + volume);
        System.out.println("Double     : " + (b1.getVolume() * 2));
    }
}
```

OUTPUT

```
Volume of the box : 180
New Volume : 180000
Double     : 360000
```

Remember — Good design: split the work

- `getVolume()` → **calculates** (calculation).
- `printVolume()` → **shows** (display), using `getVolume()` for the calculation.

If one method does both the calculation and the printing, that calculation cannot be used anywhere else. Let each method do **one job**.

Watch out — Return a value that matches the return type

Writing `return "abc";` in a method with return type `int` is a compile error. Writing `return 5;` in a `void` method is also an error.

2.5 Extra: What happens when you pass an object to a method

Extra knowledge — Pass by value — Java always sends a copy

- If you pass a **primitive** (`int`), a **copy of the value** goes. Changing it inside the method does not affect the outside variable.
- If you pass an **object**, a **copy of the reference** (the address) goes. So if the method changes the **object's fields**, the outside sees it. But if you **assign a different object** to the parameter, the outside reference is not affected.

```
class Box { int length; }

class Demo {
    static void changeInt(int x) { x = 100; }

    static void changeField(Box b) { b.length = 100; }       // modifies the object itself

    static void changeRef(Box b) { b = new Box(); b.length = 777; }  // points to a new
object; the outside reference is not affected

    public static void main(String[] args) {
        int a = 5;
        changeInt(a);
        System.out.println("a = " + a);

        Box b1 = new Box();
        b1.length = 1;
        changeField(b1);
        System.out.println("after changeField : " + b1.length);

        changeRef(b1);
        System.out.println("after changeRef   : " + b1.length);
    }
}
```

OUTPUT

```
a = 5
after changeField : 100
after changeRef   : 100
```

Chapter 2 — Summary

- A **method** = a behavior that belongs to an object.
- Call a method as `ref.method(args)` ; inside the method, fields can be used directly.
- `this` = the reference to the object that called the method.
- When a parameter and a field share a name → use `this.field = parameter` .

- `return` hands a value back; `void` = nothing.
- Java is **pass by value** — for objects, a copy of the reference goes.

> **Try it yourself — Exercises**
>
> **2.1** Add a `printDetails()` method to a `Student` class (printing id, name, marks).
>
> **2.2** Create a `Circle` class (with a `radius` field). Write `getArea()` (returns `double`) and `getCircumference()`. (Use `Math.PI`.)
>
> **2.3** Create a `Counter` class where `increment()` increases the `count` field by 1. Read the value with `getCount()`.

Chapter 3 — Encapsulation (Protecting Data)

3.1 The problem: what if anyone can change the fields?

In the previous chapters our fields (`length` , `width` , ...) had no access modifier. That means any class such as `Demo` can change them directly.

```
class Box {
    int length;
    int width;
    int height;

    public int getVolume() { return length * width * height; }
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();
        b1.length = -12;        // a negative length!! Box cannot prevent this
        b1.width = 5;
        b1.height = 3;
        System.out.println("Volume : " + b1.getVolume());
    }
}
```

OUTPUT

```
Volume : -180
```

A box cannot have a negative volume! The cause: the Box has **no control** over its own data. This is exactly what Encapsulation solves.

> **Definition — Encapsulation**
>
> **Encapsulation** = **bundling** the data (fields) and the methods that work on them inside one class, and **not letting outsiders change the data directly (`private`)** — allowing access only through public methods (getters / setters).

Analogy: an ATM machine. You cannot walk into the bank's database and change your balance. All you can do is send requests by pressing the buttons (public methods). The machine checks the rules (PIN, balance) and then carries out the request.

3.2 `private` fields and getters / setters

📄 Lecture Note: Q17 – Q18

Once fields are `private`, they can be accessed **only from inside that class**.

```
class Box {
    private int length;
    private int width;
    private int height;
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();
        b1.length = 120;      // cannot access a private field from outside
    }
}
```

```
COMPILE ERROR
Demo.java:10: error: length has private access in Box
        b1.length = 120;      // cannot access a private field from outside
          ^
1 error
```

So how do we set / read the values? Through **public methods**:

- **Setter** — sets a value: `setLength(int length)`
- **Getter** — returns a value: `int getLength()`

```
class Box {
    private int length;
    private int width;
    private int height;

    public void printVolume() {
        System.out.println("Volume of the box : " + getVolume());
    }
    public int getVolume() {
        return length * width * height;
    }

    public void setSize(int length, int width, int height) {
        setLength(length);        // this.length = length;
        setWidth(width);          // this.width = width;
        setHeight(height);        // this.height = height;
    }

    public void setLength(int length) { this.length = length; }
    public int  getLength()           { return length; }

    public void setWidth(int width)   { this.width = width; }
```

```
    public int  getWidth()            { return width; }

    public void setHeight(int height) { this.height = height; }
    public int  getHeight()           { return height; }
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();
        b1.setSize(12, 5, 3);
        b1.printVolume();                    // 180

        b1.setLength(120);
        System.out.println("Length : " + b1.getLength());
        b1.printVolume();
    }
}
```

OUTPUT

```
Volume of the box : 180
Length : 120
Volume of the box : 1800
```

Remember — Naming convention

If the field is `length` : setter → `setLength` , getter → `getLength` . For a `boolean` field the getter is written `isXxx()` (`isEmpty()` , `isActive()`).

Why is it like this? — Why getters/setters? Isn't a public field the same thing?

They look the same, but when we have a setter **we can add rules (validation) whenever we want**, without changing any outside code. With a public field we never get that chance.

3.3 Adding validation to a setter

📄 Solving the problem above — Extra

```
class Box {
    private int length = 1;

    public void setLength(int length) {
        if (length > 0) {
            this.length = length;
        } else {
            System.out.println("Invalid length : " + length + "  (must be greater than 0)");
        }
    }
    public int getLength() { return length; }
}
```

```java
class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();
        b1.setLength(-12);                // rejected
        System.out.println(b1.getLength());
        b1.setLength(12);                 // accept
        System.out.println(b1.getLength());
    }
}
```

OUTPUT

```
Invalid length : -12  (must be greater than 0)
1
12
```

Now the Box's `length` **can never be negative.** That responsibility belongs to the Box itself. This is the real power of Encapsulation.

3.4 The Student class — a complete example

📄 Lecture Note: Q19 – Q20

In Q19 of the lecture note the `Demo` is given and you must write the `Student` class. Here is how:

```java
class Student {
    private String id;
    private String name;
    private int prfMarks;
    private int dbmsMarks;

    public void setStudentDetails(String id, String name) {
        this.id = id;
        this.name = name;
    }
    public void setMarks(int prfMarks, int dbmsMarks) {
        this.prfMarks = prfMarks;
        this.dbmsMarks = dbmsMarks;
    }
    public void printStudentDetails() {
        System.out.println(id + "\t" + name + "\t" + prfMarks + "\t" + dbmsMarks);
    }

    public void setPrfMarks(int prfMarks)   { this.prfMarks = prfMarks; }
    public void setDbmsMarks(int dbmsMarks) { this.dbmsMarks = dbmsMarks; }

    public String getId()      { return id; }
    public String getName()    { return name; }
    public int getPrfMarks()   { return prfMarks; }
    public int getDbmsMarks()  { return dbmsMarks; }
    public int getTotal()      { return prfMarks + dbmsMarks; }
}
```

```java
class Demo {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.setStudentDetails("S0001", "Nimal");
        s1.setMarks(56, 70);
        s1.printStudentDetails();                // S0001 Nimal 56 70

        s1.setPrfMarks(89);
        s1.setDbmsMarks(90);
        s1.printStudentDetails();                // S0001 Nimal 89 90

        System.out.println("Student Id   : " + s1.getId());
        System.out.println("Student Name : " + s1.getName());
        System.out.println("PRF Marks    : " + s1.getPrfMarks());
        System.out.println("DBMS Marks   : " + s1.getDbmsMarks());
        System.out.println("Total Marks  : " + s1.getTotal());
    }
}
```

OUTPUT

```
S0001   Nimal   56      70
S0001   Nimal   89      90
Student Id   : S0001
Student Name : Nimal
PRF Marks    : 89
DBMS Marks   : 90
Total Marks  : 179
```

> **Note — About `getTotal()`**
>
> We did not keep a separate `total` field. It is just a method that **calculates** (derived) from the two marks. If we stored a field, forgetting to update the total when marks change would make the data inconsistent. Do not save values that can be calculated.

3.5 Access Modifiers

📄 Extra — needed for later parts of the lecture note

Java has 4 access levels:

Modifier	Same class	Same package	Subclass (other package)	Anywhere
`private`	✔	✘	✘	✘
(none — default)	✔	✔	✘	✘
`protected`	✔	✔	✔	✘
`public`	✔	✔	✔	✔

Remember — The basic rule

Fields → `private` **. Methods →** `public` (the services you offer to the outside). Helper methods needed only inside the class → `private` . We will look at `protected` again in the **Inheritance** chapter (9).

3.6 Benefits of Encapsulation

1. **Safety** — prevents wrong/invalid data from getting in (validation).
2. **Flexibility** — even if the class's internal implementation changes (for example `int` to `long`), outside code is not affected.
3. **Read-only fields** — give only a getter and no setter, and the field becomes **read-only** (for example an `id`).
4. **Maintainability** — when a bug appears you only need to look inside the class.

Extra knowledge — Do not return a mutable object from a getter

If `getXxx()` returns an array or an object (a reference), the outside receives **that very same object**, and can modify it. That is why `getAllStudent()` in the lecture note (Chapter 18) returns **a copy of the array**. Keep this in mind.

Chapter 3 — Summary

- **Encapsulation** = data `private` , access through `public` methods.
- **Setter** → sets a value (can include validation); **Getter** → returns a value.
- If a field is `private` , using `obj.field` from another class is a **compile error**.
- Access modifiers: `private` < default < `protected` < `public` .
- Do not keep calculable values (`total`) as fields; calculate them in a getter.

Try it yourself — Exercises

3.1 `BankAccount` class: `private double balance` . `deposit(double amt)` (amt must be > 0), `withdraw(double amt)` (must not exceed the balance), `getBalance()` .

3.2 Add `0 – 100` validation to the marks setters of `Student` .

3.3 Make a field `private` , access it from `Demo` , and explain the compile error you get.

Chapter 4 — Constructors and Object Initialization

4.1 The initial values of an object

📄 Lecture Note: Q21 – Q22

Right after `new Box()`, what are the values of `length`, `width` and `height` if we set nothing?

```
class Box {
    int length;
    int width;
    int height;
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();
        System.out.println("New Length : " + b1.length);
        System.out.println("New Width  : " + b1.width);
        System.out.println("New Height : " + b1.height);
    }
}
```

OUTPUT

```
New Length : 0
New Width  : 0
New Height : 0
```

The fields received **default values** (`0`). But often we want to create the object **with our own values** at the very moment it is born (for example `new Box(12, 5, 3)`). That job is done by the **Constructor**.

> **Definition — Constructor**
>
> A **constructor** is a special method that is **called automatically** when you use `new`. It: 1. Has the **same name as the class**. 2. Has **no return type** (not even `void`). 3. Runs **exactly once** when the object is created.

The constructor the compiler gives you

If **you have not written any constructor** in a class, the compiler automatically adds a **no-arg constructor** (the comment in Q22). That is why `new Box()` worked earlier.

```
class Box {
    int length;  int width;  int height;

    /* If you have not written any constructor, the compiler adds:
    Box() {
    }
    */
}
```

4.2 Types of constructors and cases

The lecture note divides Q23 – Q37 into 10 cases (Case I – X). Let's go through them one by one.

Case I & II — Default (no-arg) constructor

📄 Lecture Note: Q23 – Q24

If you write a constructor with no parameters yourself, you can give **every object the same initial values**:

```
class Box {
    int length;
    int width;
    int height;

    public Box() {                        // constructor - no return type, name = class name
        length = 12;
        width = 5;
        height = 3;
        System.out.println("Constructor... Box()");
    }
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();               // constructor is called
        Box b2 = new Box();               // called again
        System.out.println("b1 : " + b1.length + " " + b1.width + " " + b1.height);
        System.out.println("b2 : " + b2.length + " " + b2.width + " " + b2.height);
    }
}
```

OUTPUT

```
Constructor... Box()
Constructor... Box()
b1 : 12 5 3
b2 : 12 5 3
```

Every time you use `new`, the constructor runs once (two objects = printed twice).

Case III — Parameterized constructor

📄 Lecture Note: Q25

Give the constructor parameters so that each object gets different values:

```
class Box {
    int length;
    int width;
    int height;

    public Box(int length, int width, int height) {
        this.length = length;
        this.width = width;
        this.height = height;
        System.out.println("Box(int,int,int)");
    }
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box(12, 5, 3);
        Box b2 = new Box(1, 2, 1);
        Box b3 = new Box(120, 50, 30);
        System.out.println(b1.length + " " + b2.length + " " + b3.length);
    }
}
```

OUTPUT

```
Box(int,int,int)
Box(int,int,int)
Box(int,int,int)
12 1 120
```

Case IV — Once you write a constructor, the compiler does not give the default one

📄 Lecture Note: Q26

This is a **very important** rule (it comes up often in exams):

```java
class A {
    int a;
}
class B {
    int b;
    B(int i) { b = i; }
}
class Demo {
    public static void main(String[] args) {
        A a1 = new A();         // OK - A has no constructor, so the compiler gave a default
one
        B b2 = new B(100);      // OK
        B b1 = new B();         // ERROR! B has its own constructor, so there is no default
one
    }
}
```

COMPILE ERROR

```
Demo.java:12: error: constructor B in class B cannot be applied to given types;
        B b1 = new B();         // ERROR! B has its own constructor, so there is no default
one
               ^
  required: int
  found:    no arguments
  reason: actual and formal argument lists differ in length
1 error
```

Remember

- If there is **no constructor at all** → the compiler adds `ClassName() { }`.
- If you write **even one** → the compiler adds **nothing**. If you want `new B()`, you must write `B() { }` yourself.

Case V — Constructor Overloading

📄 Lecture Note: Q27

A class can have several constructors. If their **number / types / order of parameters** differ, they are separate constructors (this is called **overloading**). When you call `new`, the arguments decide which one is chosen.

```java
class Box {
    int length, width, height;

    public Box(int length, int width, int height) {
        this.length = length;
        this.width = width;
        this.height = height;
        System.out.println("Box(int,int,int)");
    }
```

```
    public Box(int length) {                    // a cube - all 3 sides are equal
        this.length = length;
        this.width = length;
        this.height = length;
        System.out.println("Box(int)");
    }
    public Box() {
        this.length = 1;
        this.width = 1;
        this.height = 1;
        System.out.println("Box()");
    }
    public void printVolume() {
        System.out.println("Volume of the box : " + (length * width * height));
    }
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box(12, 5, 3);
        Box b2 = new Box(10);
        Box b3 = new Box();
        b1.printVolume();
        b2.printVolume();
        b3.printVolume();
    }
}
```

OUTPUT

```
Box(int,int,int)
Box(int)
Box()
Volume of the box : 180
Volume of the box : 1000
Volume of the box : 1
```

Case VI & VII — Calling another constructor with `this(...)`

📄 Lecture Note: Q28 – Q30

In the code above we wrote `this.length = ...` three times — the same job again and again (**code duplication**). To reduce this we use `this(...)`. `this(args)` = "call **another constructor of this same class**".

```
class Box {
    int length, width, height;

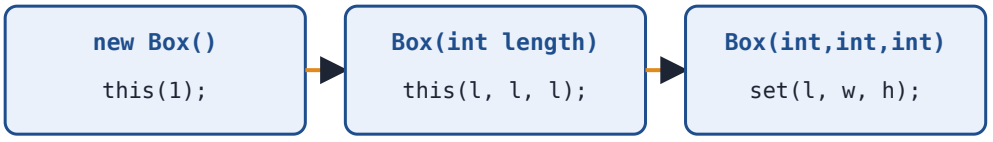
    public Box(int length, int width, int height) {
        set(length, width, height);                // the real work happens only here
        System.out.println("Box(int,int,int)");
    }
    public Box(int length) {
        this(length, length, length);              // → Box(int,int,int)
```

```
        System.out.println("Box(int)");
    }
    public Box() {
        this(1);                                    // → Box(int)
        System.out.println("Box()");
    }
    public void set(int length, int width, int height) {
        this.length = length;
        this.width = width;
        this.height = height;
    }
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();
    }
}
```

OUTPUT

```
Box(int,int,int)
Box(int)
Box()
```



① calls go forward → → → ② then work completes ← ← ← and prints / returns back

The last constructor's code runs first; the others finish as control returns.

Look at the output: we did `new Box()`, the **Box()** constructor, but **`Box(int,int,int)` printed first**. The reason: until the `this(1)` line finishes, execution does not move on to `System.out.println("Box()")`. The last constructor in the chain finishes first.

Watch out — `this(...)` must be the **first statement in the constructor**

```
class Box {
    int length;
    Box(int l) { length = l; }
    Box() {
        System.out.println("hi");    // not the first statement
        this(1);
    }
}
class Demo { public static void main(String[] a) { } }
```

COMPILE ERROR

```
Demo.java:6: error: call to this must be first statement in constructor
        this(1);
            ^
1 error
```

Case VIII — Looks like a constructor, but it's a method!

📄 Lecture Note: Q31 – Q33

A member with the same name as the class but **with a return type (`void`)** is a **method**, not a constructor. It compiles, but it is confusing (avoid it):

```
class Box {
    int length, width, height;

    public void Box(int length, int width, int height) {      // because of void, this is a METHOD
        System.out.println("Box(int,int,int) is a method");
    }
    public Box() {                                            // this is the real constructor
        System.out.println("Box()");
    }
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();         // constructor
        b1.Box(12, 5, 3);           // method call (legal)
    }
}
```

OUTPUT

```
Box()
Box(int,int,int) is a method
```

The exercise in Q32 – Q33:

```java
class A {
    A(int i) { }              // has a constructor -> no default constructor
}
class B {
    void B(int i) { }         // a method; no constructor -> the compiler gives the default
one
}
class Demo {
    public static void main(String[] args) {
        A a1 = new A();       // ERROR
        B b1 = new B();       // OK
    }
}
```

COMPILE ERROR

```
Demo.java:9: error: constructor A in class A cannot be applied to given types;
        A a1 = new A();       // ERROR
               ^
  required: int
  found:    no arguments
  reason: actual and formal argument lists differ in length
1 error
```

Case IX — Giving a value at the declaration

📄 Lecture Note: Q34 – Q35

You can give a value right where you declare the field (a **field initializer**). It runs before the constructor:

```java
class Box {
    int length = 12;
    int width = 5;
    int height = 3;

    Box() { }                                  // empty constructor
    Box(int l, int w, int h) {                 // overwrites the values
        length = l;
        width = w;
        height = h;
    }
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();
        System.out.println(b1.length + " " + b1.width + " " + b1.height);
        Box b2 = new Box(120, 50, 30);
        System.out.println(b2.length + " " + b2.width + " " + b2.height);
    }
}
```

```
OUTPUT
12 5 3
120 50 30
```

Case X — Instance Blocks

📄 Lecture Note: Q36 – Q37

A bare `{ ... }` code block written in the class is called an **instance block** (instance initializer).

When an object is created, it runs **before the constructor**, shared by every constructor.

```
class Box {
    int length, width, height;

    {                                          // instance block
        length = 12;
        width = 5;
        height = 3;
        System.out.println("Instance block...");
    }
    Box() {
        System.out.println("Box()");
    }
    Box(int l, int w, int h) {
        System.out.println("Box(int,int,int)");
    }
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();
        System.out.println(b1.length + " " + b1.width + " " + b1.height);
        System.out.println();
        Box b2 = new Box(100, 50, 30);
        System.out.println(b2.length + " " + b2.width + " " + b2.height);
    }
}
```

```
OUTPUT
Instance block...
Box()
12 5 3

Instance block...
Box(int,int,int)
12 5 3
```

Q37 — field initializers and instance blocks run **in the order they are written**:

```
class A {
    int a = 10;
    {
```

```
        a = 100;
        b = 200;
    }
    int b = 20;          // runs after the block: wipes b=200 and sets 20
}

class Demo {
    public static void main(String[] args) {
        A a1 = new A();
        System.out.println("a : " + a1.a);
        System.out.println("b : " + a1.b);
    }
}
```

OUTPUT

```
a : 100
b : 20
```

4.3 The order of things when an object is created

Step	What happens
1	Space in the Heap + **default values** (`0` , `null` ...) for the fields
2	**Field initializers** and **instance blocks** — in the order written
3	The **constructor** body (with `this(...)` , if present, first)

Exam tip — Exam Tip

For "What is the output?" questions follow the order **default → initializer/instance block → constructor**. And remember that with a `this(...)` chain, the body of the **last** constructor finishes **first**.

Chapter 4 — Summary

- Constructor = name same as the class, **no return type**, runs automatically on `new` .
- With no constructor the compiler gives a **default no-arg** one; **if you write any, it does not**.
- **Overloading** = several constructors with different parameters.
- `this(...)` = calling another constructor (must be the first statement).
- `void ClassName()` = a method, not a constructor.
- Initialization order: **default → field initializer / instance block → constructor**.

Try it yourself — Exercises

4.1 Write a constructor `(id, name)` and another `(id, name, prf, dbms)` for a `Student` class; call the first from the second using `this(...)`.

4.2 Before running, guess the output of this code, then run it: `class P { P(){ System.out.println("P"); } { System.out.println("block"); } int x = f(); int f(){ System.out.println("f"); return 1; } } — new P();`

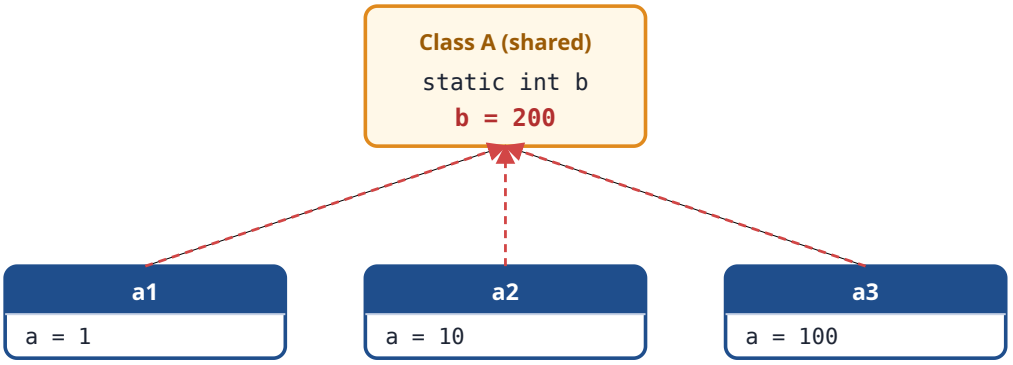
4.3 Make 3 overloaded constructors for an `Employee` class.

Chapter 5 — static Variables, Methods and Blocks

5.1 Instance variables vs static variables

📄 Lecture Note: Q38 – Q40

Every field we created before was an **instance variable** — one per object (each object has its own copy). If you give a field the keyword `static`, it **belongs to the class**; **a single copy** is **shared** by all objects.



a (instance) → one per object. b (static) → one per class, shared by all objects.

```
class A {
    int a;            // instance - one per object
    static int b;     // static   - one per class
}

class Demo {
    public static void main(String[] args) {
        A a1 = new A();  a1.a = 1;    a1.b = 2;
        A a2 = new A();  a2.a = 10;   a2.b = 20;
        A a3 = new A();  a3.a = 100;  a3.b = 200;

        System.out.println("a1 : " + a1.a + " " + a1.b);
        System.out.println("a2 : " + a2.a + " " + a2.b);
        System.out.println("a3 : " + a3.a + " " + a3.b);
    }
}
```

```
OUTPUT
a1 : 1 200
a2 : 10 200
a3 : 100 200
```

`a` is different for each object; `b` is the same for all (the last value set, 200). In Q40, what happens when the Box fields are made `static`?

```java
class Box {
    static int length;
    static int width;
    static int height;
    Box(int l, int w, int h) { length = l; width = w; height = h; }
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box(12, 5, 3);
        Box b2 = new Box(120, 50, 30);
        System.out.println("b1 : " + b1.length + " " + b1.width + " " + b1.height);
        System.out.println("b2 : " + b2.length + " " + b2.width + " " + b2.height);
    }
}
```

```
OUTPUT
b1 : 120 50 30
b2 : 120 50 30
```

Watch out — Never make an object's own data `static`

`b1`'s values were overwritten by `b2`. Data **specific** to an object (length, name) must never be `static`. Use `static` only for data **common to all** (for example constants, an object count).

Accessing a static variable: `ClassName.variable` (recommended) — `A.b`. Through an object (`a1.b`) also works, but it is confusing.

5.2 static Methods

📄 Lecture Note: Q41 – Q43

A static method can be called as `ClassName.method()` **without any object**.

```java
class MyClass {
    public static void staticMethod() {
        System.out.println("static Method");
    }
    public void instanceMethod() {
        System.out.println("instance Method");
```

```
    }
}

class Demo {
    public static void main(String[] args) {
        MyClass.staticMethod();            // no object needed
        // MyClass.instanceMethod();       // ERROR - an instance method needs an object
        MyClass c1 = new MyClass();
        c1.instanceMethod();
        c1.staticMethod();                 // legal (but not recommended)
    }
}
```

OUTPUT

```
static Method
instance Method
static Method
```

Q43 — if it is called through a `null` reference:

```
class MyClass {
    public static void staticMethod()  { System.out.println("static Method"); }
    public void instanceMethod()       { System.out.println("instance Method"); }
}

class Demo {
    public static void main(String[] args) {
        MyClass c1 = null;
        c1.staticMethod();       // OK! the object is irrelevant for a static call; the
class is used
        c1.instanceMethod();     // NullPointerException
    }
}
```

OUTPUT

```
static Method
Exception in thread "main" java.lang.NullPointerException: Cannot invoke
"MyClass.instanceMethod()" because "c1" is null
        at Demo.main(Demo.java:10)
```

Rules in a static context

```
class A {
    int x = 5;                    // instance
    static int y = 6;             // static

    static void sm() {
        System.out.println(y);    // OK
        System.out.println(x);    // ERROR - a static method cannot use an instance
variable
        System.out.println(this); // ERROR - no 'this' in a static method
    }
}
class Demo { public static void main(String[] a) { } }
```

```
COMPILE ERROR
Demo.java:7: error: non-static variable x cannot be referenced from a static context
        System.out.println(x);     // ERROR - a static method cannot use an instance variable
                           ^
Demo.java:8: error: non-static variable this cannot be referenced from a static context
        System.out.println(this);  // ERROR - no 'this' in a static method
                           ^
2 errors
```

	To static members	To instance members
From a **static method**	✔	✘ (only through an object)
From an **instance method**	✔	✔

> **Why is it like this? — Why is `main` `static`?**
>
> When the program starts, **there is no object yet**. The JVM cannot create an object just to call `main`, so `main` must be `static` — called directly through the class, like `Demo.main(args)`.
>
> That is also why `main` cannot call an instance method directly.

5.3 static Blocks

📄 Lecture Note: Q44

`static { ... }` — runs **only once, when the class is loaded into the JVM**. (An instance block runs **every time an object is created**.)

```
class MyClass {
    int a;
    static int b;
    static {
        System.out.println("static block...");
    }
```

```
    {
        System.out.println("instance block...");
    }
}

class Demo {
    public static void main(String[] args) {
        new MyClass();
        new MyClass();
        new MyClass();
    }
}
```

OUTPUT

```
static block...
instance block...
instance block...
instance block...
```

	Static block	Instance block
When it runs	When the class loads	Every time `new` is used
How many times	**Once**	As many as the objects
Used for	Initializing static data	Common initialization for all constructors

5.4 Real uses of static

1) Constants: `public static final` (a value that never changes, common to the class).

```
class Circle {
    public static final double PI = 3.14159;     // a single copy, cannot be changed
    double radius;
    Circle(double r) { radius = r; }
    double area() { return PI * radius * radius; }
}
class Demo {
    public static void main(String[] args) {
        System.out.println(new Circle(2).area());
        System.out.println(Circle.PI);
    }
}
```

OUTPUT

```
12.56636
3.14159
```

2) Counting objects: one counter shared by every object.

```java
class Student {
    private static int count = 0;
    private String name;
    Student(String name) { this.name = name; count++; }
    static int getCount() { return count; }
}
class Demo {
    public static void main(String[] args) {
        new Student("Nimal");
        new Student("Amal");
        new Student("Bimal");
        System.out.println("Students created : " + Student.getCount());
    }
}
```

OUTPUT

```
Students created : 3
```

3) Utility methods: `Math.abs()`, `Integer.parseInt()` — jobs done without an object.

4) Design patterns such as Singleton and Factory (Chapter 18).

Chapter 5 — Summary

- **static** = belongs to the class; **one copy** no matter how many objects.
- Call it as `ClassName.member`.
- A static method cannot use **instance members** or `this`.
- Why `main` is static: so the JVM can call it without an object.
- **Static block** → once when the class loads; **instance block** → every time an object is created.

Try it yourself — Exercises

5.1 In an `Employee` class create a static `companyName` common to all employees, and count the employees with `empCount`.

5.2 Create a `MathUtil` class: `static int max(int a, int b)`, `static boolean isEven(int n)`.

5.3 Try printing `static int x;` and `int y;` both from `static void m()`. What happens?

Chapter 6 — Case Study: the `Date` Class Step by Step

Let's combine everything we have learned (fields, methods, `this`, encapsulation, constructors, static) into **one complete class**. The `Date` class in Q45 – Q55 of the lecture note exists for exactly this. The goal: to make a Demo like this work.

```
Date d1 = new Date();          d1.printDate();       // 1970-1-1   (default date)
d1.set(1999, 12, 31);          d1.printDate();       // 1999-12-31
Date d2 = new Date(2020, 3, 14);                     // 2020-3-14
Date d3 = new Date();
d3.setYear(2026); d3.setMonth(9); d3.setDay(17);
System.out.println(d3);                              // 2026-9-17
```

6.1 Step I — Fields and a default constructor

📄 Lecture Note: Q46

```
class Date {
    int year;
    int month;
    int day;

    Date() {                        // default date = 1970-1-1
        year = 1970;
        month = 1;
        day = 1;
    }
    public void printDate() {
        System.out.println(year + "-" + month + "-" + day);
    }
}

class Demo {
    public static void main(String[] args) {
        Date d1 = new Date();
        d1.printDate();
    }
}
```

OUTPUT

```
1970-1-1
```

6.2 Steps II & III — `private` fields, `set()`, a parameterized constructor, setters

📄 Lecture Note: Q47 – Q48

We make the fields `private` and add `set(...)`, individual setters (`setYear`, ...) and a parameterized constructor. (There is duplication here — we remove it in the next step.)

```
private int year, month, day;
Date()                          { year = 1970; month = 1; day = 1; }
public Date(int y, int m, int d){ year = y; month = m; day = d; }      // duplication
public void set(int y, int m, int d){ year = y; month = m; day = d; }  // duplication
public void setYear(int year) { this.year = year; }
public void setMonth(int month){ this.month = month; }
public void setDay(int day)   { this.day = day; }
```

6.3 Steps IV & V — Refactoring: removing duplication

📄 Lecture Note: Q49 – Q50

We can see that the same job (assigning values) is written in more than 3 places. So:

- Default constructor → `this(1970, 1, 1)` (calls another constructor).
- Parameterized constructor → calls `set(year, month, day)`.
- `set(...)` → calls the individual setters.
- `printDate()` → uses `toString()`.

Now the real `this.year = year` line exists in **only one place** (`setYear`). If we later need validation, **changing one place** is enough!

```
class Date {
    private int year;
    private int month;
    private int day;

    Date() {
        this(1970, 1, 1);
    }
    public Date(int year, int month, int day) {
        set(year, month, day);
    }
    public void printDate() {
        System.out.println(toString());
    }
    public void set(int year, int month, int day) {
        setYear(year);
        setMonth(month);
```

```
        setDay(day);
    }
    public void setYear(int year)   { this.year = year; }
    public void setMonth(int month) { this.month = month; }
    public void setDay(int day)     { this.day = day; }

    public String toString() {
        return year + "-" + month + "-" + day;
    }
}

class Demo {
    public static void main(String[] args) {
        Date d1 = new Date();
        d1.printDate();                         // 1970-1-1
        d1.set(1999, 12, 31);
        d1.printDate();                         // 1999-12-31
        Date d2 = new Date(2020, 3, 14);
        d2.printDate();
        Date d3 = new Date();
        d3.setYear(2026);
        d3.setMonth(9);
        d3.setDay(17);
        System.out.println(d3.toString());
        System.out.println(d3);                 // toString() is called automatically
    }
}
```

```
OUTPUT
1970-1-1
1999-12-31
2020-3-14
2026-9-17
2026-9-17
```

Remember — Why does `System.out.println(d3)` work?

When you give `println` an object, Java automatically calls `d3.toString()`. If we had not written `toString()`, an address like `Date@1b6d3586` would be printed. More about `toString()` in Chapter 12.

6.4 Step VI — Constants and `set(field, value)`

📄 Lecture Note: Q51 – Q54

The lecture note has a way to tell **which field** to set with a constant, like `d1.set(Date.YEAR, 2026)`: three constants `public static final int YEAR=0, MONTH=1, DAY=2;` and a `switch`:

```
class Date {
    public static final int YEAR = 0;
```

```
    public static final int MONTH = 1;
    public static final int DAY = 2;

    private int year, month, day;

    Date() { this(1970, 1, 1); }
    public Date(int year, int month, int day) { set(year, month, day); }

    public void set(int field, int value) {            // overloaded set (2 params)
        switch (field) {
            case YEAR:  setYear(value);  break;
            case MONTH: setMonth(value); break;
            case DAY:   setDay(value);
        }
    }
    public void set(int year, int month, int day) {    // overloaded set (3 params)
        setYear(year); setMonth(month); setDay(day);
    }
    public void setYear(int year)   { this.year = year; }
    public void setMonth(int month) { this.month = month; }
    public void setDay(int day)     { this.day = day; }
    public String toString()        { return year + "-" + month + "-" + day; }
}

class Demo {
    public static void main(String[] args) {
        Date d1 = new Date();
        System.out.println(d1);
        d1.set(Date.YEAR, 2026);          // set(field, value)
        d1.set(Date.MONTH, 9);
        d1.set(Date.DAY, 17);
        System.out.println(d1);
    }
}
```

```
OUTPUT
1970-1-1
2026-9-17
```

Here the `set` method is **overloaded** (one with 2 parameters, one with 3). The compiler looks at the number of arguments and picks the right one.

6.5 Step VII — Copying another Date and today's date (a `static` factory)

📄 Lecture Note: Q55

- `set(Date date)` — copy the values from another `Date` object. (Because it is the same class, you can reach **private fields** directly, like `date.year`.)

- `static Date getDateInstance()` — a **static factory method** that gives a new `Date` object holding today's date (using Java's `java.time.LocalDate.now()`).

```
import java.time.*;

class Date {
    private int year, month, day;
    Date(int year, int month, int day) { this.year = year; this.month = month; this.day =
day; }

    public void set(Date date) {                        // copy
        this.year = date.year;
        this.month = date.month;
        this.day = date.day;
    }
    public static Date getDateInstance() {              // static factory
        LocalDate d1 = LocalDate.now();
        return new Date(d1.getYear(), d1.getMonthValue(), d1.getDayOfMonth());
    }
    public String toString() { return year + "-" + month + "-" + day; }
}

class Demo {
    public static void main(String[] args) {
        Date today = Date.getDateInstance();             // called through the class, no
object needed
        Date copy = new Date(2000, 1, 1);
        copy.set(today);
        System.out.println("copy equals today's fields: " +
copy.toString().equals(today.toString()));
    }
}
```

OUTPUT

```
copy equals today's fields: true
```

Extra knowledge — Java's built-in `java.time.LocalDate`

In real programs we do not build our own `Date` class. We use `LocalDate`, `LocalTime`, `LocalDateTime` (the java.time package). They are **immutable** (Chapter 7) and include validation (for example, no February 30). The purpose of our `Date` is OOP practice.

6.6 Box — Exercise (Q56 – Q57)

📄 Lecture Note: Q56 – Q57

Three constructors for Box (`Box()`, `Box(int)`, `Box(int,int,int)`) through a `this(...)` chain, plus getters / setters:

```java
class Box {
    private int length, width, height;

    public Box()                      { this(1); }
    public Box(int length)            { this(length, length, length); }
    public Box(int length, int width, int height) { set(length, width, height); }

    public void printVolume() { System.out.println("Volume of the box : " + getVolume()); }
    public int getVolume()    { return length * width * height; }

    public void set(int length, int width, int height) {
        setLength(length); setWidth(width); setHeight(height);
    }
    public void setLength(int length) { this.length = length; }
    public void setWidth(int width)   { this.width = width; }
    public void setHeight(int height) { this.height = height; }
    public int getLength() { return length; }
    public int getWidth()  { return width; }
    public int getHeight() { return height; }

    public String toString() { return "Box[" + length + "x" + width + "x" + height + "]"; }
}

class Demo {
    public static void main(String[] args) {
        Box b1 = new Box();
        Box b2 = new Box(5);
        Box b3 = new Box(12, 5, 3);
        System.out.println(b1 + " -> " + b1.getVolume());
        System.out.println(b2 + " -> " + b2.getVolume());
        System.out.println(b3 + " -> " + b3.getVolume());
    }
}
```

OUTPUT

```
Box[1x1x1] -> 1
Box[5x5x5] -> 125
Box[12x5x3] -> 180
```

Chapter 6 — Summary

- A complete class is developed **step by step**: fields → constructor → private + setters → refactor.
- **Refactoring** = cleaning up code without changing what it does; removing duplication.
- **Constants** → `public static final`. **`toString()`** → the text form of an object.
- **Static factory method** → a static method that creates objects.

Try it yourself — Exercises

6.1 Add an `isLeapYear()` method to `Date`.

6.2 Add `1 – 12` validation to `setMonth()` (print a message if wrong). Show that, since there is no duplication, only one place had to change.

6.3 Build a `Time` class (hour, minute, second) step by step, just like Date.

Chapter 7 — String and Immutable Objects

7.1 A String is an object

📄 Lecture Note: Q58

`String` is not a primitive type — it is an object of the `java.lang.String` **class**. That is why it has methods (`length()` , `charAt()` , ...).

```
class Demo {
    public static void main(String[] args) {
        String s1 = new String("abc");     // with new - the normal way to create an object
        String s2 = "abc";                 // literal - the short way (see the Extra below)

        System.out.println(s1.toString());   // abc
        System.out.println(s1.length());     // 3
        System.out.println(s1.charAt(1));    // b
    }
}
```

OUTPUT

```
abc
3
b
```

7.2 String objects — Immutable

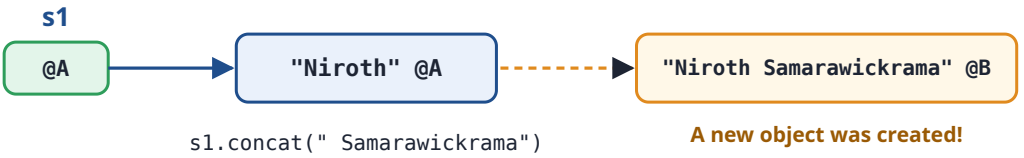
📄 Lecture Note: Q59 – Q61

Definition — Immutable Object

Immutable = an object whose **value can never be changed** once it is created. After you create a `String` , you cannot change its characters. Methods that look like they change it actually **create and return a new String object**.

```java
class Demo {
    public static void main(String[] args) {
        String s1 = new String("Niroth");
        System.out.println(s1);
        s1.concat(" Samarawickrama");       // a new String was created, but not assigned to anything!
        System.out.println(s1);             // still "Niroth"
    }
}
```

```
OUTPUT
Niroth
Niroth
```



s1 still points at @A → the old "Niroth". If the result is not assigned back to s1, it is lost.

The right way — **assign** the result:

```java
class Demo {
    public static void main(String[] args) {
        String s1 = new String(" Niroth ");
        System.out.println("length of s1 : " + s1.length());   // 8

        s1.trim();                                              // the result was lost
        System.out.println("length of s1 : " + s1.length());   // still 8

        String s2 = s1.trim();                                  // the new object goes to s2
        System.out.println("length of s2 : " + s2.length());   // 6

        String s3 = "Niroth";
        System.out.println(s3.toUpperCase());                   // NIROTH
        System.out.println(s3);                                 // Niroth (no change)
    }
}
```

```
OUTPUT
length of s1 : 8
length of s1 : 8
length of s2 : 6
NIROTH
Niroth
```

Watch out — Don't forget to assign the result of String methods

`s.trim()`, `s.toUpperCase()`, `s.replace(..)`, `s.concat(..)` — all of these return a **new String**. Only if you write `s = s.trim();` does `s` change.

Important String methods

Method	What it does	Example (`"Hello"`)
`length()`	number of characters	`5`
`charAt(i)`	the i-th character	`charAt(1)` → `e`
`substring(a,b)`	a part (b not included)	`substring(1,3)` → `el`
`toUpperCase()` / `toLowerCase()`	letter case	`HELLO`
`trim()`	removes leading/trailing spaces	
`equals(s)` / `equalsIgnoreCase(s)`	compares the **contents**	
`split(",")`	splits into parts	`String[]`
`indexOf(s)`	first position	`-1` if absent

Extra knowledge — `==` vs `equals()` and the String Pool

- `==` → **is it the same object** (compares references / addresses).
- `equals()` → **are the characters equal** (compares contents).

A **literal** such as `"abc"` is stored by Java in a **String Pool**, and literals with the same value **share the same object**. `new String("abc")` always creates a new object.

```java
class Demo {
    public static void main(String[] args) {
        String a = "abc";
        String b = "abc";
        String c = new String("abc");
        System.out.println(a == b);          // true  (the same object in the pool)
        System.out.println(a == c);          // false (c is a new object)
        System.out.println(a.equals(c));     // true  (the characters are equal)
    }
}
```

OUTPUT

```
true
false
true
```

> **Remember — The rule**
>
> When comparing Strings, **always use** `equals()`, never `==`.

7.3 Our own Immutable class — `Date`

📄 Lecture Note: Q62 – Q65

We can make our own classes immutable just like String. **The method:**

1. Fields `private` (and `final` if you like).
2. **No setters**; values are given only through the constructor.
3. The "changing" methods (`setYear`...) **return a new object** and never modify `this`.

The `A` class in Q64/Q65 shows the technique:

```
class A {
    private int a;
    private int b;

    A() { }
    A(int a, int b) { this.a = a; this.b = b; }

    public A setA(int a) { return new A(a, this.b); }     // a new object
    public A setB(int b) { return new A(this.a, b); }

    public String toString() { return a + "," + b; }
}

class Demo {
    public static void main(String[] args) {
        A a1 = new A();
        System.out.println(a1);              // 0,0
        A a2 = new A(10, 20);
        System.out.println(a2);              // 10,20
        a2.setA(100);                        // the result was not assigned
        System.out.println(a2);              // 10,20 - no change
        A a3 = a2.setA(100);
        System.out.println("a3 : " + a3);    // 100,20
        a3 = a2.setB(200);
        System.out.println("a3 : " + a3);    // 10,200
        System.out.println("a2 : " + a2);    // 10,20 - the original object never changed
    }
}
```

```
OUTPUT
0,0
10,20
10,20
a3 : 100,20
a3 : 10,200
a2 : 10,20
```

Immutable `Date` (Q63): every "setter" returns a new `Date`:

```
class Date {
    private final int year, month, day;
    Date()                                  { this(1970, 1, 1); }
    public Date(int year, int month, int day) { this.year = year; this.month = month;
this.day = day; }

    public Date set(int year, int month, int day) { return new Date(year, month, day); }
    public Date setYear(int year)   { return new Date(year, this.month, this.day); }
    public Date setMonth(int month) { return new Date(this.year, month, this.day); }
    public Date setDay(int day)     { return new Date(this.year, this.month, day); }

    public String toString() { return year + "-" + month + "-" + day; }
}

class Demo {
    public static void main(String[] args) {
        Date d1 = new Date();
        Date d2 = d1.setYear(2026).setMonth(9).setDay(17);      // chaining
        System.out.println(d1);        // 1970-1-1  (did not change)
        System.out.println(d2);        // 2026-9-17
    }
}
```

```
OUTPUT
1970-1-1
2026-9-17
```

Advantages of Immutable objects

1. **Safe** — wherever you share the object, nobody else can change it.
2. **Thread-safe** — no problems even when several threads use it at the same time.
3. Safe to use as a **hash key** (HashMap).
4. Disadvantage: every change creates a new object (memory/speed).

> **Extra knowledge — The `final` keyword**
>
> `private final int year;` → the value can be given **only once** (in the constructor); changing it later is a compile error.

Chapter 7 — Summary

- `String` = an object; its methods return a **new String** (immutable).
- **Assign** the result: `s = s.trim();`
- Compare Strings with `equals()`; `==` compares references.
- Immutable class: private final fields, no setters, "changing" = return a new object.

Try it yourself — Exercises

7.1 Trim the string `" Hello Java "`, convert it to upper case, and print only the `"JAVA"` part as a substring.

7.2 Make an immutable `Point` class (x, y), with `move(dx, dy)` returning a new Point.

7.3 `split(",")` the string `"Nimal,Amal,Bimal"` and print each name on its own line.

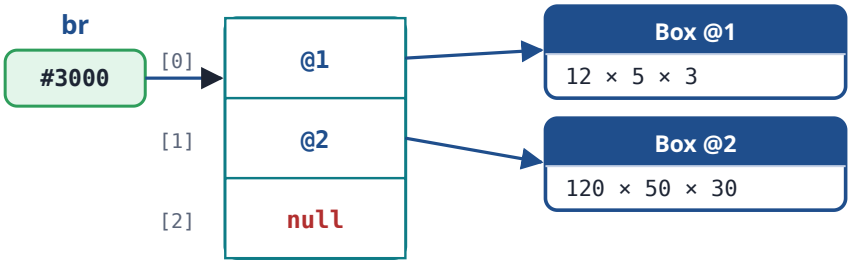
Chapter 8 — Object Reference Arrays

8.1 An array of objects

📄 Lecture Note: Q66 – Q69

`int[] ar = new int[3];` is an array holding 3 `int` values. In the same way you can make a `Box[]` array to hold **Box objects**. But the most important point is:

> **Watch out — `new Box[3]` does NOT create 3 Box objects!**
>
> What is created is **an array of 3 Box references**, and each reference is initially `null`. You must create the Box objects with `new Box(...)` and put them into the array one by one.



Box[] br = new Box[3]; — the array holds references, not Box objects!

```
class Box {
    private int length, width, height;
    public Box() { }
    public Box(int length, int width, int height) {
        this.length = length; this.width = width; this.height = height;
    }
    public void printVolume() {
        System.out.println("Volume of the box : " + (length * width * height));
    }
}

class Demo {
    public static void main(String[] args) {
        Box[] br = new Box[3];                      // 3 references, all null
        System.out.println(br[0] + " " + br[1] + " " + br[2]);

        br[0] = new Box(12, 5, 3);                  // create the object and put it in the
array
        br[1] = new Box(120, 50, 30);
        br[2] = new Box(1, 2, 3);

        for (int i = 0; i < br.length; i++) {
```

```
            br[i].printVolume();
        }
    }
}
```

```
OUTPUT
null null null
Volume of the box : 180
Volume of the box : 180000
Volume of the box : 6
```

> **Remember — The for-each loop**
>
> The **enhanced for** (for-each) loop is convenient for going through the whole array: `for (Box b : br) { b.printVolume(); }`. Use it when you do not need the index.

If you call `br[1].printVolume()` without giving `br[1]` an object, you get a **NullPointerException**. When looping, it is safer to add a `null` check: `if (br[i] != null) {...}`.

8.2 Student Array — a mini Student Management

📄 Lecture Note: Q70 – Q71

In Q70/Q71 an array collects students and performs tasks like **print, add (growing the array), best student**. An array has a **fixed size**; to add an element you must create a new (one larger) array and copy the old one into it. (The idea of this technique is in `extendsStudentArray()` in Q71.)

```
class Student {
    private String id;
    private String name;
    private int prfMarks, dbmsMarks;

    public Student(String id, String name, int prfMarks, int dbmsMarks) {
        this.id = id; this.name = name; this.prfMarks = prfMarks; this.dbmsMarks =
dbmsMarks;
    }
    public String toString() { return id + "\t" + name + "\t" + prfMarks + "\t" + dbmsMarks;
}
    public String getId()       { return id; }
    public int getPrfMarks()    { return prfMarks; }
}

class Demo {
    private static Student[] studentArray = {
        new Student("S0001", "Nimal", 56, 87),
        new Student("S0002", "Amal", 76, 97),
        new Student("S0003", "Bimal", 36, 47),
        new Student("S0004", "Ranil", 96, 97),
    };
```

```
    public static void printStudentDetails() {
        System.out.println("StId\tName\tPRF\tDBMS");
        System.out.println("============================");
        for (int i = 0; i < studentArray.length; i++) {
            System.out.println(studentArray[i]);       // toString() auto call
        }
    }

    public static void extendsStudentArray() {         // growing the array by 1
        Student[] temp = new Student[studentArray.length + 1];
        for (int i = 0; i < studentArray.length; i++) {
            temp[i] = studentArray[i];
        }
        studentArray = temp;
    }

    public static void addNewStudent(Student s) {
        extendsStudentArray();
        studentArray[studentArray.length - 1] = s;
    }

    public static String generateStudentId() {          // next id from the last id
        Student last = studentArray[studentArray.length - 1];
        int lastNo = Integer.parseInt(last.getId().substring(1));
        return String.format("S%04d", lastNo + 1);
    }

    public static void main(String[] args) {
        printStudentDetails();
        addNewStudent(new Student(generateStudentId(), "Kamal", 66, 70));
        System.out.println();
        printStudentDetails();
    }
}
```

OUTPUT

```
StId    Name    PRF     DBMS
============================
S0001   Nimal   56      87
S0002   Amal    76      97
S0003   Bimal   36      47
S0004   Ranil   96      97

StId    Name    PRF     DBMS
============================
S0001   Nimal   56      87
S0002   Amal    76      97
S0003   Bimal   36      47
S0004   Ranil   96      97
S0005   Kamal   66      70
```

> **Note — Why are the methods `static` here?**
>
> In Q70/Q71 all the methods are `static` in the `Demo` class — so that `main` (static) can call them without an object. In Chapter 18 we move them into a separate class called `StudentCollection`, which is a much better OOP design.

Sorting the array (Bubble sort — `bestInPrf` in Q71)

The `bestInPrf()` approach: make a **copy so the original array stays unchanged**, sort that copy in descending order of PRF marks, and print it.

```
Student[] temp = new Student[studentArray.length];           // copy
for (int i = 0; i < studentArray.length; i++) temp[i] = studentArray[i];

for (int i = 0; i < temp.length - 1; i++) {
    for (int j = 0; j < temp.length - 1; j++) {
        if (temp[j].getPrfMarks() < temp[j + 1].getPrfMarks()) {   // swap
            Student t = temp[j];  temp[j] = temp[j + 1];  temp[j + 1] = t;
        }
    }
}
```

> **Remember — When swapping objects**
>
> The array holds **references**, so only references are swapped — the objects stay in the Heap just as they were. That is very fast (no need to copy objects).

> **Extra knowledge — `ArrayList` — Java's built-in "growing array"**
>
> Growing an array by hand is tiresome. Java's `java.util.ArrayList<Student>` does it automatically (`add`, `remove`, `get`, `size`). In Chapter 12 we build **such a collection class ourselves**, and in Chapter 19 we look at using `ArrayList`.

Chapter 8 — Summary

- `Box[] br = new Box[3]` → **3 null references**; the objects must be created separately with `new`.
- The array holds references; calling a method on a `null` element throws NullPointerException.
- An array's size cannot change → create a new array + copy.
- When sorting, references are swapped.

Try it yourself — Exercises

8.1 Make a `Book[]` array (3 books) and calculate their total price.

8.2 Write a `searchStudent(String id)` method that **searches** the Student array by `id` (returns `null` if not found).

8.3 Write `removeStudent(String id)` (shrinking the array by 1).

Part Two — Inheritance and Polymorphism

Chapter 9 — Inheritance

9.1 Why do we need Inheritance?

📄 Lecture Note: Q164 – Q166

The lecture note lists 3 points: **Code Reusability**, **Easier Maintenance**, **Abstraction and Polymorphism**.

Imagine `Student`, `Lecturer` and `Admin` classes all need `name`, `address`, `phone` and `printContact()`. Will you write that code in **three places**? With Inheritance you write the common part **once in one class** (`Person`), and the other classes **inherit** from it.

> **Definition — Inheritance**
>
> **Inheritance** = taking the fields and methods of an existing class (the **Superclass / Parent / Base**) into a new class (the **Subclass / Child / Derived**) using the `extends` keyword. The relationship is **IS-A** (a Student **IS-A** Person).

Benefit	Description
Code Reusability	Common code is written in only one place.
Easier Maintenance	A bug fix / change in one place affects all subclasses.
Polymorphism	Subclass objects can be handled through a Superclass reference (Chapter 10).

9.2 `extends` — the simplest example

📄 Lecture Note: Q167 – Q172

Earlier in the lecture note's Swing code we already saw `class Calculator extends JFrame`. We do not need to write any code to make a window for Calculator — `setSize`, `setTitle`, `setVisible`... are all inherited from `JFrame` (Q167 – Q168).

Let's look at a simple example:

```
class A {
    int a;
    public void printA() {
        System.out.println("A : " + a);
```

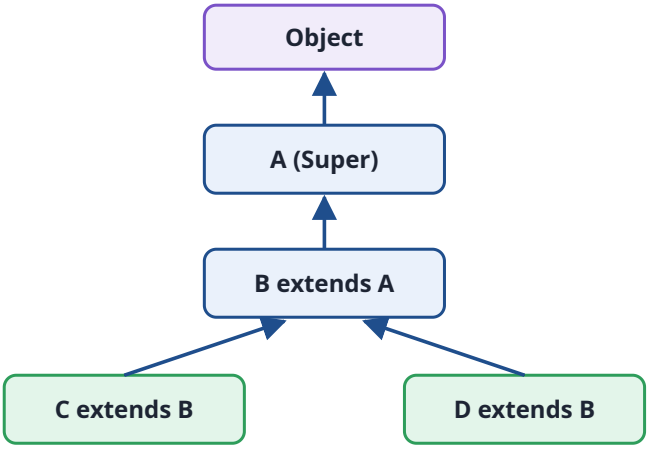
```
    }
}

class B extends A {          // B IS-A A
    int b;
    public void printB() {
        System.out.println("B : " + b);
    }
    public void printAB() {
        System.out.println("A : " + a);    // A's field was inherited -> can use it directly
        printA();                          // A's method too
        printB();
    }
}

class Demo {
    public static void main(String[] args) {
        B b1 = new B();
        b1.a = 100;        // inherited field
        b1.b = 200;        // its own field
        b1.printA();       // inherited method
        b1.printB();
        b1.printAB();
    }
}
```

OUTPUT

```
A : 100
B : 200
A : 100
A : 100
B : 200
```



Hollow arrow = "IS-A"

C IS-A B, B IS-A A

D IS-A B, D IS-A A

Class hierarchy: arrows point from Subclass → Superclass.

What is inside the object?

When you do `new B()`, the **single object** created in the Heap contains **both** the `A` part (`a`) and the `B` part (`b`). A separate Superclass object is not created.

Watch out — Q171: a field through `null`

In Q171, `A a1 = null; a1.a = 100;` → **NullPointerException**. If the reference does not point at an object, inheritance does not help.

Rules of Inheritance

- Java has **Single inheritance** — a class can `extends` **only one class** (`class C extends A, B` ✘). (Multiple "inheritance" through interfaces is in Chapter 11.)
- **Multilevel** — `C extends B`, `B extends A` (C inherits from A as well).
- **Hierarchical** — `B extends A`, `D extends A` (one parent, several children).
- **Private** members are inherited, but the subclass **cannot access them directly** (only through getters/setters).
- **Constructors** are **not inherited**.
- `static` members (Q172) can also be used through a subclass.

```java
class A {
    private int secret = 5;
    protected int shared = 6;
}
class B extends A {
    void show() {
        System.out.println(shared);     // OK  (protected)
        System.out.println(secret);     // ERROR – private
    }
}
class Demo { public static void main(String[] a) { } }
```

```
COMPILE ERROR
Demo.java:8: error: secret has private access in A
        System.out.println(secret);     // ERROR – private
                           ^
1 error
```

Remember — The `protected` modifier

`protected` members can be reached **from the same class, the same package, and subclasses** (see the table in Chapter 3). Use it for things meant only for subclasses.

9.3 The subclass constructor and `super()`

📄 Lecture Note: Q173 – Q179

When creating a subclass object, **the Superclass constructor must run first** (to initialize its part). As the **first statement** of a subclass constructor the compiler automatically adds `super();` (a call to the Superclass's no-arg constructor).

```
class A {
    int a;
    A()               { System.out.println("A()"); }
    A(int i)          { System.out.println("A(int)"); }
    A(int i, int j)   { System.out.println("A(int,int)"); }
}

class B extends A {
    int b;
    B() {
        // super();   <- added by the compiler
        System.out.println("B()");
    }
    B(int i) {
        // super();   <- added by the compiler (NOT A(int)!)
        System.out.println("B(int)");
    }
    B(int i, int j) {
        super(i, j);                        // we choose A(int,int) ourselves
        System.out.println("B(int,int)");
    }
}

class Demo {
    public static void main(String[] args) {
        new B();
        System.out.println("----");
        new B(10);
        System.out.println("----");
        new B(10, 20);
    }
}
```

OUTPUT

```
A()
B()
----
A()
B(int)
----
A(int,int)
B(int,int)
```

Remember — `super(...)`

- To call **another constructor** of the Superclass, write `super(args)` (Q175).
- It must be the **first statement** of the constructor (just like `this(...)`). So you cannot write both `this(...)` and `super(...)` in one constructor.
- `super.methodName()` / `super.field` = the **Superclass's** method / field (see 9.6 below).

Q176 – Q179: What if the Superclass has no no-arg constructor?

```
class A {
    A(int i) { System.out.println("A(int)"); }     // our own constructor -> no default
}
class B extends A {
    // the compiler adds B() { super(); }  ->  but there is no A()!
}
class Demo { public static void main(String[] a) { } }
```

COMPILE ERROR

```
Demo.java:4: error: constructor A in class A cannot be applied to given types;
class B extends A {
^
  required: int
  found:    no arguments
  reason: actual and formal argument lists differ in length
1 error
```

Two solutions (Q178, Q179):

```
class A {
    A(int i) { System.out.println("A(int)"); }
}
class B extends A {
    B() {
        super(100);            // (Q178) we call A(int) ourselves
        System.out.println("B()");
    }
}
// (Q179) or add an A() { } constructor to A
class Demo {
    public static void main(String[] args) { new B(); }
}
```

OUTPUT

```
A(int)
B()
```

> **Exam tip — Exam Tip**
>
> "If the Superclass has no no-arg constructor, the Subclass **must write `super(args)` itself**." — an extension of the very same rule as Case IV (Q26).

9.4 Order with Static / Instance Blocks and Constructors in Inheritance

📄 Lecture Note: Q182

A class hierarchy `A ← B ← C`, `B ← D` (in the diagram). When we do `new D()`:

```java
class A { static { System.out.println("Static block A"); } }
class B extends A { static { System.out.println("Static block B"); } }
class C extends B { static { System.out.println("Static block C"); } }
class D extends B { static { System.out.println("Static block D"); } }

class Demo {
    static { System.out.println("Static block Demo"); }
    public static void main(String[] args) {
        System.out.println("main method starting");
        D d1 = new D();
    }
}
```

OUTPUT

```
Static block Demo
main method starting
Static block A
Static block B
Static block D
```

Rules: `Demo` is loaded first (static block Demo, then `main`). When `new D()` runs, **before D is loaded its Superclasses (A → B)** are loaded, then D. `C` is never loaded (it has nothing to do with D).

Let's add to this the order of **instance blocks + constructors** when an object is created:

```java
class A {
    { System.out.println("A instance block"); }
    A() { System.out.println("A()"); }
}
class B extends A {
    { System.out.println("B instance block"); }
    B() { System.out.println("B()"); }
}
class Demo {
    public static void main(String[] args) { new B(); }
}
```

OUTPUT

```
A instance block
A()
B instance block
B()
```

Remember — the creation order

1. Class load: **Super static → Sub static**.
2. Creating the object: **Super (instance block → constructor) → Sub (instance block → constructor)**.

The Superclass part is finished **first**, because the Subclass is built on top of it.

9.5 Reference type and Object type

📄 Lecture Note: Q180 – Q183

Because of the `IS-A` relationship in Inheritance, a **Subclass object can be assigned to a Superclass reference** (B IS-A A):

```java
A a1 = new B();     // Legal   - a B object in an A reference
B b1 = new A();     // Illegal - an A object is not a B (A IS-NOT-A B)
```

Analogy: "Every dog is an animal" (Dog IS-A Animal), so `Animal x = new Dog()` is fine. But "every animal is a dog" is false, so `Dog d = new Animal()` is wrong.

Q181 Exercise — legal / illegal for `A ← B ← C`, `B ← D`

Reference ↓ \ Object →	`new A()`	`new B()`	`new C()`	`new D()`
`A`	✔	✔	✔	✔
`B`	✘	✔	✔	✔
`C`	✘	✘	✔	✘
`D`	✘	✘	✘	✔

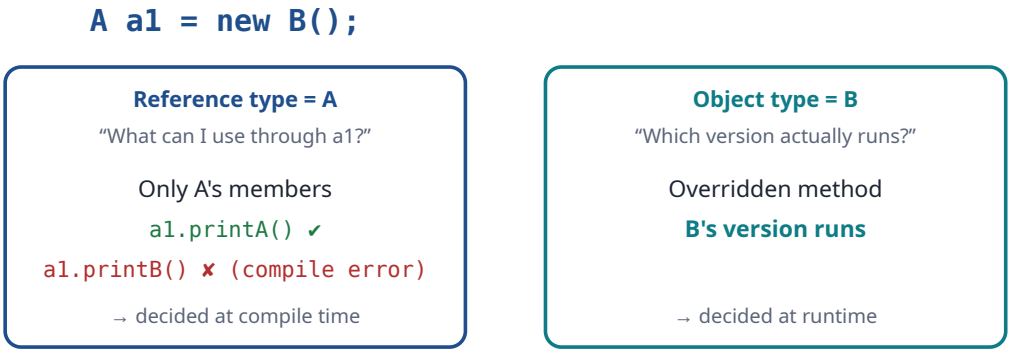
The rule: the Object's type must be the Reference type **or one of its subclasses**.

Q183 — Members can be accessed only according to the Reference type

```java
class A {
    int a;
    public void printA() { System.out.println("A : " + a); }
}
class B extends A {
    int b;
    public void printB() { System.out.println("B : " + b); }
}
class Demo {
    public static void main(String[] args) {
        A a1 = new B();
        a1.a = 100;          // OK
        a1.printA();         // OK
        a1.b = 200;          // ERROR - the B field is not visible through an A reference
        a1.printB();         // ERROR
    }
}
```

COMPILE ERROR

```
Demo.java:14: error: cannot find symbol
        a1.b = 200;          // ERROR - the B field is not visible through an A reference
          ^
  symbol:   variable b
  location: variable a1 of type A
Demo.java:15: error: cannot find symbol
        a1.printB();         // ERROR
          ^
  symbol:   method printB()
  location: variable a1 of type A
2 errors
```



> **Why is it like this? — Why can't a1.printB() work?**
>
> The compiler looks at the **reference type (A)**. Since the `A` class has no `printB()`, it does not compile — even though the object is really a `B`. (To reach the B part you need **Casting**; Chapter 19.)

9.6 Method Overriding

📄 Lecture Note: Q184 – Q187

If a Subclass wants to **change how a Superclass method works to suit itself**, it writes the method again with **the same signature** (name + parameters). This is called **Method Overriding**.

```java
class A {
    public void addStudent(String stName) {
        System.out.println("addStudent of Class A : " + stName);
    }
}

class B extends A {
    @Override
    public void addStudent(String stName) {          // Overriding
        System.out.println("addStudent of Class B : " + stName);
    }
}

class Demo {
    public static void main(String[] args) {
        B b1 = new B();
        b1.addStudent("Niroth");
        new A().addStudent("Niroth");
    }
}
```

OUTPUT

```
addStudent of Class B : Niroth
addStudent of Class A : Niroth
```

> **Remember — The `@Override` annotation**
>
> If you add `@Override`, the compiler checks that you **really are overriding**. If you mistype the name (`addStudnt`), normally it silently becomes a new method; with `@Override` you get a **compile error**. Always use it.

Overriding rules

Rule	
Method **name and parameters** must be the same	
Return type the same (or a subclass type)	
Access level **cannot be reduced** (`public` → `private` ✘)	
`static`, `final`, `private` methods are not overridden	

A real example (Q185 – Q186): overriding `setSize` of JFrame

The `Calculator` window must always be **square**. We override `JFrame`'s `setSize(int w, int h)` and give both dimensions the larger of the length and width:

```
class Calculator extends JFrame {
    public void setSize(int length, int width) {          // Overriding
        if (length < width) length = width;
        super.setSize(length, length);                     // calls the Superclass version
    }
}
// c1.setSize(300, 100)  ->  a 300 x 300 window
```

Look at `super.setSize(...)` here: to call the Superclass's version we use `super.`. If you wrote `setSize(length,length)` without `super.`, it would call itself over and over (**infinite recursion**) and throw a StackOverflowError.

Overloading vs Overriding (Q187)

```
class Calculator extends JFrame {
    public void setSize(short length, short width) { ... }   // Overloading, not Overriding!
}
// c1.setSize(300, 100);  -> int arguments; matches JFrame's setSize(int,int), not the short
version
```

	Overloading	Overriding
Where	In the same class (or inherited)	In the Subclass
Signature	**Different** (params)	**Same**
Decided at	**Compile time**	**Run time**
Relationship	Polymorphism (compile-time)	Polymorphism (run-time)

9.7 Q189 — Calling an overridden method inside a constructor (a trick question!)

📄 Lecture Note: Q188 – Q190

```
class A {
    A() {
        addStudent("Super class A()");        // calls an overridden method
    }
    public void addStudent(String stName) {
        System.out.println("addStudent of Class A : " + stName);
    }
}

class B extends A {
    public void addStudent(String stName) {
        System.out.println("addStudent of Class B : " + stName);
    }
}

class Demo {
    public static void main(String[] args) {
        new B();
    }
}
```

OUTPUT

```
addStudent of Class B : Super class A()
```

Surprising, isn't it? Even though it was called inside the `A()` constructor, **B's** version ran. The reason is that the object is really a `B`, and at **run time** the method is chosen by the object's type (**Dynamic Method Dispatch** — Chapter 10).

> **Watch out — Do not call overridable methods from a constructor**
>
> The overridden method runs at a moment when the Subclass part is **not yet initialized** (B's fields are still default). This leads to bugs.

Chapter 9 — Summary

- `extends` → inherits the Superclass's fields/methods; **IS-A** relationship; Java = single inheritance.
- Subclass constructor → `super(...)` **first** (automatically `super()`). If the Superclass has no no-arg constructor, `super(args)` is mandatory.
- Order: Super static → Sub static → Super (init + ctor) → Sub (init + ctor).
- `A a = new B()` ✔ (upcasting), `B b = new A()` ✘. Member access is controlled by the Reference type.
- **Overriding** = same signature, a new implementation in the subclass. `super.method()` gives the parent version.

Try it yourself — Exercises

9.1 `Person` (name, address) → `Student` (id, marks) → print the automatically inherited fields.

9.2 A `Shape` class (`area()` returns 0) → `Circle`, `Rectangle` subclasses override `area()`.

9.3 In Q182, if you do `new C()`, in what order are the static blocks printed? Run it and confirm.

Chapter 10 — Polymorphism

> **Definition — Polymorphism**
>
> **Poly** = many, **morph** = forms. **The same name / the same interface behaving in different ways.** For example, when you say "make a sound", the dog `barks`, the cat `meows`, the cow `moos` — the request is the same, the behavior is different.

Java has two kinds:

	Compile-time (Static)	Run-time (Dynamic)
Mechanism	**Method Overloading**	**Method Overriding**
Decided by	The compiler (by the arguments)	The JVM (by the object's type)

10.1 Compile-time Polymorphism — Overloading

📄 Lecture Note: Q191 – Q193

`Math.abs()` works with the same name for `int`, `long`, `float` and `double`:

```java
class Demo {
    public static void main(String[] args) {
        int x = -1220;
        long y = -2244;
        float f = -12.2323f;
        double d = -12.2323;
        x = Math.abs(x);
        y = Math.abs(y);
        f = Math.abs(f);
        d = Math.abs(d);
        System.out.println(x);
        System.out.println(y);
        System.out.println(f);
        System.out.println(d);
    }
}
```

OUTPUT

```
1220
2244
12.2323
12.2323
```

In Q192 – Q193 we build our own `Math` class the same way — **4 overloaded static methods** with the same name (`abs`) and 4 parameter types:

```
class MyMath {
    public static int    abs(int num)    { return num < 0 ? -num : num; }
    public static long   abs(long num)   { return num < 0 ? -num : num; }
    public static float  abs(float num)  { return num < 0 ? -num : num; }
    public static double abs(double num) { return num < 0 ? -num : num; }
}

class Demo {
    public static void main(String[] args) {
        System.out.println(MyMath.abs(-1220));        // int version
        System.out.println(MyMath.abs(-2244L));       // long version
        System.out.println(MyMath.abs(-12.5f));       // float version
        System.out.println(MyMath.abs(-12.5));        // double version
    }
}
```

OUTPUT

```
1220
2244
12.5
12.5
```

The compiler looks at **the type of the argument** and chooses the matching method **at compile time**.

> **Note — Overloading rules**
>
> The method name is the same, and the **parameters (number / types / order) must differ**.
>
> You cannot overload by changing **only the return type**.

10.2 Run-time Polymorphism — Overriding + Dynamic Method Dispatch

📄 Lecture Note: Q188, Q194 – Q196

`A a1 = new B();` — reference type `A`, object type `B`. When `a1.addStudent(..)` is called, **the object's real type (B)** decides, and B's overridden method runs. This is **Dynamic Method Dispatch**:

```java
class A {
    public void addStudent(String name) { System.out.println("A.addStudent : " + name); }
}
class B extends A {
    public void addStudent(String name) { System.out.println("B.addStudent : " + name); }
}
class Demo {
    public static void main(String[] args) {
        A a1 = new B();
        a1.addStudent("Niroth");      // the B version
    }
}
```

OUTPUT

```
B.addStudent : Niroth
```

Q190 — a complex chain (A ← B ← C ← D, all overriding `print()`):

```java
class A { public void print() { System.out.println("print() of Class A"); } }
class B extends A { public void print() { System.out.println("print() of Class B"); } }
class C extends B { public void print() { System.out.println("print() of Class C"); } }
class D extends C { public void print() { System.out.println("print() of Class D"); } }

class Demo {
    public static void main(String[] args) {
        A a1 = new A();  A a2 = new B();  A a3 = new C();  A a4 = new D();
        B b1 = new B();  B b2 = new C();  B b3 = new D();
        C c1 = new C();  C c2 = new D();
        D d1 = new D();
        a1.print(); a2.print(); a3.print(); a4.print();
        System.out.println("--");
        b1.print(); b2.print(); b3.print();
        System.out.println("--");
        c1.print(); c2.print();
        System.out.println("--");
        d1.print();
    }
}
```

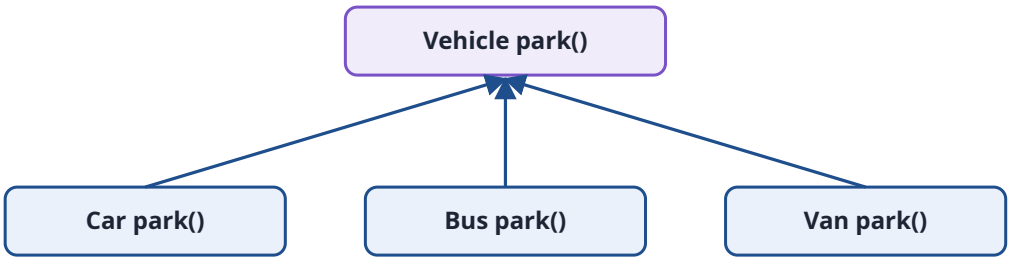
OUTPUT

```
print() of Class A
print() of Class B
print() of Class C
print() of Class D
--
print() of Class B
print() of Class C
print() of Class D
--
print() of Class C
print() of Class D
--
print() of Class D
```

The rule: *No matter which reference you call through, what runs is the object's version (of the class you used `new` on).*

10.3 The Vehicle example — different actions through one interface

📄 Lecture Note: Q194 – Q196



`for (Vehicle v : vehicles) v.park();`

Same call → behavior depends on the object's type

```
class Vehicle {
    public void park() { }              // generic, empty
}
class Car extends Vehicle {
    public void park() { System.out.println("Car Parking"); }
}
class Bus extends Vehicle {
    public void park() { System.out.println("Bus Parking"); }
}
class Van extends Vehicle {
    public void park() { System.out.println("Van Parking"); }
}

class Demo {
    public static void main(String[] args) {
        Vehicle[] vehicles = { new Car(), new Car(), new Bus(), new Van(), new Bus() };
        for (Vehicle v1 : vehicles) {
            v1.park();                    // Single interface, multiple behaviors
        }
    }
}
```

OUTPUT

```
Car Parking
Car Parking
Bus Parking
Van Parking
Bus Parking
```

The `vehicles` array is of type `Vehicle`; you can put **all** of Car, Bus and Van in it (IS-A). The loop knows only about `Vehicle`; when it calls `park()`, the correct version runs according to the real type. You do not need to write things like `if (v instanceof Car) ... else if (v instanceof Bus)...`!

The CarPark class (Q196)

```
class Vehicle { public void park() { } }
class Car extends Vehicle { public void park() { System.out.println("Car Parking"); } }
class Bus extends Vehicle { public void park() { System.out.println("Bus Parking"); } }
class Van extends Vehicle { public void park() { System.out.println("Van Parking"); } }

class CarPark {
    private Vehicle[] vehicleArray = new Vehicle[0];

    public void addVehicle(Vehicle vehicle) {          // any Vehicle subclass
        extendsVehicleArray();
        vehicleArray[vehicleArray.length - 1] = vehicle;
    }
    public void callPark() {
        for (Vehicle vehicle : vehicleArray) {
            vehicle.park();
        }
    }
    private void extendsVehicleArray() {
        Vehicle[] temp = new Vehicle[vehicleArray.length + 1];
        for (int i = 0; i < vehicleArray.length; i++) temp[i] = vehicleArray[i];
        vehicleArray = temp;
    }
}

class Demo {
    public static void main(String[] args) {
        CarPark carPark = new CarPark();
        carPark.addVehicle(new Car());
        carPark.addVehicle(new Bus());
        carPark.addVehicle(new Van());
        carPark.callPark();
    }
}
```

OUTPUT

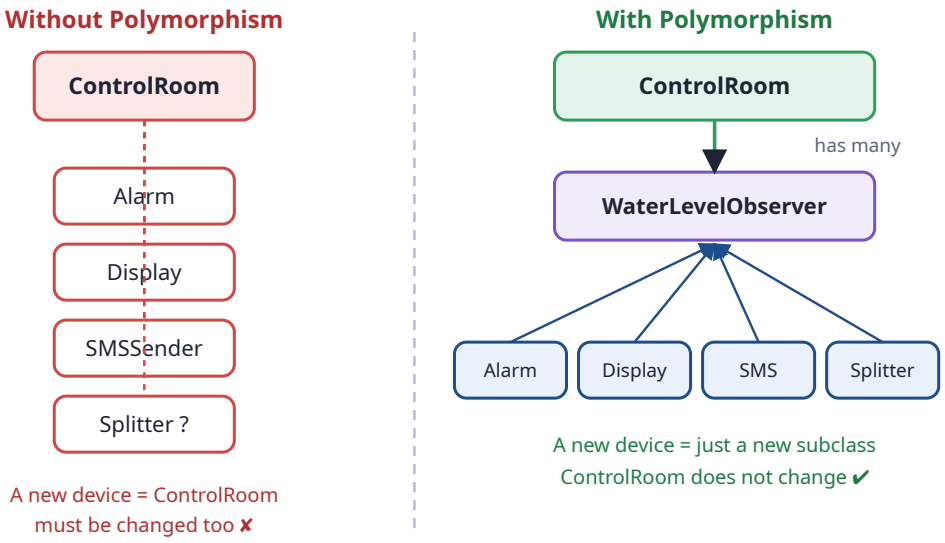
```
Car Parking
Bus Parking
Van Parking
```

The parameter `addVehicle(Vehicle vehicle)` is of **type Vehicle** — so it accepts Car, Bus, Van, and even a `Lorry` **class built in the future**, without changing the `CarPark` class!

10.4 Polymorphism = Software Extensibility (Q197 – Q201)

📄 Lecture Note: Q197 – Q201

This is the **most important** example in the lecture note. A **water level control room (ControlRoom)**: when the water level changes, it notifies the **Alarm** (ON if >50), the **Display** (shows the value) and the **SMS Sender** (sends an SMS). Later we must add a new device called **Splitter** (ON if >75).



Method 1 — Without Polymorphism (Q197 – Q199)

`ControlRoom` has **a separate field and an `addXxx()` for each device**:

```
class ControlRoom {
    private Alarm alarm;  private Display display;  private SMSSender smsSender;
    public void addAlarm(Alarm alarm) { this.alarm = alarm; }
    ...
    private void notifyDevices() {
        alarm.operateAlarm(waterLevel);        // a separate call for each device
        display.display(waterLevel);
        smsSender.sendSMS(waterLevel);
    }
}
```

Problems: (1) Each has its own method name (`operateAlarm`, `display`, `sendSMS`). (2) **To add Splitter, the `ControlRoom` class itself must be changed** (a new field, a new `addSplitter`, and another line in `notifyDevices`). (3) It is harder still if there are several devices of the same kind (2 Alarms).

Method 2 — With Polymorphism (Q200 – Q201)

1. Create **one Superclass**: `WaterLevelObserver` — with an `update(int waterLevel)` method.
2. Every device **extends** it and **overrides** `update()`.
3. `ControlRoom` has **one** `WaterLevelObserver[]` **array** — it knows nothing about the device types.

```
class WaterLevelObserver {
    public void update(int waterLevel) { }
}

class Alarm extends WaterLevelObserver {
    private String id;
    Alarm(String id) { this.id = id; }
    public void update(int waterLevel) {
        System.out.println(waterLevel > 50 ? id + "-->Alarm ON" : id + "-->Alarm OFF");
    }
}
class Display extends WaterLevelObserver {
    private String id;
    Display(String id) { this.id = id; }
    public void update(int waterLevel) {
        System.out.println(id + "-->Water Level : " + waterLevel);
    }
}
class SMSSender extends WaterLevelObserver {
    private String id;
    SMSSender(String id) { this.id = id; }
    public void update(int waterLevel) {
        System.out.println(id + "-->Sending SMS..." + waterLevel);
    }
}

class ControlRoom {
    private WaterLevelObserver[] observers = new WaterLevelObserver[0];
    private int waterLevel;

    public void addWaterLevelObserver(WaterLevelObserver ob) {
        extendArray();
        observers[observers.length - 1] = ob;
    }
    public void setWaterLevel(int waterLevel) {
        if (this.waterLevel != waterLevel) {
            this.waterLevel = waterLevel;
            notifyDevices();
        }
    }
    private void notifyDevices() {
        for (WaterLevelObserver ob : observers) {
            ob.update(waterLevel);              // polymorphic call
        }
    }
    private void extendArray() {
        WaterLevelObserver[] temp = new WaterLevelObserver[observers.length + 1];
        for (int i = 0; i < observers.length; i++) temp[i] = observers[i];
```

```
        observers = temp;
    }
}

class Demo {
    public static void main(String[] args) {
        ControlRoom controlRoom = new ControlRoom();
        controlRoom.addWaterLevelObserver(new Alarm("A0001"));
        controlRoom.addWaterLevelObserver(new Display("D0001"));
        controlRoom.addWaterLevelObserver(new SMSSender("SM0001"));
        controlRoom.addWaterLevelObserver(new Alarm("A0002"));

        controlRoom.setWaterLevel(40);
        System.out.println();
        controlRoom.setWaterLevel(80);
    }
}
```

OUTPUT

```
A0001-->Alarm OFF
D0001-->Water Level : 40
SM0001-->Sending SMS...40
A0002-->Alarm OFF

A0001-->Alarm ON
D0001-->Water Level : 80
SM0001-->Sending SMS...80
A0002-->Alarm ON
```

Adding Splitter — without changing `ControlRoom`!

```
class WaterLevelObserver { public void update(int waterLevel) { } }

class Splitter extends WaterLevelObserver {          // only a new class
    private String id;
    Splitter(String id) { this.id = id; }
    public void update(int waterLevel) {
        System.out.println(waterLevel > 75 ? id + "-->Splitter ON" : id + "-->Splitter
OFF");
    }
}

class ControlRoom {                                   // no change at all
    private WaterLevelObserver[] observers = new WaterLevelObserver[0];
    public void addWaterLevelObserver(WaterLevelObserver ob) {
        WaterLevelObserver[] t = new WaterLevelObserver[observers.length + 1];
        for (int i = 0; i < observers.length; i++) t[i] = observers[i];
        t[t.length - 1] = ob;
        observers = t;
    }
    public void setWaterLevel(int level) {
        for (WaterLevelObserver ob : observers) ob.update(level);
    }
}
```

```
class Demo {
    public static void main(String[] args) {
        ControlRoom room = new ControlRoom();
        room.addWaterLevelObserver(new Splitter("SP001"));
        room.setWaterLevel(90);
        room.setWaterLevel(60);
    }
}
```

OUTPUT

```
SP001-->Splitter ON
SP001-->Splitter OFF
```

Remember — What to learn from this

A new device = just a new subclass. You can extend the system **without touching** the existing `ControlRoom` code. This is called the **Open/Closed Principle** (open for extension, closed for modification). The design is known as the **Observer Design Pattern**.

10.5 Things to watch in Polymorphism questions

```
class Animal {
    String sound() { return "..."; }
    String name = "Animal";
}
class Dog extends Animal {
    String sound() { return "Woof"; }
    String name = "Dog";
}
class Demo {
    public static void main(String[] args) {
        Animal a = new Dog();
        System.out.println(a.sound());     // methods -> by Object type (Dog)
        System.out.println(a.name);        // fields -> by Reference type (Animal) !
    }
}
```

OUTPUT

```
Woof
Animal
```

Watch out — Fields have no polymorphism

Only **methods** are decided at runtime by the object type. **Fields** (and static methods) are decided by the **reference type**. A trick question that appears in exams.

Chapter 10 — Summary

- **Compile-time** = Overloading (same name, different params) — the compiler chooses.
- **Run-time** = Overriding + a Superclass reference → **Dynamic Method Dispatch** — the JVM chooses (by the object type).
- A Superclass-type array/parameter can be given any subclass object.
- With Polymorphism you can add new subclasses **without changing existing code** (**extensibility**; the Observer pattern).
- Methods → object type; Fields → reference type.

Try it yourself — Exercises

10.1 `Shape` (`area()`), `Circle`, `Rectangle`, `Triangle`. Make a `Shape[]` array and calculate the total area.

10.2 Add a new observer called `Logger` to the water-level example (without changing ControlRoom).

10.3 `Employee` ← `Manager`, `Developer` — override `calculateSalary()` and calculate the company's total salary bill.

Chapter 11 — Abstract Classes and Interfaces EXTRA

Extra knowledge — This chapter is not in the lecture note

To complete Polymorphism (Chapter 10) and **Abstraction** (the 4th pillar of OOP), **Abstract classes** and **Interfaces** are essential. Many OOP exams include them.

11.1 What is Abstraction?

Definition — Abstraction

Abstraction = hiding unnecessary details and showing **only what is essential**. To drive a car you only need to know `accelerate()` and `brake()`; you do not need to know how combustion happens inside the engine.

In the last chapter we wrote `Vehicle.park()` with an empty body. But does `new Vehicle()` really **mean anything?** There is no such thing as "a general vehicle"; only real ones such as Car and Bus. That idea is expressed in code by an **abstract class**.

11.2 Abstract Class

- Declared with the `abstract` keyword.
- **Cannot be instantiated with** `new` (you cannot create an object).
- Can have **abstract methods** (no body, ending with `;`) as well as **normal methods**, fields and constructors.
- **A Subclass must override all abstract methods** (otherwise it too must be abstract).

```
abstract class Vehicle {
    private String regNo;
    Vehicle(String regNo) { this.regNo = regNo; }       // abstract classes can have
constructors

    public abstract void park();                        // no body - mandatory for the
subclass

    public void showReg() {                             // normal method (inherited)
        System.out.println("Reg No : " + regNo);
    }
```

```
}

class Car extends Vehicle {
    Car(String regNo) { super(regNo); }
    public void park() { System.out.println("Car Parking"); }
}
class Bus extends Vehicle {
    Bus(String regNo) { super(regNo); }
    public void park() { System.out.println("Bus Parking"); }
}

class Demo {
    public static void main(String[] args) {
        // Vehicle v = new Vehicle("X");        // ERROR: abstract class
        Vehicle[] vs = { new Car("CAB-1234"), new Bus("NB-5678") };
        for (Vehicle v : vs) {
            v.showReg();
            v.park();
        }
    }
}
```

OUTPUT

```
Reg No : CAB-1234
Car Parking
Reg No : NB-5678
Bus Parking
```

```
abstract class Vehicle {
    public abstract void park();
}
class Car extends Vehicle {
    // park() was not overridden
}
class Demo {
    public static void main(String[] args) {
        Vehicle v = new Vehicle();
    }
}
```

COMPILE ERROR

```
Demo.java:4: error: Car is not abstract and does not override abstract method park() in
Vehicle
class Car extends Vehicle {
^
Demo.java:9: error: Vehicle is abstract; cannot be instantiated
        Vehicle v = new Vehicle();
                    ^
2 errors
```

> **Remember — When do you need an abstract class?**
>
> When several Subclasses have **common code** (fields, normal methods) and **some methods are implemented differently in each Subclass**. *"IS-A + sharing some code."*

11.3 Interface

An **Interface** = **purely a "contract"**: only a list of *what* a class must do (methods), without saying *how*.

- The `interface` keyword; methods are automatically `public abstract`.
- A class `implements` an interface and must write all the methods.
- **One class can implement several interfaces** → Java's way of "multiple inheritance".
- Fields, if any, are automatically `public static final` (constants).

```
interface Flyable {
    void fly();
}
interface Swimmable {
    void swim();
}

class Duck implements Flyable, Swimmable {           // implements both interfaces
    public void fly()  { System.out.println("Duck flies"); }
    public void swim() { System.out.println("Duck swims"); }
}
class Fish implements Swimmable {
    public void swim() { System.out.println("Fish swims"); }
}

class Demo {
    public static void main(String[] args) {
        Swimmable[] swimmers = { new Duck(), new Fish() };   // interface type reference
        for (Swimmable s : swimmers) s.swim();               // Polymorphism !
        Flyable f = new Duck();
        f.fly();
    }
}
```

```
OUTPUT
Duck swims
Fish swims
Duck flies
```

Water Level Observer — as an Interface

In Chapter 10, `WaterLevelObserver` (a class with an empty body) is better as an **interface** — it is only a contract:

```java
interface WaterLevelObserver {
    void update(int waterLevel);
}
class Alarm implements WaterLevelObserver {
    public void update(int waterLevel) { ... }
}
```

11.4 Abstract class vs Interface

	Abstract class	Interface
Keyword	`abstract class` / `extends`	`interface` / `implements`
Multiple	✘ (only one `extends`)	✔ (many `implements`)
Fields	Any kind	`public static final` (constants) only
Constructors	✔	✘
Methods	abstract + normal	abstract (+ `default`, `static` in Java 8+)
Relationship	**IS-A** (a family of similar things)	**CAN-DO** (a capability: Flyable, Comparable)

Extra knowledge — Java 8+ — `default` methods

You can add to an interface a **method with a body** using the `default` keyword. This extends the interface without breaking existing implementors.

```java
interface Greeter {
    void greet();
    default void bye() { System.out.println("Bye!"); }
}
```

Remember — Which should you choose?

- Similar classes that **share common code/state** → **abstract class**.
- A **capability common to unrelated classes** (Duck and Plane are both `Flyable`) or you need multiple types → **interface**.

Chapter 11 — Summary

- **abstract class**: no `new`; abstract (no body) + normal methods; subclasses must override.
- **interface**: a contract; `implements`; a class can implement many.

- Abstraction = showing only what is essential; abstract/interface type references work very well with Polymorphism.

Try it yourself — Exercises

11.1 `abstract class Shape` (`abstract double area()`), `Circle`, `Square`. Print the areas through a `Shape[]`.

11.2 `interface Payable { double getPayment(); }` — have both `Employee` and `Invoice` implement it.

11.3 Why can an `abstract` method not be `private` or `final`?

Chapter 12 — Collection Classes and `class` `Object`

12.1 Our own Collection class — `IntegerCollection`

📄 Lecture Note: Q202 – Q214

The problem with an array: its size is **fixed**. Let's build a class that **"grows as needed, and supports adding / removing elements"**. Inside it we keep an ordinary array (`dataArray`), and **hide the complexity of managing that array inside the class** (Encapsulation + Abstraction). The outside code sees only simple methods such as `add()` , `remove()` , `get()` . This is the basic way Java's `ArrayList` works.

Step by step (Q203 – Q214):

Step	Method added	Job
I – II	`add(int data)` , `printData()`	add a value at the end; print
III – IV	`add(int index, int data)`	insert at a specific index
V	`remove(int index)`	remove an index
VI	`isEmpty()` , `size()` , `get(index)`	information
VII	`addFirst` , `addLast` , `removeFirst` , `removeLast`	helper methods (they call the others)

```
class IntegerCollection {
    private int[] dataArray = new int[0];

    // ---- add at the end ----
    public void add(int data) {
        extendsArray();                                  // grows the array by 1
        dataArray[dataArray.length - 1] = data;
    }

    // ---- insert at a given index ----
    public boolean add(int index, int data) {
        if (index >= 0 && index <= dataArray.length) {
            int[] temp = new int[dataArray.length + 1];
            for (int i = 0; i < dataArray.length; i++) {
                temp[i] = dataArray[i];
            }
            for (int i = temp.length - 1; i > index; i--) {   // shift the elements after
the index to the right
```

```
                temp[i] = temp[i - 1];
            }
            temp[index] = data;
            dataArray = temp;
            return true;
        }
        return false;                                   // invalid index
    }

    // ---- remove ----
    public boolean remove(int index) {
        if (index >= 0 && index < dataArray.length) {
            int[] temp = new int[dataArray.length - 1];
            for (int i = 0; i < index; i++) temp[i] = dataArray[i];
            for (int i = index; i < temp.length; i++) temp[i] = dataArray[i + 1];   // shift
left
            dataArray = temp;
            return true;
        }
        return false;
    }

    public boolean isEmpty() { return dataArray.length == 0; }
    public int size()        { return dataArray.length; }

    public int get(int index) {
        if (index >= 0 && index < dataArray.length) return dataArray[index];
        return -1;
    }

    // ---- helper methods (the real work is done by the ones above) ----
    public boolean addFirst(int data) { return add(0, data); }
    public boolean addLast(int data)  { add(data); return true; }
    public boolean removeFirst()      { return remove(0); }
    public boolean removeLast()       { return remove(dataArray.length - 1); }

    public void printData() {
        System.out.print("[");
        for (int i = 0; i < dataArray.length; i++) {
            System.out.print(dataArray[i] + (i < dataArray.length - 1 ? ", " : ""));
        }
        System.out.println("]");
    }

    private void extendsArray() {
        int[] temp = new int[dataArray.length + 1];
        for (int i = 0; i < dataArray.length; i++) temp[i] = dataArray[i];
        dataArray = temp;
    }
}

class Demo {
    public static void main(String[] args) {
        IntegerCollection c = new IntegerCollection();
        System.out.println("Empty : " + c.isEmpty() + "   Size : " + c.size());
        c.add(10); c.add(20); c.add(30); c.add(40); c.add(50);
```

```
        c.printData();                    // [10, 20, 30, 40, 50]

        c.add(2, 25);                     // 25 at index 2
        c.printData();
        c.remove(0);                      // remove the first one
        c.printData();

        c.addFirst(9);
        c.addLast(60);
        c.printData();
        c.removeFirst();
        c.removeLast();
        c.printData();
        System.out.println("get(1) = " + c.get(1) + "   size = " + c.size());
    }
}
```

OUTPUT

```
Empty : true   Size : 0
[10, 20, 30, 40, 50]
[10, 20, 25, 30, 40, 50]
[20, 25, 30, 40, 50]
[9, 20, 25, 30, 40, 50, 60]
[20, 25, 30, 40, 50]
get(1) = 25   size = 5
```

Remember — How the array "grows"

A Java array's size cannot change. So **on every** `add` we create a bigger new array and copy the old elements into it (`extendsArray()`). It is simple, but **slow**. The real `ArrayList` doubles its capacity so this work happens less often.

Why is it like this? — `add(int)` and `add(int,int)` — why the same name?

That is **Method Overloading** (Chapter 10). `add(10)` → at the end; `add(2, 25)` → at index 2. The same "add" concept, with different params.

12.2 `StudentCollection` — for Student objects

📄 Lecture Note: Q215 – Q217

To hold `Student` objects instead of `int`, we write **the same code** again, changing the `int` places to `Student` (`private Student[] studentArray`, `add(Student s)`...). `printStudents()` must print like `{[S0001,Nimal,56,87],[S0002,Amal,76,97]...}`.

```
class Student {
    private String id, name;
    private int prfMarks, dbmsMarks;
```

```
    public Student(String id, String name, int prfMarks, int dbmsMarks) {
        this.id = id; this.name = name; this.prfMarks = prfMarks; this.dbmsMarks =
dbmsMarks;
    }
    public String toString() { return "[" + id + "," + name + "," + prfMarks + "," +
dbmsMarks + "]"; }
    public String getId() { return id; }
}

class StudentCollection {
    private Student[] studentArray = new Student[0];

    public void add(Student student) {
        extendsArray();
        studentArray[studentArray.length - 1] = student;
    }
    public void printStudents() {
        System.out.print("{");
        for (int i = 0; i < studentArray.length; i++) {
            System.out.print(studentArray[i].toString() + (i < studentArray.length - 1 ? ",
" : ""));
        }
        System.out.println("}");
    }
    private void extendsArray() {
        Student[] temp = new Student[studentArray.length + 1];
        for (int i = 0; i < studentArray.length; i++) temp[i] = studentArray[i];
        studentArray = temp;
    }
}

class Demo {
    public static void main(String[] args) {
        StudentCollection sc = new StudentCollection();
        sc.add(new Student("S0001", "Nimal", 56, 87));
        sc.add(new Student("S0002", "Amal", 76, 97));
        sc.add(new Student("S0003", "Bimal", 36, 47));
        sc.printStudents();
    }
}
```

OUTPUT

```
{[S0001,Nimal,56,87], [S0002,Amal,76,97], [S0003,Bimal,36,47]}
```

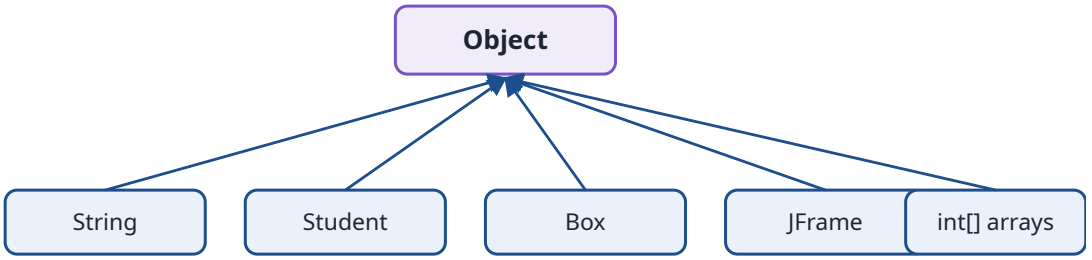
The problem: we have to write **separate classes** — `IntegerCollection`, `StudentCollection`, `BoxCollection`, `DoubleCollection`... — for each type, with **the same logic**. To remove that code duplication we need **one class** that can hold **any object**. What helps here is `class Object` — the **root** class of all Java classes.

12.3 `class Object` — the root of all classes

📄 Lecture Note: Q220 – Q227

Definition — class Object

`java.lang.Object` = the automatic **Superclass** of **every class** in Java. For a class where you have not written any `extends`, the compiler adds `extends Object`. (The comment in Q220: `class Student /*compiler inserts extends Object */`.)



`toString()` `equals()` `hashCode()` `getClass()`

Every class inherits these methods from Object

In Q221: `Student() { super(); }` — this `super()` calls the **`Object()`** constructor.

The 4 important methods of Object

```java
class Student { }

class Demo {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.toString();          // Legal - inherited from Object
        s1.hashCode();
        s1.equals(s1);
        s1.getClass();
        System.out.println("All legal! (it compiled)");
    }
}
```

OUTPUT

```
All legal! (it compiled)
```

Method	What it does	Default behavior
`String toString()`	the object as text	`ClassName@hashHex`
`boolean equals(Object o)`	are they equal?	compares **references** (like `==`)
`int hashCode()`	a hash value	(usually) based on the object's location
`Class getClass()`	the run-time class	

`getClass()` (Q223)

```
class Student {
    private String id;
    Student(String id) { this.id = id; }
}
class Demo {
    public static void main(String[] args) {
        Student s1 = new Student("S0001");
        Class tempObj = s1.getClass();
        System.out.println(tempObj.getName());        // class name
        System.out.println(s1.getClass() == new Student("S0002").getClass());   // the same
class?
    }
}
```

OUTPUT

```
Student
true
```

`toString()` (Q225 – Q226)

`System.out.println(s1)` → `s1.toString()`. If you do not override it, the default is:

`getClass().getName() + "@" + Integer.toHexString(hashCode())` (the comment in Q226). **If you override it**, you get our own format:

```
class Student {
    private String id, name;
    private int prfMarks, dbmsMarks;
    Student(String id, String name, int prfMarks, int dbmsMarks) {
        this.id = id; this.name = name; this.prfMarks = prfMarks; this.dbmsMarks =
dbmsMarks;
    }
    public String toString() {                          // Overriding Object.toString()
        return "[" + id + "," + name + "," + prfMarks + "," + dbmsMarks + "]";
    }
}
class Demo {
    public static void main(String[] args) {
        Student s1 = new Student("S0001", "Niroth", 56, 34);
        System.out.println(s1);                 // [S0001,Niroth,56,34]
        System.out.println(s1.toString());      // same
        System.out.println("Student : " + s1);  // toString() is also called automatically
```

```
in String concatenation
    }
}
```

OUTPUT

```
[S0001,Niroth,56,34]
[S0001,Niroth,56,34]
Student : [S0001,Niroth,56,34]
```

`equals()` and `hashCode()` (Q224)

```
class Student {
    private String id;
    Student(String id) { this.id = id; }
}
class Demo {
    public static void main(String[] args) {
        Student s1 = new Student("S0001");
        Student s2 = new Student("S0001");          // contents are equal
        System.out.println(s1.equals(s2));          // default equals() = compares
references -> false
        System.out.println(s1.equals(s1));          // true
        System.out.println(s1.hashCode() == s2.hashCode());   // usually false
    }
}
```

OUTPUT

```
false
true
false
```

How to **override** `equals()` / `hashCode()` is in Chapter 19.

An Object reference — can hold anything (Q222)

Since every class is a subclass of Object, an `Object` reference can hold **any object** (Polymorphism).

Even arrays and primitives such as `int`, `char` (through auto-boxing):

```
class A {}
class B extends A {}
class C extends B {}
class D extends B {}

class Demo {
    public static void main(String[] args) {
        A[] ar = { new A(), new B(), new C(), new D() };          // Legal
        // B[] br = { new A(), new B() };                          // Illegal - A is not a B

        Object[] objR = { new A(), new B(), new C(), new D(), 100, 'A', "GG" };   // Legal
        Object obj = new int[4];                                  // Legal - an array is
also an object
        Object objS = "Niroth";                                   // Legal
```

```
        for (Object o : objR) System.out.println(o.getClass().getName());
    }
}
```

OUTPUT

```
A
B
C
D
java.lang.Integer
java.lang.Character
java.lang.String
```

> **Note — Auto-boxing**
>
> Java automatically turns `100` (int) into an `Integer` and `'A'` into a `Character` object. That is why primitives can be put into an `Object[]`.

12.4 The general `Collection` class — with `Object[]`

📄 Lecture Note: Q218, Q227

Now let's use `Object[]` to build a collection **from one single class** that can hold Student, Integer, String... **anything**. The call to `objArray[i].toString()` inside `printCollection()` is **polymorphism** — the real object's `toString()` runs.

In Q227, we override `toString()` in the `Collection` class itself, so that `System.out.println(studentCollection)` prints directly:

```
class Student {
    private String id, name;
    private int prfMarks, dbmsMarks;
    public Student(String id, String name, int prfMarks, int dbmsMarks) {
        this.id = id; this.name = name; this.prfMarks = prfMarks; this.dbmsMarks =
dbmsMarks;
    }
    public String toString() { return "[" + id + "," + name + "," + prfMarks + "," +
dbmsMarks + "]"; }
    public String getId() { return id; }
}

class Collection {
    private Object[] objArray = new Object[0];

    public void add(Object obj) {                       // any object
        extendsArray();
        objArray[objArray.length - 1] = obj;
    }
    public Object get(int index) {
```

```
        if (index >= 0 && index < objArray.length) return objArray[index];
        return null;
    }
    public boolean remove(int index) {
        if (index < 0 || index >= objArray.length) return false;
        Object[] temp = new Object[objArray.length - 1];
        for (int i = 0; i < index; i++) temp[i] = objArray[i];
        for (int i = index; i < temp.length; i++) temp[i] = objArray[i + 1];
        objArray = temp;
        return true;
    }
    public int size()        { return objArray.length; }
    public boolean isEmpty() { return objArray.length == 0; }

    public String toString() {                          // the Collection's own toString()
        String s = "{";
        for (int i = 0; i < objArray.length; i++) {
            s += objArray[i] + (i < objArray.length - 1 ? ", " : "");   //
element.toString() auto
        }
        return s + "}";
    }
    private void extendsArray() {
        Object[] temp = new Object[objArray.length + 1];
        for (int i = 0; i < objArray.length; i++) temp[i] = objArray[i];
        objArray = temp;
    }
}

class Demo {
    public static void main(String[] args) {
        Collection studentCollection = new Collection();
        studentCollection.add(new Student("S0001", "Nimal", 56, 87));
        studentCollection.add(new Student("S0002", "Amal", 76, 97));
        System.out.println(studentCollection);

        Collection integerCollection = new Collection();
        integerCollection.add(10);
        integerCollection.add(20);
        integerCollection.add(30);
        System.out.println(integerCollection);          // in the form [10, 20, 30]

        Collection mixed = new Collection();
        mixed.add("Hello");  mixed.add(3.14);  mixed.add(new Student("S9", "Zed", 1, 2));
        System.out.println(mixed);
    }
}
```

OUTPUT

```
{[S0001,Nimal,56,87], [S0002,Amal,76,97]}
{10, 20, 30}
{Hello, 3.14, [S9,Zed,1,2]}
```

Here we used **one `Collection` class** for any type. But there is **a weakness**:

Watch out — Losing type safety with `Object`

`Object get(int i)` returns an `Object`. You must **cast**: `Student s = (Student) c.get(0);`. If you cast to the wrong type you get a **ClassCastException** (at runtime). A better solution is Java's **Generics** (`ArrayList<Student>`) — we look at it in Chapter 19.

Chapter 12 — Summary

- A Collection class = an array inside + methods like `add/remove/get/size` (Encapsulation).
- Writing separate collections for `int`, `Student` = duplication → solved with `Object[]`.
- `class Object` = the root of all classes. `toString()`, `equals()`, `hashCode()`, `getClass()` exist on every object.
- Override `toString()` (for printing) and `equals()` / `hashCode()` (for comparison — Chapter 19).
- An `Object` reference can hold any object; **casting** is needed when getting it back.

Try it yourself — Exercises

12.1 Add `contains(int data)` and `indexOf(int data)` methods to `IntegerCollection`.

12.2 Write `addFirst(Object)` and `clear()` for `Collection`.

12.3 Override `toString()` in a `Box` class, put 3 Boxes into a `Collection`, and print it.

Part Three — Java Applications

Chapter 13 — Exception Handling

13.1 What is an Exception?

📄 Lecture Note: Q159 – Q162

When an **unexpected problem** occurs while the program is running (a wrong array index, a missing file, division by 0, a database that cannot connect), Java creates an **Exception object** and **throws** it. If we do not catch it, the program **crashes**.

```
class Demo {
    public static void main(String[] args) {
        int[] ar = new int[3];
        System.out.println("Before");
        ar[5] = 100;                          // there is no index 5 !
        System.out.println("After");          // never reaches here
    }
}
```

OUTPUT

```
Before
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for
length 3
        at Demo.main(Demo.java:5)
```

13.2 `try – catch`

```
class Demo {
    public static void main(String[] args) {
        int[] ar = new int[3];
        try {
            ar[5] = 100;                                          // the exception occurs here
            System.out.println("This will not run");
        } catch (ArrayIndexOutOfBoundsException ex) {
            System.out.println("Wrong array index, try again... : " + ex.getMessage());
        }
        System.out.println("Program continues");
    }
}
```

OUTPUT

```
Wrong array index, try again... : Index 5 out of bounds for length 3
Program continues
```

In Q159, there is a loop that keeps asking the user again until a valid index is given (`continue` inside the `catch`). In Q160, the index is checked beforehand with an **if** (`index<0 || index>=ar.length`).

> **Why is it like this? — Exception handling vs checking with if**
>
> - **if check**: when the problem can be **predicted in advance** (index range).
> - **try-catch**: when the problem is **outside your control** (a missing file, network, DB error).

Multiple catch and finally

```
class Demo {
    static void test(String s, int d) {
        try {
            int n = Integer.parseInt(s);
            System.out.println("Result : " + (n / d));
        } catch (NumberFormatException ex) {
            System.out.println("Not a number : " + s);
        } catch (ArithmeticException ex) {
            System.out.println("Division error : " + ex.getMessage());
        } finally {
            System.out.println("finally - always runs");
        }
    }
    public static void main(String[] args) {
        test("10", 2);
        test("abc", 2);
        test("10", 0);
    }
}
```

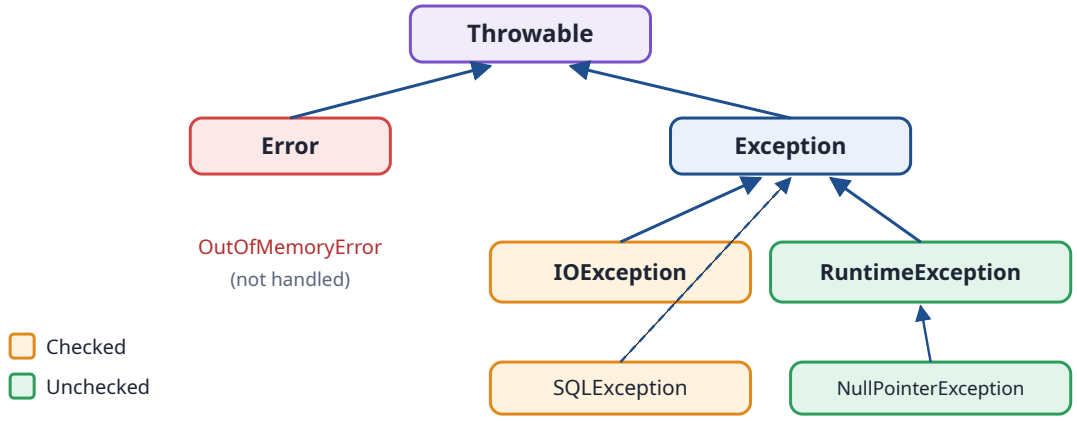
OUTPUT

```
Result : 5
finally - always runs
Not a number : abc
finally - always runs
Division error : / by zero
finally - always runs
```

- **Several `catch` blocks**: the most **specific** exception first, the general one (`Exception`) last.
- `finally` → runs **always**, whether or not an exception occurred (to close files / connections).

13.3 Checked vs Unchecked Exceptions

📄 Lecture Note: Q161 – Q162



Exception class hierarchy.

	Unchecked (Runtime)	Checked
Base class	`RuntimeException` (and its subclasses)	`Exception` (those not RuntimeException)
Examples	`NullPointerException`, `ArrayIndexOutOfBoundsException`, `ArithmeticException`	`IOException`, `SQLException`, `FileNotFoundException`
Compiler check	✘ — handling is not mandatory	✔ — **you must handle it or `throws` it**
Reason	Usually a programmer's bug	Outside problems (file, network)

In Q161, calling `myMethod() throws IOException` (checked) without try-catch is a **compile error**; with `aMethod() throws ArithmeticException` (unchecked) there is no problem:

```java
import java.io.*;
class MyClass {
    public void myMethod() throws IOException { }          // Checked
    public void aMethod() throws ArithmeticException { }   // Unchecked
}
class Demo {
    public static void main(String[] args) {
        MyClass c1 = new MyClass();
        c1.myMethod();      // ERROR - not handled
        c1.aMethod();       // OK
    }
}
```

```
COMPILE ERROR
Demo.java:9: error: unreported exception IOException; must be caught or declared to be thrown
        c1.myMethod();      // ERROR - not handled
                   ^
1 error
```

Two ways to handle (Q162):

```java
import java.io.*;
class MyClass {
    public void myMethod() throws IOException { throw new IOException("demo problem"); }
}
class Demo {
    // Way 2: do not handle it here, pass it to the caller  ->  throws
    static void caller() throws IOException {
        new MyClass().myMethod();
    }
    public static void main(String[] args) {
        // Way 1: handle it with try-catch
        try {
            new MyClass().myMethod();
        } catch (IOException ex) {
            System.out.println("Caught : " + ex.getMessage());
        }
        try { caller(); } catch (IOException ex) { System.out.println("Caught later : " +
ex.getMessage()); }
    }
}
```

OUTPUT

```
Caught : demo problem
Caught later : demo problem
```

Remember — `throw` vs `throws`

- `throw` = the statement that **throws** an exception object (`throw new IOException("..")`).
- `throws` = in the method signature, the declaration "**this exception may come** from this method".

13.4 The Exception hierarchy and our own Exception

Exceptions are also **classes** — an Inheritance tree (`Throwable` → `Exception` → `IOException` ...). We can create our own Exception class:

```java
class InvalidMarksException extends Exception {          // checked exception
    public InvalidMarksException(String msg) { super(msg); }
}

class Student {
    private int marks;
    public void setMarks(int marks) throws InvalidMarksException {
        if (marks < 0 || marks > 100) {
            throw new InvalidMarksException("Marks must be 0-100, but got " + marks);
        }
        this.marks = marks;
    }
    public int getMarks() { return marks; }
}
```

```java
class Demo {
    public static void main(String[] args) {
        Student s = new Student();
        try {
            s.setMarks(85);
            System.out.println("Marks : " + s.getMarks());
            s.setMarks(150);
        } catch (InvalidMarksException ex) {
            System.out.println("Error : " + ex.getMessage());
        }
    }
}
```

OUTPUT

```
Marks : 85
Error : Marks must be 0-100, but got 150
```

Extra knowledge — try-with-resources

For resources that **must be closed**, such as files / DB connections, Java 7+ lets you write `try (FileWriter fw = new FileWriter("a.txt")) { ... }`. When the block ends, `fw.close()` is called **automatically** (Chapter 14).

Chapter 13 — Summary

- **Exception** = an object representing a run-time problem. `try { } catch (Type ex) { } finally { }`.
- **Unchecked** (`RuntimeException`) — handling not mandatory. **Checked** (`IOException` , `SQLException`) — try-catch or `throws` is mandatory.
- `throw` = to throw; `throws` = to declare. Our own exceptions `extends Exception`.

Try it yourself — Exercises

13.1 Handle a call to `Integer.parseInt("12a")` with try-catch and print `"Invalid number"` .

13.2 In `BankAccount.withdraw()` , throw an `InsufficientFundsException` (custom) if the balance is not enough.

13.3 What happens if you write `catch (Exception e)` first and `catch (ArithmeticException e)` second? (Hint: compile error)

Chapter 14 — Java IO: Reading and Writing Files

14.1 Why do we need files?

The data in variables, objects and arrays lives in **RAM**. The moment the program closes, it is gone. To keep data **permanently**, we must save it in a **file** (or a database). The classes in Java's `java.io` package do this job.

> **Note — All IO operations can throw `IOException` (checked)**
>
> So they must be written inside a `try-catch` (Chapter 13).

14.2 `FileWriter` — writing to a file

📄 Lecture Note: Q132 – Q134

```java
import java.io.*;

class Demo {
    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("Test.txt");      // creates the file if missing;
if it exists, writes it fresh (overwrite)
            fw.write("Hello java");
            fw.close();                                      // flush + close (mandatory)
            System.out.println("Written!");
        } catch (IOException ex) {
            System.out.println(ex.getMessage());
        }
    }
}
```

```
OUTPUT
Written!
```

> **Watch out — Do not forget `flush()` / `close()`!**
>
> What you `write()` first goes into a **buffer** (temporary memory). If you do not call `flush()` (empties the buffer into the file) or `close()` (flush + releases the resource), **nothing may reach the file**. Q132 shows `fw.flush()` and Q133 shows `fw.close()`; **`close()`** is better.

Overwrite vs Append (Q134)

`new FileWriter("Test.txt")` → every time, it **erases the old content** and writes afresh. `new FileWriter("Test.txt", true)` → **adds after the old content** (append).

```
import java.io.*;

class Demo {
    public static void main(String[] args) throws IOException {
        for (int i = 1; i <= 3; i++) {
            FileWriter fw = new FileWriter("Test.txt", true);      // append mode
            fw.write("Hello java " + i + "\n");                    // \n = a new line
            fw.close();
        }
        System.out.println(new java.util.Scanner(new
File("Test.txt")).useDelimiter("\\Z").next());
    }
}
```

OUTPUT

```
Hello java 1
Hello java 2
Hello java 3
```

14.3 The `File` class

📄 Lecture Note: Q135 – Q137

The `File` class represents the **name and path** of a file / folder; it is not for writing / reading but for **information about the file** and operations like **delete, exists....**

```
import java.io.*;

class Demo {
    public static void main(String[] args) throws Exception {
        File f1 = new File("Test.txt");
        System.out.println("Exists before : " + f1.exists());

        FileWriter fw = new FileWriter(f1, true);        // FileWriter can also take a File
object
        fw.write("Hello java\n");
        fw.close();

        System.out.println("Exists after  : " + f1.exists());
        System.out.println("Length        : " + f1.length() + " bytes");
        System.out.println("Name          : " + f1.getName());

        f1.delete();                                      // Q137 - deleting the file
        System.out.println("After delete : " + f1.exists());
    }
}
```

```
OUTPUT
Exists before : false
Exists after  : true
Length        : 11 bytes
Name          : Test.txt
After delete : false
```

Kinds of paths (Q136):

Kind	Example	
Absolute path	`D:/ICD122125/IO/SMS/Test.txt`	the full location (starting from the drive)
Relative path	`../IO/SMS/Test.txt` or `Test.txt`	from where it runs now (the **working directory**)

> **Remember — On Windows paths**
>
> Use `/` instead of `\` (`"D:/data/a.txt"`), or write `"D:\\data\\a.txt"` (two backslashes). `\` is the escape character in Java strings.

14.4 Reading a file

📄 Lecture Note: Q138 – Q141

Method 1 — `FileReader`: one character at a time (you get a character code as an `int`; `-1` at the end).

Method 2 — `BufferedReader`: one line at a time with `readLine()` (`null` at the end).

Method 3 — `Scanner`: the easiest. `hasNextLine()` / `nextLine()`.

```java
import java.io.*;
import java.util.*;

class Demo {
    public static void main(String[] args) throws IOException {
        FileWriter fw = new FileWriter("Demo.txt");
        fw.write("Line one\nLine two\nLine three\n");
        fw.close();

        // 1) FileReader - one character at a time
        FileReader fr = new FileReader("Demo.txt");
        int code = fr.read();
        System.out.println(code + " -> " + (char) code);
        fr.close();

        // 2) BufferedReader - one line at a time
        BufferedReader br = new BufferedReader(new FileReader("Demo.txt"));
        String line = br.readLine();
```

```
        while (line != null) {
            System.out.println("BR : " + line);
            line = br.readLine();
        }
        br.close();

        // 3) Scanner
        Scanner input = new Scanner(new File("Demo.txt"));
        while (input.hasNextLine()) {
            System.out.println("SC : " + input.nextLine());
        }
        input.close();
    }
}
```

OUTPUT

```
76 -> L
BR : Line one
BR : Line two
BR : Line three
SC : Line one
SC : Line two
SC : Line three
```

14.5 Student data in a file — CSV format

📄 Lecture Note: Q142 – Q145

We keep student data as **one student per line**, separated by commas (`,`) (**CSV**: Comma Separated Values):

```
S0001,Nimal,65,89
S0002,Amal,75,39
```

Read a line, split it into parts with `split(",")`, and create a `Student` object (Q143 – Q145):

```
import java.io.*;
import java.util.*;

class Student {
    private String id, name;
    private int prfMarks, dbmsMarks;
    public Student(String id, String name, int prf, int dbms) {
        this.id = id; this.name = name; this.prfMarks = prf; this.dbmsMarks = dbms;
    }
    public String toString() { return id + "\t" + name + "\t" + prfMarks + "\t" + dbmsMarks;
}
}

class Demo {
    public static void main(String[] args) throws IOException {
```

```
        FileWriter fw = new FileWriter("Student.txt");
        fw.write("S0001,Nimal,65,89\nS0002,Amal,75,39\n");
        fw.close();

        Student[] studentArray = new Student[0];
        Scanner input = new Scanner(new File("Student.txt"));
        while (input.hasNextLine()) {
            String line = input.nextLine();
            String[] rowData = line.split(",");                 //
["S0001","Nimal","65","89"]
            Student s = new Student(rowData[0], rowData[1],
                                    Integer.parseInt(rowData[2]),    // String → int
                                    Integer.parseInt(rowData[3]));
            Student[] temp = new Student[studentArray.length + 1];
            for (int i = 0; i < studentArray.length; i++) temp[i] = studentArray[i];
            temp[temp.length - 1] = s;
            studentArray = temp;
        }
        input.close();

        for (Student s : studentArray) System.out.println(s);
    }
}
```

```
OUTPUT
S0001   Nimal   65      89
S0002   Amal    75      39
```

Updating the file — Delete / Update (Q163)

There is no direct way to remove (or change) **a line in the middle** of a file. **The standard method:**

1. Create a **temporary file** (`TempStudent.txt`).
2. Read the original file line by line; write **every line except the one to delete** into the temp file.
3. **Delete** the original, and **give the temp file the original name** (or copy it back).

```
import java.io.*;
import java.util.*;

class Demo {
    public static boolean deleteStudent(String id) throws IOException {
        boolean found = false;
        FileWriter fw = new FileWriter("TempStudent.txt");
        Scanner input = new Scanner(new File("Student.txt"));
        while (input.hasNextLine()) {
            String line = input.nextLine();
            if (line.substring(0, 5).equalsIgnoreCase(id)) {     // skip if the id matches
                found = true;
                continue;
            }
            fw.write(line + "\n");
        }
        fw.close();
```

```
        input.close();

        new File("Student.txt").delete();                       // delete the old one
        new File("TempStudent.txt").renameTo(new File("Student.txt"));   // give the temp
file the original name
        return found;
    }

    public static void main(String[] args) throws IOException {
        FileWriter fw = new FileWriter("Student.txt");
        fw.write("S0001,Nimal,65,89\nS0002,Amal,75,39\nS0003,Bimal,55,49\n");
        fw.close();

        System.out.println("Deleted S0002 : " + deleteStudent("S0002"));
        Scanner input = new Scanner(new File("Student.txt"));
        while (input.hasNextLine()) System.out.println(input.nextLine());
    }
}
```

OUTPUT

```
Deleted S0002 : true
S0001,Nimal,65,89
S0003,Bimal,55,49
```

Extra knowledge — try-with-resources and `Files` (Java 7+)

```
try (Scanner in = new Scanner(new File("Student.txt"))) {
    while (in.hasNextLine()) System.out.println(in.nextLine());
}   // in.close() automatically
```

`java.nio.file.Files.readAllLines(Path.of("Student.txt"))` — the entire file as a `List<String>` in one line.

Why is it like this? — Why a DB rather than a file?

With a file, **search / update / delete** require rewriting the whole file, and it is hard when several users work at once. A **database** (next chapter) does all of that efficiently.

Chapter 14 — Summary

- Writing: `FileWriter(file)` (overwrite), `FileWriter(file, true)` (append); **`close()`** is mandatory.
- Reading: `FileReader` (a character), `BufferedReader.readLine()` (a line; `null` = end), `Scanner` (easiest).
- The `File` class: `exists()`, `delete()`, `length()`; **absolute vs relative** path.
- CSV lines → `split(",")` + `Integer.parseInt()` → object.

- Update / delete = the **temp file method**.

Try it yourself — Exercises

14.1 Write an `addStudent(Student s)` method that **appends** a student to `Student.txt` .

14.2 Read all students from the file and print the student with the **highest PRF marks**.

14.3 Write `updateStudent(id, newPrf)` using the temp file method.

Chapter 15 — JDBC (Java Database Connectivity)

15.1 What is JDBC?

📄 Lecture Note: Q146 – Q158

JDBC = the **API** (the `java.sql` package) Java provides so that a Java program can **connect to a Database** (MySQL, Oracle, ...) and execute **SQL commands** (insert, update, delete, select).





ResultSet (for select) / int (for insert, update, delete) comes back

To talk to a database you need **Driver software** (for MySQL, **MySQL Connector/J** — a `.jar` file); it must be added to the project's **classpath**.

The 5 steps of JDBC (Q148 – Q149)

#	Step	Code
1	Prepare the **SQL** statement	`String SQL = "Insert into Student values(...)";`
2	Load the **Driver**	`Class.forName("com.mysql.cj.jdbc.Driver");`
3	Create a **Connection**	`DriverManager.getConnection(url, user, password);`
4	Create a **Statement**	`conn.createStatement();`
5	**Execute** the SQL	`stm.executeUpdate(SQL);` / `stm.executeQuery(SQL);`

15.2 Setting up the database

📄 Lecture Note: Q150 – Q151

The lecture note's database in MySQL:

```
CREATE DATABASE StudentDB;
USE StudentDB;
CREATE TABLE Student(
    id        VARCHAR(6) NOT NULL,
    name      VARCHAR(30),
    prfMarks  INTEGER(4),
    dbmsMarks INTEGER(4),
    CONSTRAINT PRIMARY KEY (id)
);
INSERT INTO Student VALUES ('S0001','Niroth',65,17);
INSERT INTO Student VALUES ('S0002','Nimal',95,47);
INSERT INTO Student VALUES ('S0003','Bimal',35,97);
```

The connection URL: `jdbc:mysql://localhost/StudentDB` (`localhost` = this computer, `StudentDB` = the database name), username `root`, and the password you set when installing.

15.3 Insert — adding data

📄 Lecture Note: Q152 – Q153

```
import java.sql.*;

class Demo {
    public static void main(String[] args) {
        String SQL = "Insert into Student values('S0005','Lakmal',87,34)";
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");                    // 2) Driver
            Connection conn = DriverManager.getConnection(
                    "jdbc:mysql://localhost/StudentDB", "root", "1234");    // 3) Connection
            Statement stm = conn.createStatement();                        // 4) Statement
            stm.executeUpdate(SQL);                                        // 5) Execute
            System.out.println("Inserted...");
        } catch (ClassNotFoundException ex) {
            System.out.println("Driver s/w not found...");                 // if the jar
file is missing
        } catch (SQLException ex) {
            System.out.println(ex.getMessage());                           // SQL /
connection error
        }
    }
}
```

You must handle **2 exceptions** (Q152): `ClassNotFoundException` (if the driver is missing) and `SQLException` (checked — a DB error).

15.4 Delete and Update

📄 Lecture Note: Q154 – Q155

For `insert`, `update`, `delete` we use **`executeUpdate()`** — it returns **the number of rows affected** (an `int`). If `res > 0` it succeeded:

```
String SQL = "delete from Student where id='S0005'";
// ... (steps 1-4 same as before)
int res = stm.executeUpdate(SQL);
if (res > 0) {
    System.out.println("Deleted...");
}

String SQL2 = "Update Student set prfMarks=99 where id='S0001'";
int res2 = stm.executeUpdate(SQL2);
if (res2 > 0) System.out.println("Update Success...");
```

15.5 Select — reading data (`ResultSet`)

📄 Lecture Note: Q156 – Q157

For `select` we use **`executeQuery()`** — it returns a **`ResultSet`** (the rows of the table). The `ResultSet` cursor starts **before the first row**; **`rst.next()`** → moves to the next row (`true` if a row exists).

```
String SQL = "Select * From Student";
ResultSet rst = stm.executeQuery(SQL);

while (rst.next()) {                              // row by row
    String id       = rst.getString("id");        // or rst.getString(1)
    String name     = rst.getString("name");      //      rst.getString(2)
    int prfMarks    = rst.getInt("prfMarks");
    int dbmsMarks   = rst.getInt("dbmsMarks");
    System.out.println(id + "\t" + name + "\t" + prfMarks + "\t" + dbmsMarks);
}
```

Method	Meaning
`executeUpdate(sql)`	insert / update / delete → **int** (row count)
`executeQuery(sql)`	select → **ResultSet**
`rst.next()`	the next row; `false` if none
`rst.getString/getInt(column)`	the column's value (by name or by an index starting at 1)

15.6 With OOP — a Student object into the DB

📄 Lecture Note: Q158

The SQL string is built **from the Student object**:

```
public static boolean addNewStudent(Student student) {
    String SQL = "Insert into Student values('" + student.getId() + "','" +
student.getName()
                + "'," + student.getPrfMarks() + "," + student.getDbmsMarks() + ")";
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection conn = DriverManager.getConnection("jdbc:mysql://localhost/StudentDB",
"root", "1234");
        Statement stm = conn.createStatement();
        return stm.executeUpdate(SQL) > 0;
    } catch (ClassNotFoundException | SQLException ex) {
        System.out.println(ex.getMessage());
        return false;
    }
}
```

Watch out — SQL Injection — the danger of string concatenation (very important!)

If a user enters something like `x'); DROP TABLE Student; --` as the name, the SQL itself changes and can **destroy the database**. In real applications always use `PreparedStatement`.

Extra knowledge — PreparedStatement — the safe way

```
String SQL = "Insert into Student values(?,?,?,?)";          // ? = placeholders
PreparedStatement pstm = conn.prepareStatement(SQL);
pstm.setString(1, student.getId());
pstm.setString(2, student.getName());
pstm.setInt(3, student.getPrfMarks());
pstm.setInt(4, student.getDbmsMarks());
int res = pstm.executeUpdate();
```

Because the values are sent **separately**, they never become part of the SQL command; it is also faster.

Remember — Creating the connection again and again

In the examples above we created a new Connection in every method. In real projects we create **one Connection class (Singleton — Chapter 18)** and use it everywhere. Then the DB credentials live in one place.

Chapter 15 — Summary

- The 5 JDBC steps: **SQL → Driver → Connection → Statement → Execute**.
- `executeUpdate()` → int (insert/update/delete); `executeQuery()` → `ResultSet` (select; `while(rst.next())`).
- Exceptions: handle `ClassNotFoundException`, `SQLException`.
- Use **PreparedStatement** to prevent SQL Injection.

> **Try it yourself — Exercises**
>
> **15.1** Write a `searchStudent(String id)` method (returns a `Student` object, or `null`).
>
> **15.2** `getAllStudents()` — fill all rows into a `Student[]` / `Collection`.
>
> **15.3** Write `addNewStudent` using PreparedStatement.

Chapter 16 — Java Swing: GUI Basics

16.1 What is a GUI?

So far our programs ran on the **console** (the black screen). A **GUI (Graphical User Interface)** means windows, buttons, text boxes. Java's **Swing** (`javax.swing` , `java.awt`) is the library for building GUIs. Swing is built entirely on **OOP** — `JFrame` and `JButton` are all **classes**; we create them as objects and call their methods.

> **Note — Running the code in this chapter**
>
> Swing code needs a **display** (on your computer). So the code blocks in this chapter show no output; **type and run them on your own computer** — that is how you will really understand them.

Component	Class	Use
Window	`JFrame`	the main window
Button	`JButton`	to click
Label	`JLabel`	to show text
Text box	`JTextField`	to enter text
Panel	`JPanel`	a container that groups components
Table	`JTable`	a table of data

16.2 Let's make a window — `JFrame`

📄 Lecture Note: Q72 – Q77

```java
import javax.swing.*;

class Demo {
    public static void main(String[] args) {
        JFrame f1 = new JFrame();                       // Q72: the window object is created,
but not visible
        f1.setSize(300, 300);                           // Q74: size (width, height) -
otherwise tiny/invisible
        f1.setTitle("Calculator");                      // text in the title bar
        f1.setLocationRelativeTo(null);                 // center of the screen
        f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);   // what happens when X is
clicked
        f1.setVisible(true);                            // Q73: show it (otherwise
invisible!)
    }
}
```

Values of `setDefaultCloseOperation` (Q75 – Q77)

Constant	Value	When X is clicked
`JFrame.DO_NOTHING_ON_CLOSE`	0	nothing
`JFrame.HIDE_ON_CLOSE`	1	hides the window (the program remains) — **default**
`JFrame.DISPOSE_ON_CLOSE`	2	closes **only this window**
`JFrame.EXIT_ON_CLOSE`	3	stops **the entire program**

> **Remember — DISPOSE vs EXIT**
>
> In an application with several windows, give ordinary **sub-windows** `DISPOSE_ON_CLOSE` and the **main window** `EXIT_ON_CLOSE`. (In Q77, if both f1 and f2 are given DISPOSE, the program ends by itself once both are closed.)

16.3 Adding components

📄 Lecture Note: Q78 – Q79

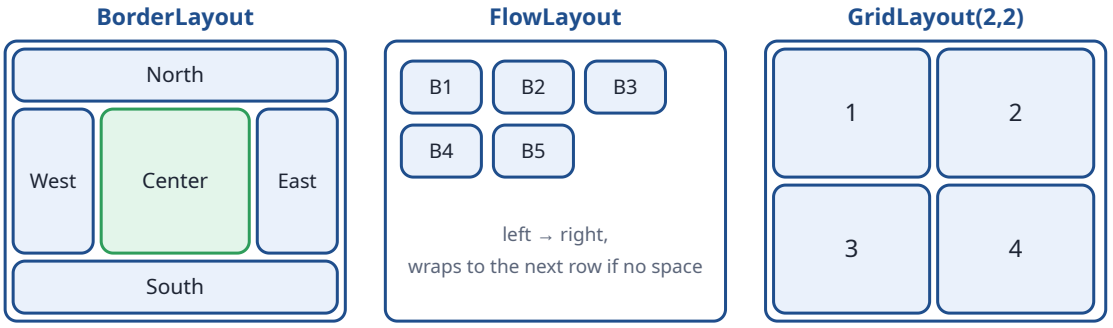
```java
JButton btnExit = new JButton();
btnExit.setText("Exit");
btnExit.setFont(new Font("", Font.BOLD, 25));
f1.add(btnExit);                                    // adding to the window
```

If you add one button, it fills the entire window. If you add a second one too (Q79), **the first one gets covered!** — because the window has a **Layout Manager**.

16.4 Layout Managers

📄 Lecture Note: Q80 – Q88

A **Layout Manager** = the object that decides **where and in what size** components are placed in the window. You do not handle pixels with things like `setLocation(x,y)`.



(a) BorderLayout — the default of `JFrame`

5 regions: **North, South, East, West, Center**. **Only one component** per region (Q81 – Q83):

```
f1.add("North", btnNorth);        // or f1.add(btnNorth, BorderLayout.NORTH)
f1.add("South", btnSouth);
f1.add("West",  btnWest);
f1.add("East",  btnEast);
f1.add("Center", btnCenter);
```

(b) FlowLayout — left to right (Q84 – Q85)

```
f1.setLayout(new FlowLayout());                    // center aligned
f1.setLayout(new FlowLayout(FlowLayout.RIGHT));    // right aligned
```

Components are placed one after another in a row at their **preferred size**; when space runs out, they go to the next row.

(c) GridLayout — like a table (Q86 – Q88)

```
f1.setLayout(new GridLayout(4, 4, 2, 2));          // rows, columns, horizontal gap,
vertical gap
```

Every cell is **the same size**. Components fill in the order they are added. How to add the **Calculator**'s 16 buttons using an Array + for loop (Q88):

```
f1.setLayout(new GridLayout(4, 4, 2, 2));
JButton[] buttonArray = new JButton[16];
String[] buttonName = {"7","8","9","*","4","5","6","/","1","2","3","+","0",".","=","-"};
for (int i = 0; i < 16; i++) {
    buttonArray[i] = new JButton(buttonName[i]);
    buttonArray[i].setFont(new Font("", Font.BOLD, 25));
    f1.add(buttonArray[i]);
}
```

Remember — `JButton[]` — an Object reference array!

This is another place where we meet the **object reference array** from Chapter 8. Only after `buttonArray[i] = new JButton(..)` is the object actually created.

16.5 `JTextField`, `JLabel` and `JPanel`

📄 Lecture Note: Q89 – Q97

- `JTextField(5)` — 5 = width (columns). `setHorizontalAlignment(JTextField.RIGHT)`, `setEditable(false)` (read-only).
- `JLabel("Student ID")` — displays text.
- Because of the layout, a `JTextField` placed in the `South` of a `BorderLayout` stretches to **the full width**; in a `FlowLayout` it keeps its preferred size (Q89 – Q90).

JPanel — layouts inside layouts

You cannot make a complex design with one layout. The solution: create a `JPanel` (an inner container), give **it its own layout**, then add that panel to the window (Q97):

```
JFrame f1 = new JFrame();
f1.setLayout(new BorderLayout());

JTextField txtDisplay = new JTextField();
txtDisplay.setFont(new Font("", Font.BOLD, 25));
txtDisplay.setHorizontalAlignment(JTextField.RIGHT);
f1.add("North", txtDisplay);                          // display at the top

JPanel buttonPanel = new JPanel(new GridLayout(4, 4, 4, 4));   // grid panel for the 16
buttons
for (int i = 0; i < 16; i++) {
    JButton b = new JButton(buttonName[i]);
    b.setFont(new Font("", Font.BOLD, 25));
    buttonPanel.add(b);
}
f1.add("Center", buttonPanel);                         // to the center
```

Region	Component	Layout
Window	—	BorderLayout
North	`txtDisplay`	—
Center	`buttonPanel` (JPanel)	**GridLayout(4,4)**

Remember — How to build a form (Q92 – Q95)

The **Add Student Form**: Window → BorderLayout. **North** = the Title label; **South** = a `JPanel(FlowLayout.RIGHT)` holding the `Add`, `Cancel` buttons; **Center** = a `JPanel(GridLayout(4,2))` holding 4 pairs of `Label + TextField`. If you put a panel (`FlowLayout.LEFT`) inside, the text field is not stretched and keeps its preferred size (Q111).

Chapter 16 — Summary

- Swing components = **classes**; create with `new` and configure with **methods**.
- Without `setVisible(true)` the window is invisible; `setDefaultCloseOperation` controls the X button.
- **Layouts**: Border (5 regions, JFrame default), Flow (row by row), Grid (equal cells).
- **JPanel + another layout** = nested layouts → complex forms.

Try it yourself — Exercises

16.1 Show a "Hello" label (JLabel) in the middle of a 300×300 window.

16.2 Q82: show 5 buttons in a BorderLayout (North, South, East, West, Center).

16.3 A login form: Username, Password (JPasswordField) and Login, Cancel buttons.

Chapter 17 — Swing + OOP: Forms, Inheritance and Events

17.1 Instead of building the GUI in `Demo` — `extends JFrame`

📄 Lecture Note: Q98 – Q104

In Chapter 16 we built the GUI inside `main` with `JFrame f1 = new JFrame()`. The problems: **`main` becomes huge**, **the same GUI cannot be reused**, and there is **no class of our own called `Calculator`**.

The lecture note's **6 steps** (Q98 – Q104):

Step	Code	Idea
I (Q98)	`Calculator c1 = new Calculator();` `c1.setSize(...)`	the way we would like it — does not compile without the `Calculator` class
II (Q99)	`class Calculator { }`	an empty class → no `setSize`
III (Q100)	`class Calculator extends JFrame { }`	**`Calculator IS-A JFrame`** — `setSize`, `setTitle` are all **inherited**
IV (Q101)	`new Calculator().setVisible(true)`	`main` is short!
V (Q102)	`setTitle(..)`, `setSize(..)` in the **Constructor**	the window settings go **inside Calculator**
VI (Q103–104)	Fields (`txtDisplay`, `buttonArray`), components created in the constructor	the complete calculator

The complete Calculator class (Q104):

```
import javax.swing.*;
import java.awt.*;

class Calculator extends JFrame {                    // Calculator IS-A JFrame
    private JTextField txtDisplay;                   // components → private fields
(Encapsulation)
    private JButton[] buttonArray;

    Calculator() {                                    // the constructor builds the GUI
        setTitle("Calculator");                       // this.setTitle - inherited from
JFrame
```

```
        setSize(300, 300);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        buttonArray = new JButton[16];
        String[] buttonName =
{"7","8","9","*","4","5","6","/","1","2","3","+","0",".","=","-"};
        JPanel buttonPanel = new JPanel(new GridLayout(4, 4, 4, 4));
        for (int i = 0; i < 16; i++) {
            buttonArray[i] = new JButton(buttonName[i]);
            buttonArray[i].setFont(new Font("", Font.BOLD, 25));
            buttonPanel.add(buttonArray[i]);
        }
        add("Center", buttonPanel);

        txtDisplay = new JTextField(5);
        txtDisplay.setFont(new Font("", 1, 20));
        add("North", txtDisplay);
    }
}

class Demo {
    public static void main(String[] args) {
        Calculator c1 = new Calculator();
        c1.setVisible(true);                          // main is now very short
    }
}
```

Remember — Inheritance + Encapsulation + Constructor — all together

- **Inheritance**: all of JFrame's abilities for free.
- **Encapsulation**: components are `private` fields.
- **Constructor**: the initialization that builds the window.
- Result: `new Calculator().setVisible(true);` — one line to open the Calculator from anywhere else!

Why is it like this? — Why `setTitle(...)` inside the constructor without `this.`?

It refers to `this`. Since the Calculator object **is a JFrame**, `setTitle(..)` = `this.setTitle(..)` = the inherited method.

17.2 Forms — `AddStudentForm`, `SearchStudentForm`...

📄 Lecture Note: Q105 – Q115

The Student Management System has several forms (Add, Search, Update, Delete, View). Each one is `extends JFrame`. Steps:

1. **Step I (Q106):** `class AddStudentForm extends JFrame` — title / size / close / location in the constructor.
2. **Step II (Q107):** South — `btnAdd`, `btnCancel` in a `JPanel(FlowLayout.RIGHT)`.
3. **Step III (Q108):** North — the title `JLabel` ("Add New Student Form").
4. **Steps IV – VI (Q109 – Q111):** Center — labels and text fields in a `JPanel(GridLayout(4,2))`; the size of the text fields is controlled by placing them inside `FlowLayout.LEFT` panels.

```
class AddStudentForm extends JFrame {
    private JTextField txtId, txtName, txtPrfMarks, txtDbmsMarks;      // Encapsulation
    private JButton btnAdd, btnCancel;

    AddStudentForm() {
        setTitle("Add Student Form");
        setSize(400, 300);
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);      // sub-window → DISPOSE
        setLocationRelativeTo(null);

        JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT));
        btnAdd = new JButton("Add Student");   btnAdd.setFont(new Font("", 1, 20));
        btnCancel = new JButton("Cancel");     btnCancel.setFont(new Font("", 1, 20));
        buttonPanel.add(btnAdd);  buttonPanel.add(btnCancel);
        add("South", buttonPanel);

        JLabel lblTitle = new JLabel("Add New Student Form");
        lblTitle.setFont(new Font("", 1, 28));
        lblTitle.setHorizontalAlignment(JLabel.CENTER);
        add("North", lblTitle);

        // ... Center: labels + text fields in a GridLayout(4, ...)
    }
}
```

The **Search / Update / Delete Forms** in Q112 – Q114 have **almost the same code** — only the title and the button name differ. How do we reduce this **duplication**? With **Inheritance**: put the common form layout in a superclass called `StudentForm`, then `AddStudentForm extends StudentForm` … (**an Extra in Chapter 19**).

17.3 `JTable` and `DefaultTableModel`

📄 Lecture Note: Q116 – Q119

`JTable` shows data as a table. Its data lives in a `DefaultTableModel` (the Model) — **`JTable` = the look (View), `DefaultTableModel` = the data (Data)**. You change data through the **model**.

```
import javax.swing.*;
import javax.swing.table.*;
```

```
import java.awt.*;

class ViewStudentDetailsForm extends JFrame {
    private JTable tblStudentDetails;
    private DefaultTableModel dtm;
    private JButton btnReload;

    ViewStudentDetailsForm() {
        setTitle("View Student Form");
        setSize(400, 300);
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);
        setLocationRelativeTo(null);

        JLabel lblTitle = new JLabel("View Student Details Form");
        lblTitle.setFont(new Font("", 1, 28));
        lblTitle.setHorizontalAlignment(JLabel.CENTER);
        add("North", lblTitle);

        JPanel buttonPanel = new JPanel();                    // default layout =
FlowLayout(CENTER)
        btnReload = new JButton("Reload");
        buttonPanel.add(btnReload);
        add("South", buttonPanel);

        String[] columnsName = {"Id", "Name", "PRF", "DBMS"};   // column names
        dtm = new DefaultTableModel(columnsName, 0);             // 0 = initial rows
        tblStudentDetails = new JTable(dtm);
        add("Center", new JScrollPane(tblStudentDetails));       // JScrollPane for scroll
bars
    }
}
```

To add a row (Q119): `Object[] rowData = {"S0001", "Niroth", 76, 45}; dtm.addRow(rowData);` — why `Object[]`? **Strings and ints are mixed** in one row — `Object[]` (Chapter 12) can hold any type.

17.4 Event Handling — making things happen when a button is clicked

📄 Lecture Note: Q120 – Q122

The buttons show now, but **nothing happens** when you click. For that we need **Event Handling**:

1. The click = an **Event**.
2. The code that responds to the event = an **ActionListener** (an Interface — Chapter 11!).
3. We **register** the listener with the button: `btn.addActionListener(listener)`.

`ActionListener` has **one method**: `void actionPerformed(ActionEvent evt)`. The lecture note (Q121) uses an **anonymous inner class** (a class with no name, created right there and used as an object):

```
import java.awt.event.*;

btnExit.addActionListener(new ActionListener() {          // an anonymous class that
implements the interface
    public void actionPerformed(ActionEvent evt) {        // this runs when the button is
clicked
        System.out.println("Exited....");
        System.exit(0);                                    // stops the program
    }
});
```

Remember — This is a use of Polymorphism / Interfaces

The parameter type of `addActionListener(ActionListener l)` is an **interface**. It accepts any class you give it (as long as it implements `ActionListener`). The button only needs to know that the `actionPerformed()` method exists. This is the very **Observer pattern** (Chapter 10)!

Q122: when the Reload button is pressed, fill the table with data:

```
btnReload.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent evt) {
        dtm.setRowCount(0);                                          // remove the old rows
        Student[] studentArray = /* get the data (Chapter 18) */;
        for (Student s : studentArray) {
            Object[] rowData = {s.getId(), s.getName(), s.getPrfMarks(), s.getDbmsMarks()};
            dtm.addRow(rowData);
        }
    }
});
```

Extra knowledge — Lambda expression (Java 8+)

Since `ActionListener` has only one method, the anonymous class can be shortened with a **lambda**:

```
btnExit.addActionListener(evt -> System.exit(0));
```

17.5 The Main Form — opening several forms

📄 Lecture Note: Q131 (StudentMainForm)

The buttons in `StudentMainForm` open the other forms (Add, Search, Update, View). Instead of **creating the form again and again**, create it once, and next time just `setVisible(true)`:

```
private AddStudentForm addStudentForm;                    // as a field (initially null)

btnAddStudentForm.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent evt) {
        if (addStudentForm == null) {                     // create it only if not created
yet
            addStudentForm = new AddStudentForm();
        }
        addStudentForm.setVisible(true);                  // show it
    }
});
```

Chapter 17 — Summary

- A GUI window = `class X extends JFrame`; components → private fields; **build** it in the constructor.
- `JTable` (View) + `DefaultTableModel` (Data); a row = `Object[]`; `dtm.addRow()`, `dtm.setRowCount(0)`.
- **Event handling** = `addActionListener(new ActionListener(){ actionPerformed(...) })`. Interface + anonymous class (or lambda).
- Instead of creating forms again and again, create them only once with a `null` check.

Try it yourself — Exercises

17.1 Using `extends JFrame`, build a "Counter" window: a JLabel + a "+" button; each click increases the value by 1.

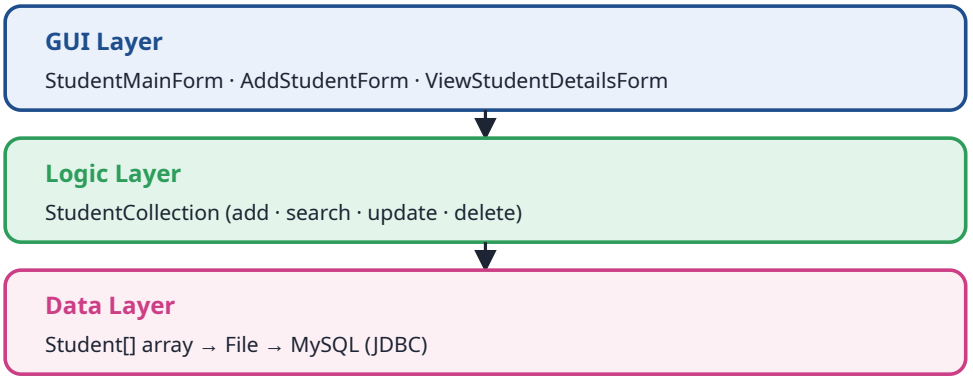
17.2 In `SearchStudentForm`, when the Search button is clicked, find the student by the id in the text field and fill the other fields (using `StudentCollection` from Chapter 18).

Chapter 18 — Coursework: Student Marks Management System

18.1 The project

📄 Lecture Note: Q123 – Q131 (Coursework — Student Marks management system)

A **complete application** that ties all the chapters together. The goal: manage students — **Add, Search, Update, Delete, View** — through a GUI.



Layered design: GUI → Logic → Data.

A **Layered Design** — each layer has **one responsibility**:

Layer	Classes	Responsibility
GUI	`StudentMainForm`, `AddStudentForm`, …	interaction with the user
Logic	`StudentCollection`	add, search, update, delete — data management
Data	`Student` (model), array → file → DB	storing the data

> **Remember — Do not store data in the GUI layer**
>
> If each form keeps a `Student[]` array, 5 forms = 5 arrays = **inconsistent data**. **One** `StudentCollection` must be shared by all forms. The technique for doing that is **Singleton**.

18.2 `Student` and `StudentCollection` (Q123 – Q126)

`Student` = the **Model class** (data and getters/setters). `StudentCollection` = holds the `Student[]` array inside, plus the operations.

```
class Student {
    private String id, name;
    private int prfMarks, dbmsMarks;

    public Student(String id, String name, int prfMarks, int dbmsMarks) {
        this.id = id; this.name = name; this.prfMarks = prfMarks; this.dbmsMarks =
dbmsMarks;
    }
    public String toString()  { return id + "\t" + name + "\t" + prfMarks + "\t" +
dbmsMarks; }
    public String getId()     { return id; }
    public String getName()   { return name; }
    public int getPrfMarks()  { return prfMarks; }
    public int getDbmsMarks() { return dbmsMarks; }
    public void setName(String name)        { this.name = name; }
    public void setPrfMarks(int prfMarks)   { this.prfMarks = prfMarks; }
    public void setDbmsMarks(int dbmsMarks) { this.dbmsMarks = dbmsMarks; }
}

class StudentCollection {
    private Student[] studentArray = new Student[]{
        new Student("S0001", "Nimal", 76, 55),
        new Student("S0002", "Bimal", 96, 75),
        new Student("S0003", "Amal", 26, 95),
    };

    public boolean addNewStudent(Student student) {
        extendsStudentArray();
        studentArray[studentArray.length - 1] = student;
        return true;
    }

    public Student[] getAllStudent() {                // a COPY of the array -
encapsulation (Chapter 3)
        Student[] temp = new Student[studentArray.length];
        for (int i = 0; i < studentArray.length; i++) temp[i] = studentArray[i];
        return temp;
    }

    public Student searchStudent(String id) {         // the Student if found; otherwise
null
        for (Student s : studentArray) {
            if (s.getId().equalsIgnoreCase(id)) return s;
        }
        return null;
    }

    public int search(Student student) {              // the index (-1 if not found)
        for (int i = 0; i < studentArray.length; i++) {
```

```
            if (studentArray[i].getId().equalsIgnoreCase(student.getId())) return i;
        }
        return -1;
    }

    public boolean updateStudent(Student student) {
        int index = search(student);
        if (index == -1) return false;
        studentArray[index] = student;
        return true;
    }

    public boolean deleteStudent(String id) {
        Student s = searchStudent(id);
        if (s == null) return false;
        Student[] temp = new Student[studentArray.length - 1];
        int j = 0;
        for (Student x : studentArray) if (x != s) temp[j++] = x;
        studentArray = temp;
        return true;
    }

    private void extendsStudentArray() {
        Student[] temp = new Student[studentArray.length + 1];
        for (int i = 0; i < studentArray.length; i++) temp[i] = studentArray[i];
        studentArray = temp;
    }
}

class Demo {
    public static void main(String[] args) {
        StudentCollection sc = new StudentCollection();
        sc.addNewStudent(new Student("S0004", "Ranil", 86, 55));

        System.out.println("--- All ---");
        for (Student s : sc.getAllStudent()) System.out.println(s);

        System.out.println("Search S0002 : " + sc.searchStudent("S0002"));
        System.out.println("Search S9999 : " + sc.searchStudent("S9999"));

        sc.updateStudent(new Student("S0001", "Nimal", 99, 99));
        System.out.println("Deleted S0003 : " + sc.deleteStudent("S0003"));
        System.out.println("--- After update/delete ---");
        for (Student s : sc.getAllStudent()) System.out.println(s);
    }
}
```

```
OUTPUT
--- All ---
S0001   Nimal   76      55
S0002   Bimal   96      75
S0003   Amal    26      95
S0004   Ranil   86      55
Search S0002 : S0002    Bimal   96      75
Search S9999 : null
Deleted S0003 : true
--- After update/delete ---
S0001   Nimal   99      99
S0002   Bimal   96      75
S0004   Ranil   86      55
```

18.3 The Singleton Design Pattern

📄 Lecture Note: Q129 – Q131

The problem: `AddStudentForm` does `new StudentCollection()`; `ViewStudentDetailsForm` also does `new StudentCollection()`. Then there are **two different collections** — a student added in Add does not appear in View!

The solution: there must be **only one** `StudentCollection` **object** for the whole application, and every form must share it. A design that lets you create **only a single object** is called the **Singleton Pattern**.

> **Definition — Singleton Pattern**
>
> A design pattern that allows **only one object** of a class to be created, and provides **a global access point** (`getInstance()`) to it.

The method (3 steps):

1. **Make the constructor** `private` → `new` is impossible from outside.
2. A `private static` field (to hold that single object) — why static? Chapter 5: one per class.
3. `public static getInstance()` — creates the object the first time, and returns **the same object** on later calls.

```
class A {
    private static A ob;                    // the single object (initially null)

    private A() { }                         // private constructor

    public static A getInstance() {
        if (ob == null) {                   // created only the first time
            ob = new A();
```

```
        }
        return ob;
    }
}

class Demo {
    public static void main(String[] args) {
        // A a1 = new A();                  // ERROR — constructor private
        A a1 = A.getInstance();
        A a2 = A.getInstance();
        System.out.println(a1 == a2);       // true - the same object
    }
}
```

OUTPUT

```
true
```

Singleton + StudentCollection (Q130)

The Singleton class **holds the** `StudentCollection` and hands it out through a getter:

```
class StudentCollection {
    private int count = 0;
    public void add() { count++; }
    public int getCount() { return count; }
}

class DataBase {                                        // class A in the lecture note
    private static DataBase ob;
    private StudentCollection studentCollection;

    private DataBase() {
        studentCollection = new StudentCollection();       // the collection is created only
once
    }
    public static DataBase getInstance() {
        if (ob == null) ob = new DataBase();
        return ob;
    }
    public StudentCollection getStudentCollection() { return studentCollection; }
}

class Demo {
    public static void main(String[] args) {
        // "Add Form":
        DataBase.getInstance().getStudentCollection().add();
        // "View Form" (somewhere else):
        DataBase.getInstance().getStudentCollection().add();
        System.out.println("Students : " +
DataBase.getInstance().getStudentCollection().getCount());
    }
}
```

```
OUTPUT
Students : 2
```

In the forms, everyone now uses `DataBase.getInstance().getStudentCollection()` instead of `new StudentCollection()` — **the same collection for all of them**.

```
// AddStudentForm.btnAdd click:
Student s = new Student(txtId.getText(), txtName.getText(),
                        Integer.parseInt(txtPrfMarks.getText()),
                        Integer.parseInt(txtDbmsMarks.getText()));
DataBase.getInstance().getStudentCollection().addNewStudent(s);
```

Extra knowledge — More uses of Singleton

- The **Database Connection** (Chapter 15) — sharing one connection.
- **Logger**, **Configuration settings**.
- In multi-threaded environments, `getInstance()` needs `synchronized` to be **thread-safe** (advanced).

18.4 Steps to build the application (Roadmap)

Step	What to do
1	The `Student` class (model)
2	`StudentCollection` (array → can now be moved to a file or DB, Chapters 14/15)
3	Singleton (`DataBase`)
4	`StudentMainForm` + Add / Search / Update / Delete / View forms
5	Calling `StudentCollection` methods from the forms' `actionPerformed`
6	Validation (marks 0-100, id format), error messages (`JOptionPane.showMessageDialog`)

Remember — The magic of layered design

Even if `StudentCollection` is **moved from an array to a file/DB, the Forms (GUI) do not need to change**, because they only know about methods like `addNewStudent()` and `searchStudent()` (Encapsulation + Abstraction).

Chapter 18 — Summary

- **Layered design**: GUI → Logic (`StudentCollection`) → Data.

- `getAllStudent()` returns a **copy** of the array (protecting the original).
- **Singleton**: private constructor + `private static` instance + `public static getInstance()`.
- Singleton to share the same data among forms.

Try it yourself — Exercises

18.1 Verify the Demo above including `deleteStudent`; check that `updateStudent` returns false for a wrong id.

18.2 Add a `getTopStudent()` method (the highest PRF+DBMS total) to `StudentCollection`.

18.3 Write a Singleton for a `Logger` class with a `log(String msg)` method.

Part Four — Advanced and Exam Preparation

Chapter 19 — Advanced OOP Topics EXTRA

Extra knowledge — Beyond the lecture note

These are Advanced topics for **understanding OOP well**, and needed for real projects and interviews.

19.1 Composition (HAS-A) vs Inheritance (IS-A)

Inheritance = **IS-A** (a Car IS-A Vehicle). **Composition** = **HAS-A**: an object **holding other objects inside itself** (a Car HAS-A Engine). You have already used this — `ControlRoom` HAS-A `WaterLevelObserver[]`, `DataBase` HAS-A `StudentCollection`, `StudentCollection` HAS-A `Student[]`.

```
class Engine {
    private int hp;
    Engine(int hp) { this.hp = hp; }
    void start() { System.out.println("Engine (" + hp + "hp) started"); }
}

class Car {
    private Engine engine;                      // HAS-A
    private String model;
    Car(String model, Engine engine) { this.model = model; this.engine = engine; }
    void drive() {
        engine.start();                          // delegates the work
        System.out.println(model + " is moving");
    }
}

class Demo {
    public static void main(String[] args) {
        new Car("Toyota", new Engine(120)).drive();
    }
}
```

OUTPUT

```
Engine (120hp) started
Toyota is moving
```

Remember — Which to choose?

If you can **truthfully** say "A **IS-A** B", use Inheritance; otherwise use **Composition**. The rule: *"Favor composition over inheritance"* — Composition is more **flexible** (parts can be swapped at runtime) and has less tight coupling (loose coupling). Java's old API where Stack was built as a Vector (IS-A) is an example: "Stack IS-A Vector" turned out to be a poor design.

19.2 `instanceof` and Casting

To call a subclass-specific method through a Superclass reference you must **cast** (the a1.printB() problem in 9.5):

```
class Animal { void eat() { System.out.println("eating"); } }
class Dog extends Animal { void bark() { System.out.println("Woof"); } }
class Cat extends Animal { }

class Demo {
    public static void main(String[] args) {
        Animal a = new Dog();               // Upcasting - automatic and safe
        a.eat();
        // a.bark();                        // ERROR - not possible through an Animal
reference

        if (a instanceof Dog) {             // is the object a Dog at runtime?
            Dog d = (Dog) a;                // Downcasting - done carefully
            d.bark();
        }

        Animal c = new Cat();
        try {
            Dog bad = (Dog) c;              // a Cat cannot become a Dog
        } catch (ClassCastException ex) {
            System.out.println("ClassCastException : Cat is not a Dog");
        }
    }
}
```

OUTPUT

```
eating
Woof
ClassCastException : Cat is not a Dog
```

	Upcasting	Downcasting
Direction	Sub → Super	Super → Sub
Syntax	automatic	`(SubType) ref` (explicit)
Safety	always safe	risk of **ClassCastException** → check with `instanceof`

19.3 Overriding `equals()` and `hashCode()`

In Chapter 12 the default `equals()` compares **references**. Often we want to compare the **contents** (is the id equal):

```
import java.util.*;

class Student {
    private String id, name;
    Student(String id, String name) { this.id = id; this.name = name; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;                          // the same object
        if (o == null || getClass() != o.getClass()) return false;
        Student other = (Student) o;
        return id.equals(other.id);                          // compare by id
    }
    @Override
    public int hashCode() { return id.hashCode(); }          // must be consistent with
equals
}

class Demo {
    public static void main(String[] args) {
        Student s1 = new Student("S0001", "Nimal");
        Student s2 = new Student("S0001", "Nimal K.");
        System.out.println(s1.equals(s2));            // true - ids are equal
        System.out.println(s1 == s2);                 // false - different objects

        Set<Student> set = new HashSet<>();
        set.add(s1); set.add(s2);
        System.out.println(set.size());               // 1 - the duplicate was removed
    }
}
```

OUTPUT

```
true
false
1
```

Watch out — Rule: if you override `equals()`, override `hashCode()` too

If `a.equals(b)` is true then `a.hashCode() == b.hashCode()` **must** hold. Otherwise collections like `HashSet` and `HashMap` behave wrongly.

19.4 The `final` keyword

Applied to	Effect
`final` variable	the value can be set **only once** (a constant)
`final` method	a Subclass **cannot override** it
`final` class	it **cannot be inherited** (`String` , `Integer` are final)

```
final class A { }
class B extends A { }                    // ERROR - cannot inherit a final class

class C { final void m() { } }
class D extends C { void m() { } }       // ERROR - cannot override a final method
class Demo { public static void main(String[] a) { final int x = 5; x = 6; } }  // ERROR
```

```
COMPILE ERROR
Demo.java:2: error: cannot inherit from final A
class B extends A { }                    // ERROR - cannot inherit a final class
                ^
Demo.java:5: error: m() in D cannot override m() in C
class D extends C { void m() { } }       // ERROR - cannot override a final method
                        ^
  overridden method is final
```

19.5 Generics and `ArrayList` (the Java Collections Framework)

The problem of the `Object[]` collection in Chapter 12, **no type safety**, is solved with **Generics**: `ArrayList<Student>` = "Student objects only". `ArrayList` = the **professional version** of the `Collection` you wrote:

```
import java.util.*;

class Student {
    private String name; private int marks;
    Student(String name, int marks) { this.name = name; this.marks = marks; }
    String getName() { return name; }
    int getMarks() { return marks; }
    public String toString() { return name + "(" + marks + ")"; }
}

class Demo {
    public static void main(String[] args) {
        ArrayList<Student> list = new ArrayList<>();       // <Student> = generic type
        list.add(new Student("Nimal", 56));
        list.add(new Student("Amal", 76));
        list.add(new Student("Bimal", 36));
        // list.add("abc");                                // ERROR - not a Student (type
safety)
```

```
        Student s = list.get(0);                          // no cast needed
        System.out.println(s + "   size=" + list.size());

        list.remove(1);
        for (Student x : list) System.out.println(x);

        list.sort((a, b) -> b.getMarks() - a.getMarks());    // marks descending (lambda)
        System.out.println(list);
    }
}
```

OUTPUT

```
Nimal(56)   size=3
Nimal(56)
Bimal(36)
[Nimal(56), Bimal(36)]
```

Collection	Characteristic
`ArrayList<T>`	has **order**, access by index, duplicates ✔
`HashSet<T>`	**no duplicates** (equals/hashCode), no order
`HashMap<K,V>`	**Key → Value** pairs (id → Student)

```
import java.util.*;
class Demo {
    public static void main(String[] args) {
        HashMap<String, Integer> marks = new HashMap<>();
        marks.put("Nimal", 56);
        marks.put("Amal", 76);
        System.out.println(marks.get("Amal"));
        System.out.println(marks.containsKey("Bimal"));
    }
}
```

OUTPUT

```
76
false
```

19.6 `enum`

An **enum** = **a fixed set of values** (days of the week, months, statuses). It is more **type-safe** than the constants `YEAR=0, MONTH=1, DAY=2` in Chapter 6:

```
enum Field { YEAR, MONTH, DAY }

class Demo {
    static void set(Field f, int value) {
        switch (f) {
```

```
            case YEAR:  System.out.println("Year = " + value);  break;
            case MONTH: System.out.println("Month = " + value); break;
            case DAY:   System.out.println("Day = " + value);
        }
    }
    public static void main(String[] args) {
        set(Field.YEAR, 2026);
        // set(5, 10);              // ERROR - an int is not a Field type
        for (Field f : Field.values()) System.out.println(f);
    }
}
```

OUTPUT

```
Year = 2026
YEAR
MONTH
DAY
```

19.7 Packages and Imports

A **Package** = a folder of related classes (`java.util` , `javax.swing`). Benefits: avoiding name clashes, access control (**default / protected** access works at the package level), organization.

```
package com.icet.sms;                // first line of the file

import java.util.ArrayList;          // import a single class
import javax.swing.*;                // everything in the package
```

`java.lang` (String, Math, Object…) is **imported automatically**.

19.8 OOP Design Principles — SOLID (introduction)

Letter	Principle	Idea	Where we saw it
S	Single Responsibility	A class has **one responsibility**	`Student` (data) vs `StudentCollection` (management)
O	Open/Closed	open for extension, closed for modification	Observer — the new Splitter
L	Liskov Substitution	a Subclass can be **substituted** for its Superclass	`Vehicle v = new Car()`
I	Interface Segregation	several small interfaces instead of one big one	`Flyable` , `Swimmable`
D	Dependency Inversion	depend on **abstractions** (interfaces)	`ControlRoom` → `WaterLevelObserver`

19.9 UML Class Diagram — how to draw a class

```
+----------------------+
|       Student        |         <-  Class name
+----------------------+
| - id : String        |         ←  Fields  ( - = private , + = public , # = protected )
| - name : String      |
| - prfMarks : int     |
+----------------------+
| + getId() : String   |         ←  Methods
| + setMarks(int, int) |
| + toString() : String|
+----------------------+
        ^   (Inheritance - hollow triangle arrow)
        |
+----------------------+
|   PostgradStudent    |
+----------------------+
```

Relationship	UML	Java
Inheritance	▷ (hollow triangle)	`extends`
Interface implementation	⇢▷ (dotted line)	`implements`
Composition / Association	◆ / →	a field

Chapter 19 — Summary

- **Composition (HAS-A)** is often better than Inheritance.
- **Upcast** is automatic; **downcast** with an `instanceof` check.
- Override `equals()` + `hashCode()` together.
- `final` → a variable/method/class that cannot be changed.
- **Generics** (`ArrayList<T>`) → type-safe collections.
- **SOLID** — the principles of good OOP design.

Try it yourself — Exercises

19.1 A `Library` class holds an `ArrayList<Book>` inside and has `addBook` , `findByTitle` (Composition).

19.2 Override equals/hashCode in `Student` , and show that when two students with the same id are put into a HashSet only 1 remains.

19.3 A `Shape` interface + `Circle` , `Rectangle` ; calculate the total area through an `ArrayList<Shape>` .

Chapter 20 — Tricky Questions

The lecture note has many questions of the type "**Legal ?**", "**Illegal ?**", "**What is the output?**". Here are the most important questions of that kind. **First guess the answer, then look at the Output.** Every code block was actually compiled / run to get the output.

T1 — An uninitialized reference (Q170)

```
class A { int a; }
class Demo {
    public static void main(String[] args) {
        A a1;                   // only a reference variable, no object, no value either
        a1.a = 100;
    }
}
```

COMPILE ERROR

```
Demo.java:5: error: variable a1 might not have been initialized
        a1.a = 100;
        ^
1 error
```

Answer: You cannot use a local variable **without initializing it** — a compile error. (If you write `A a1 = null;` it compiles, but you get a NullPointerException at runtime.) **Fields** have default values; **local variables** do not!

T2 — A static counter (based on Q38 – Q40)

```
class A {
    static int count;
    int id;
    A() { id = ++count; }
}
class Demo {
    public static void main(String[] args) {
        new A();
        new A();
        A a3 = new A();
        System.out.println(a3.id + " " + A.count);
    }
}
```

OUTPUT

```
3 3
```

Answer: `id` is one per object; `count` is one per class. So `3 3`.

T3 — The constructor chain (this + super)

```
class A {
    A()      { this(1); System.out.println("A()"); }
    A(int i) { System.out.println("A(int)"); }
}
class B extends A {
    B()      { System.out.println("B()"); }
    B(int i) { this(); System.out.println("B(int)"); }
}
class Demo {
    public static void main(String[] args) {
        new B(5);
    }
}
```

OUTPUT

```
A(int)
A()
B()
B(int)
```

Step by step: `new B(5)` → `B(int)` → the first statement `this()` → `B()` → (the first statement `super()` is automatic) → `A()` → `this(1)` → `A(int)` prints → back to `A()` prints → `B()` prints → `B(int)` prints. **The constructor called last in the chain finishes first.**

T4 — Overload resolution (Compile-time Polymorphism)

```
class Demo {
    static void m(long x)    { System.out.println("long"); }
    static void m(Integer x) { System.out.println("Integer"); }
    static void m(Object x)  { System.out.println("Object"); }
    public static void main(String[] args) {
        m(5);
    }
}
```

OUTPUT

```
long
```

Answer: An `int` argument — Java first tries **widening** (`int → long`); that takes priority over boxing (`Integer`). **Order:** exact → widening → boxing → varargs.

T4b — Recalling Q187: Overloading or Overriding?

When you write `setSize(short, short)` in `Calculator extends JFrame`, and then call `setSize(300, 100)` (int arguments), `JFrame`'s `setSize(int,int)` matches. The `short` version is only an **overload** — not overriding! For **overriding**, the signature (the **parameter types**) must be **exactly the same**.

T5 — String `==`

```
class Demo {
    public static void main(String[] args) {
        String s1 = "Hello";
        String s2 = "Hel" + "lo";            // compile-time constant -> the same object in
the pool
        String h = "Hel";
        String s3 = h + "lo";                // created at run time -> a new object
        System.out.println(s1 == s2);
        System.out.println(s1 == s3);
        System.out.println(s1.equals(s3));
    }
}
```

OUTPUT

```
true
false
true
```

T6 — The `Integer` cache trick

```
class Demo {
    public static void main(String[] args) {
        Integer a = 127, b = 127;
        Integer c = 128, d = 128;
        System.out.println(a == b);
        System.out.println(c == d);
        System.out.println(c.equals(d));
    }
}
```

OUTPUT

```
true
false
true
```

Answer: Java **caches** the Integer objects from `-128 … 127` (the same object). 128 creates new objects. **To compare objects use `equals()`!**

T7 — A method that cannot swap (Pass by value)

```
class Box { int v; Box(int v) { this.v = v; } }
class Demo {
    static void swap(Box a, Box b) {
        Box t = a;  a = b;  b = t;          // only the local references are swapped
    }
    public static void main(String[] args) {
        Box x = new Box(1), y = new Box(2);
        swap(x, y);
        System.out.println(x.v + " " + y.v);
    }
}
```

OUTPUT

```
1 2
```

Answer: `1 2` — Java is **pass by value**: a **copy** of the reference goes. Inside the method the copies were swapped; `x` and `y` did not change. (If `a.v` had been changed, the outside would see it.)

T8 — Static methods are not overridden (Hiding)

```
class P {
    static void s() { System.out.println("P.s"); }
    void i()        { System.out.println("P.i"); }
}
class C extends P {
    static void s() { System.out.println("C.s"); }
    void i()        { System.out.println("C.i"); }
}
class Demo {
    public static void main(String[] args) {
        P p = new C();
        p.s();         // static → reference type (P)
        p.i();         // instance → object type (C)
    }
}
```

OUTPUT

```
P.s
C.i
```

The rule: Instance methods → the object type (Run-time). **Static methods and fields** → the reference type (Compile-time).

T9 — Illegal reference assignment (Q181)

```java
class A { }
class B extends A { }
class Demo {
    public static void main(String[] args) {
        A a1 = new B();      // OK  (B IS-A A)
        B b1 = new A();      // ERROR — A IS-NOT-A B
    }
}
```

COMPILE ERROR

```
Demo.java:6: error: incompatible types: A cannot be converted to B
        B b1 = new A();      // ERROR — A IS-NOT-A B
               ^
1 error
```

The answer: `B b1 = (B) a1;` (a cast) **compiles** (and runs, because a1 really is a B object). But casting a `new A()` to B throws a **ClassCastException**.

T10 — Static context (Q41)

```java
class Demo {
    void f() { System.out.println("f"); }
    public static void main(String[] args) {
        f();                 // an instance method from static main without an object
    }
}
```

COMPILE ERROR

```
Demo.java:4: error: non-static method f() cannot be referenced from a static context
        f();                 // an instance method from static main without an object
        ^
1 error
```

The fix: create an object and call it as `new Demo().f();`.

T11 — Instance block + constructor + field initializer (Q36 – Q37)

```java
class A {
    int x = init("x");
    { System.out.println("block"); }
    A() { System.out.println("A()"); }
    int y = init("y");
    int init(String s) { System.out.println("init " + s); return 1; }
}
class Demo {
    public static void main(String[] args) { new A(); }
}
```

OUTPUT

```
init x
block
init y
A()
```

Answer: Field initializers and instance blocks run **in the order written** (x → block → y), **then** the constructor.

T12 — An overridden method from a constructor (Q189 — recalled)

```java
class A {
    A() { show(); }
    void show() { System.out.println("A.show"); }
}
class B extends A {
    int v = 5;
    void show() { System.out.println("B.show v=" + v); }
}
class Demo {
    public static void main(String[] args) { new B(); }
}
```

OUTPUT

```
B.show v=0
```

Answer: `B.show v=0` ! B's method ran (dynamic dispatch), but **B's fields are not yet initialized** (`v` = the default 0; `v = 5` happens only after A() finishes). **Do not call overridable methods from a constructor.**

T13 — Quick True / False

#	Statement	Answer
1	A constructor can have a return type	✘
2	An `abstract` class can have a constructor	✔
3	A class can `extends` 2 classes	✘
4	A class can `implements` 2 interfaces	✔
5	A `private` method can be overridden	✘
6	`this` can be used from a static method	✘
7	`String` is mutable	✘ (immutable)
8	`super()` and `this()` can both be written in the same constructor	✘
9	You can reach a field through a `null` reference without `new`	✘ (NPE)
10	Every class inherits from `Object`	✔

Chapter 21 — Cheat Sheet (Quick Summary)

21.1 OOP concepts at a glance

Concept	Simple meaning	Java
Class	The template of objects	`class Student { }`
Object	A real instance of a class	`new Student()`
Encapsulation	Data private, access via getters/setters	`private int x;`
Inheritance	Inherit from a parent (IS-A)	`class B extends A`
Polymorphism	The same call, different actions	Overload / Override
Abstraction	Hide the unnecessary, show the essential	`abstract` , `interface`

21.2 Keywords

Keyword	Meaning
`this`	The current object; `this.x` , `this(args)` (constructor call)
`super`	The parent: `super.m()` , `super(args)`
`static`	Belongs to the class (no object needed)
`final`	Cannot be changed / overridden / inherited
`abstract`	Incomplete (no new / no body)
`extends` / `implements`	inherit a class / implement an interface
`new`	Creates an object in the Heap
`null`	A reference that points to no object
`instanceof`	A run-time type check
`throw` / `throws`	Throw an exception / declare it

21.3 The Top 12 rules (for exams)

1. An object is created with `new`; field defaults: `0`, `false`, `null`.
2. If **you write** a constructor, the compiler **does not give** the default no-arg one.
3. `this(...)` / `super(...)` must be the **first statement**; you cannot use both together.
4. Subclass constructor → the **Superclass constructor first** (`super()` is automatic).
5. Init order: default → field initializer & instance block → constructor.
6. Class load: Super static → Sub static; a static block runs once.
7. `A a = new B()` ✔; `B b = new A()` ✘. Access = **reference type**; overridden method = **object type**.
8. **Methods** → run-time (object type); **fields & static methods** → compile-time (reference type).
9. Overriding: same signature; access cannot be reduced; `static/final/private` are not overridden.
10. A static method cannot use instance members / `this`.
11. Java is **pass by value** (objects → a copy of the reference).
12. Strings are immutable; `equals()` for content, `==` for references.

21.4 Access modifiers

	Same class	Same package	Subclass	Anywhere
`private`	✔	✘	✘	✘
default	✔	✔	✘	✘
`protected`	✔	✔	✔	✘
`public`	✔	✔	✔	✔

21.5 Overloading vs Overriding

	Overloading	Overriding
Parameters	different	same
Classes	the same class	Sub & Super
Binding	Compile-time	Run-time

21.6 Abstract class vs Interface

	Abstract class	Interface
Inherit	one `extends`	many `implements`
State (fields)	✔	constants only
Constructor	✔	✘
Relationship	IS-A	CAN-DO

21.7 Design Patterns (seen in this guide)

Pattern	Idea	Where
Singleton	Only one object of a class	Chapter 18 (`DataBase`)
Observer	A subject notifies its observers	ControlRoom (Ch.10), ActionListener (Ch.17)
Static Factory	A static method that creates objects	`Date.getDateInstance()` (Ch.6)
Immutable object	An object that cannot be changed	`String` , `Date` (Ch.7)

21.8 Good coding habits

- Class names **PascalCase** (`StudentCollection`), methods/variables **camelCase** (`getTotal`), constants **UPPER_CASE** (`PI`).
- Fields `private` ; methods `public` (only if needed).
- One method does **one job**; remove duplication (`this(...)` , helper methods).
- `@Override` always; a `toString()` in every model class.
- Compare Strings with `equals()` ; `close()` resources (or try-with-resources).
- Do not call overridable methods from a constructor.

Appendix — Answers to the Exercises (short)

> **Note**
>
> These are **short answers / hints**. Always write the code and run it yourself — before looking at the answer!

Exercise	Answer / hint
1.2	`Amal` — `s2 = s1` copies the reference; it is the same object.
2.2	`double getArea(){ return Math.PI*radius*radius; }` , `getCircumference(){ return 2*Math.PI*radius; }`
3.1	`deposit` : `if(amt>0) balance+=amt;` `withdraw` : `if(amt<=balance) balance-=amt;`
3.3	`error: length has private access in Box` — a private field cannot be accessed from outside the class.
4.2	`block` , `f` , `P` — the instance block → the field initializer (`x=f()`) in the order written, then the constructor.
5.3	`static void m(){ System.out.println(y); }` — `non-static variable y cannot be referenced from a static context` .
7.1	`" Hello Java ".trim().toUpperCase().substring(6)` → `JAVA`
7.3	`for (String n : "Nimal,Amal,Bimal".split(",")) System.out.println(n);`
8.2	`for (Student s : arr) if (s.getId().equals(id)) return s; return null;`
9.3	Static blocks: Demo → A → B → C (Demo, main starts, A, B, C).
10.1	`double total=0; for (Shape s : shapes) total += s.area();`
11.3	`private` / `final` methods cannot be overridden; an abstract method must be overridden by the subclass → a conflict.
12.1	`contains(int d){ for (int x : dataArray) if (x==d) return true; return false; }`
13.3	Compile error: `exception ArithmeticException has already been caught` — write the specific catch first.
19.2	`HashSet` size = 1 (equals/hashCode on id).

Remember — A final word

The way to learn OOP: **read → type → run → break it → build your own project.** Your own **Student Marks System** is excellent practice. Best wishes!